

A Comparative Analysis and Predictive Model for Reporting Depression in U.S. Adults

Kiran Mai Jaiswal Charpuria, Kruthika Gaddam, Mohith Surya Kiran Kasula, Bala Samantula, Sri Harsha Sudalagunta, April Taylor, Alan Varkey
[kichar, kgaddam, mkasula, bsamantu, ssudala, taylorad, alvarkey] @iu.edu

Indiana University-Purdue University, Indianapolis, USA

Abstract. A comparative analysis and predictive model for the reporting of depression in adults was performed. Discrepancies were investigated between self-reporting and healthcare professional reporting of depression. The findings were then used to predict specific groups who are more likely to experience depression.

Keywords: Depression; Reporting; NHANES database; NAMCS database; Predictive model

1 Project Scope

1.1 Introduction

People with mental health disorders are at an increased risk of social, educational, racial, and physical difficulty[1]. The incidence of depression globally in 2019 is 280 million, and the World Health Organization reports an estimated 28% increase in 2020 due to the Covid pandemic[2]. The American Academy of Pediatrics, American Academy of Child and Adolescent Psychiatry, and Children's Hospital Association declared a national mental health emergency for children and adolescents in the United States[3].

Despite the rise in depression and the effects of the disorder across all age groups, there remains a personal and medical stigma to a depression diagnosis[4]. Even mental healthcare providers have demonstrated stigma toward patients with mental health disorders[5]. Stigma, whether self-imposed or experienced, particularly affects the disclosure of mental health issues. Garcia et al.[6] demonstrated that standardized screening increased the diagnosis of depression in disparaged groups. Appropriate screening in an emotionally safe healthcare setting is a critical first step to addressing the global issue of depression

The focus of this project was the screening and diagnosis of depression in adults. Due to access limitations to depression data, minors were excluded. The object of this comparison was to analyze the reporting behaviors of the adult population and the diagnosis behaviors of healthcare providers regarding depression and then to create a model for predicting depression reporting.

1.2 Aims

To evaluate any significant differences between self-reported and provider-reported cases of depression in adults in the U.S.

To identify and correlate the main factors impacting the reporting of depression by self-reporters and providers in adults in the U.S. and identify the groups that report depression the most.

To develop a multi-variant model that predicts the diagnosis of depression cases from the features of age, gender, income, co-morbidity, medical insurance, education level, and race.

1.3 Purpose

This project investigated discrepancies between self-reporting and healthcare professional reporting of depression, and the findings were used to predict specific groups, for example, a specific age range, who were more likely to experience depression. The results can be used to improve screening standards for healthcare professionals to increase the diagnosis of depression in groups predicted to have depression.

1.4 Hypotheses:

Null Hypothesis. There will be no increase in the reporting of depression by adults as compared to the reporting of major depressive disorder by healthcare providers in medical records in the US.

Alternate Hypothesis. There will be an increase in reporting depression by adults compared to major depressive disorder by healthcare providers in medical records in the US.

2 Methodology

This project was a comparative analysis between adults and healthcare providers. The project used two datasets to compare depression indicators from the NHANES dataset with depression indicators from the NAMCS dataset. Once the data was compared, machine learning models were developed to predict the factors impacting the depression indicators in 1) adults and 2) healthcare providers. Tools for the project included Python Jupyter notebook, phpMyAdmin, Google Colab, and Microsoft Teams.

Stages of Project:

1. Data Collection and Extraction
2. Data Cleaning and Storage
3. Data Analysis
 - a. Exploratory Data Analysis
 - b. Prediction Modeling
4. Data Visualization

2.5 Team members Contributions

Everyone was incredibly supportive of our team, as everyone participated in the project actively. We met every Friday at 11 A.M., and for those who couldn't make it, we made it a habit to update notes on the canvas to keep track of our project and receive feedback from our professors and TAs so they could monitor our progress. We scheduled meetings even when there were no submissions to make it a habit to be in touch and obtain a corporate feel, which was a positive aspect of this. Canvas was utilized efficiently for discussions to conclude while considering everyone's viewpoints. In addition to the lecturers' lecture materials and videos, we exchanged YouTube videos in our group as we came from different backgrounds and had varying levels of knowledge about Python, ML, GitHub, and colab. Microsoft teams were effectively used to upload files and record meetings. We utilized Wrike, a project management platform, to keep track of activities because we were divided into sub-teams and had weekly responsibilities to complete.

Team Members							
Tasks	April Taylor	Kruthika Gaddam	Bala Samantula	Sri Harsha Sudalagunta	Mohith Surya Kiran Kasula	Kiran Mai Jaiswal Charpuria	Alan Varkey
Project Management							
Background/Research							
Proposal Development							
Editing/Proofreading							
Data Collection							
Data Analysis							
Hypothesis Testing							
Project Presentation							
Report Development							
Data Cleaning							
Model Development and Testing							
Data Visualization							

Data Description

Data from two existing publicly available datasets was utilized for this project. Both datasets consisted of secondary data collected by the National Center for Health Statistics (NCHS) and contained data from surveys and physical examinations of persons in the United States (U.S.) from 2017 to 2018[7]. The project used 2018 survey data for both datasets because more recent full datasets were limited due to restrictions to in-person contact during the Covid pandemic. The project team also considered the possible compounding effect the Covid pandemic may have on depression rates and decided to assess the pre-pandemic state of depression.

NHANES. The NCHS routinely conducts the National Health and Nutrition Examination Survey (NHANES) to gather information on the health of the general U.S. population. Nutrition, disease, health behaviors and demographic information are among the data collected from participants through a “mobile examination center” [8]. The 2018 source dataset included data from 9,254 persons[9].

NAMCS. The NCHS also routinely conducts the National Ambulatory Medical Care Survey to gather information on the direct patient care received in ambulatory care centers. Examples of available data from this dataset include patient diagnosis codes, insurance coverage, provider types and practice descriptors[10]. The 2018 source dataset included data from 9,953 patient record forms submitted by 496 physicians[11].

3 Data Collection and Extraction

Collection. The NHANES dataset is sectioned into multiple subset data files based on topic or survey instrument. The subset datasets are available from the NCHS website as XPORT files (xpt) along with codebooks for interpretation[8]. The subsets which included variables pertaining to this project were chosen. The NAMCS data is available on a NCHS website in a SAS file (sas7bdat) in a zip file. The NCHS provides a micro-file data codebook for interpretation[12]. The source files (Figure 1) were downloaded and stored in a shared repository (Teams) and the class repository (Canvas). The source files were not stored directly in MySQL due to error messages when attempting to upload the files in their original state.

The files were uploaded into Jupyter, for processing with Python. The SAS and xpt files were converted to csv files using the Python Panda method. The 5 NHANES CSV files were merged into one CSV file using a Python Panda merge method with parameters for an outer merge. The SEQN column was used as a primary key in each set to allow for the retention of all rows and columns from all five datasets. Since not all surveys were conducted on all participants, the merging of the datasets led to multiple empty cells in the merged NHANES set. The empty cells were filled with ‘0’ to aid in the data analysis process.

Extraction. Both surveys include a broad amount of health data. This project did not analyze the full datasets. It focused on the data within each dataset that was anticipated to correlate with depression or depression reporting (Figure 1). The datasets were reduced with the Pandas method of selecting only the columns designated for this project. Medications were also excluded from both datasets, except for the Boolean variable of “medication for depression.” Medications are varied in their indication for use, and the datasets did not provide information on initiation or discontinuation dates for correlation with depression onset or improvements. Due to this complexity, the team chose not to include this variable in the project, despite the potential to indicate or predict depression.

PROJECT SOURCE FILES	
2018 NHANES	
Demographics	<u>DEMO J.XPT</u>
Disabilities	<u>DLQ J.XPT</u>
Depression Screener	<u>DPQ J.XPT</u>
Medical Conditions	<u>MCQ J.XPT</u>
Health Insurance	<u>HIQ J.XPT</u>
2018 NAMCS	
Office-based visits	<u>namcs2018 sas.sas7bdat</u>

Figure 1. NHANES & NAMCS Source files

Removing irrelevant data. The data was further reduced to only data relevant to the project aims. After merging the NHANES subsets, the datasets were cleaned of duplicates.

AGE. Age was limited to adults only in both sets. The NCHS restricts the release of depression scores for minors in the NHANES survey, therefore participate data for those under age 18 was removed for all variables. Age was limited in the NAMCS dataset with the selection of only the column, “AGE” which limited the participant’s data to those aged over 17.

Handling Missing Data.

NHANES. Per the NHCS [13] recommendation, data was evaluated for missing values and was prespecified to be imputed if more than 10% were missing using multiple imputation method (Figure 2). The following values in the set indicated no data was available:

NAMCS Missing Values(%)		NHANES Missing Values(%)			
Code	-9	Code	9/99	7/77	Blank
RACER	0%	INDHHIN2	0.1%	2.0%	0.0%
RACEUN	28.6%	HIQ031A	0.7%	0.1%	0.0%
PAYTYPER	2.5%	HIQ031B	0.0%	0.0%	0.0%
DIAG1	0.4%	HIQ031D	0.0%	0.0%	0.0%
DIAG2	34.2%	HIQ031E	0.0%	0.0%	0.0%
DIAG3	57.9%	HIQ031H	0.0%	0.0%	0.0%
DIAG4	73.1%	HIQ031I	0.0%	0.0%	0.0%
DIAG5	84.3%	HIQ031AA	0.0%	0.0%	0.0%
OWNSR	4.5%	DMDEDUC2	0.0%	0.2%	0.0%
AGE	0.0%	DMDEDUC3	0.0%	0.0%	0.0%
SEX	0.0%	DLQ140	0.0%	0.0%	1.6%
DEPRN	0.0%	DLQ150	0.1%	0.0%	1.6%
NOCHRON ¹	67.0%	DLQ170	0.2%	0.0%	48.0%
DEPRESS	0.0%	DPQ010	0.1%	0.7%	7.5%
MENTAL	0.0%	DPQ020	0.1%	0.0%	7.5%
PSYCHOTH	0.0%	DPQ030	0.1%	0.5%	7.5%
NOPROVID	0.0%	DPQ040	0.1%	0.1%	7.5%
PHYS	0.0%	DPQ050	0.1%	0.0%	7.5%
PHYSASST	0.0%	DPQ060	0.1%	0.1%	7.5%
NPNWM	0.0%	DPQ070	0.1%	0.0%	7.5%
RNLPN	0.0%	DPQ080	0.1%	0.0%	7.5%
MHP	0.0%	DPQ090	0.1%	0.0%	7.5%
OTHPROV	0.0%	RIDAGEYR	0.0%	0.0%	0.0%
PROVONE	0.0%	RIAGENDR	0.0%	0.0%	0.0%
SPECCAT	0.0%	RIDRETH3	0.0%	0.0%	0.0%
	¹ Blank	>10%			

Figure 2. NAMCS & NHANES Missing Data Evaluation

- missing = blank
- “don’t know” = 9/99
- refused = 7/77

For all variables in the reduced dataset, the above were considered in the missing data calculations. Not all survey questions were asked or appropriate for all participants. For those variables, blanks were converted from NaN to “0”.

PHQ-9 Questions. The blank data in the PHQ questions was reviewed in detail to ensure the blank values that were converted to “0” for the depression screening questions did not skew the PHQ total score to lower values or the overall rate of depression in NHANES. Each question for the PHQ had a 7.5% missing rate. This consisted of about 743 participants for which all questions were blank for the PHQ-9. This indicates the participants did not consent to the depression survey. The team chose not to drop these participants to preserve the other data obtained

for the other variables (including other depression indicators) as well as the missing data rate being less than the prespecified 10%.

As further discussed in section 3.a, the NHANES dataset depression indicator proportion was higher than the NAMCS total depression indicator proportion. The team did not feel the missing surveys responses significantly affected the outcome of the findings.

DLQ-170 How depressed did you feel? This question was the only variable with more than 10% missing within NHANES. The team chose not to impute this variable considering that a participant may feel uncomfortable or uncertain answering this question. Also, a positive was captured in the depression indicator. Imputing may falsely increase the dependent variable.

NAMCS. For most of the variables in the NAMCS, the data was imputed by NAMCS, therefore no adjustments were required. Diagnosis (including no chronic illness), payer type and practice type were not imputed.

Race. Initially, the variable for race, “RACEUN” was chosen, however, it was discovered that 29% of that variable was missing and was also not imputed. On further investigation, the “RACER” variable was an acceptable alternative. The “RACER” variable contained less categories within racial groups (black, white, other) which provides less insight into the depression of multiple races. However, the “RACER” variable is imputed by NCHS, and the team agreed having a complete dataset for this variable was more important for this project considering the risk of limited findings from missing values.

Diagnoses. Diagnosis codes, “DIAG” 1-5, were not imputed by NCHS. Due to the nature of U.S. office visits scheduling, most patients are seen for short appointments where only a primary diagnosis (DIAG1) is treated. Rarely are up to 5 diagnoses treated in one office visit (DIAG 1-5). Imputing a diagnosis would introduce inaccurate findings considering the 339 diagnosis categories and numerous subcategories in each diagnosis category. Missing percentages were calculated for these variables, but empty cells were not imputed. Positive findings for diagnoses were used for each participant in the data analyses and predictive models.

No Chronic Illness. A significant amount of missing data (67%), for “no chronic illness” was found, however this category would be difficult to impute accurately considering the wide range of chronic illnesses that could be imputed. As with the depression survey, the team chose not to drop these participants to retain the other data obtained from these surveys.

3.6 Data Cleaning and Storage

Feature Engineering. After cleaning, the datasets were prepared for data analysis by conducting feature engineering. Most of the variables were categorical and were transformed into Boolean (0/1) features. Several variables were combined or transformed by normalization or standardized across datasets for consistency and simplicity. The variables combined and/or renamed include insurance, education, age, gender, and race.

Education. The NHANES source dataset included two education variables; under age 20 and over age 20. Considering that our sample population is all adults over 17, the team created an EDUC feature that merged the two variables of education and consolidated the multiple primary education categories into one category. This allowed for a simplified 5 category education feature.

Insurance. Health insurance was a common variable between the NHANES and NAMCS datasets. However, the datasets differed in the categorization of the types of insurance. The insurance columns for each dataset were consolidated to 4 common U.S. insurance types: private, state-funded, Medicare, and no insurance.

Race. Race was also a common variable between the datasets. However, NAMCS categorized the imputed RACER variable into three categories while NHANES used 6. For standardization, NHANES categories were consolidated into black, white, and other and both datasets were renamed to be consistent features names.

Comorbid Conditions. Both datasets included medical diagnoses information. NHANES survey captured Boolean answers for chronic medical conditions. The team chose chronic conditions that were anticipated to correlated with a depression reporting. The NAMCS survey captured all diagnosis codes as described in the prior section. However, each code was grouped per participant and per visit. The team separated the diagnosis codes into 16 medical condition features using Python Pandas as a yes/no Boolean feature for each NAMCS participant. The diagnoses codes for depression were excluded from these features and were used in the creation of the Depression Indicator (see below).

Normalizing depression data. The depression data was normalized to a single depression classification by creating a the “Depression Indicator” (DI) feature for each dataset. For each participant, any positive dependent variable (depression) was considered a “yes” for the Depression Indicator. For example, if a participant received a PHQ total of 10 or more and reported having a chronic condition of depression, that participant was considered as a “yes” for reporting depression. If a provider reported a patient with a depression diagnosis code and depression medication treatment that patient was considered as a “yes” for reporting depression. Any indication of depression was classified on a binary (yes/no) scale for standardization across the datasets. The variables from each set to compile DI included:

- NHANES
 - PHQSCR: PHQ Total>10-- (indicative of depression)
 - DLQ140: Feelings of depression-- >=Monthly,
 - DLQ150: Medications for depression-- Yes
 - DLQ170: Depression Level-- A lot, A little or in between a lot and a little
- NAMCS
 - DEPRN-- depression chronic disease
 - DEPRESS-- depression screen
 - MENTAL-- mental health counseling referral
 - PSYCHOTH—psychotherapist referral

Once feature engineering was complete, NHANES contained 26 features and NAMCS contained 41 features used for hypothesis testing and modeling.

4 Data Analysis

To view and comprehend our data, Python was used. Many built-in Python libraries were utilized, including pandas, NumPy, Seaborn, Matplotlib, and Scipy. Matplotlib is a Python library for creating interactive visualizations such as bar charts and pie charts. Seaborn is a visualization library based on Matplotlib used to generate heatmaps. Scipy is a Python package used for scientific computing and hypothesis testing. To help comprehend the data, we developed the visualizations listed below.

4.1 Descriptive Statistics.

Descriptive statistics allow data to be presented more easily, making interpretation meaningful. Since most variables were categorical, only the mean and median were calculated for the continuous variables of age in the NHANES and NAMCS datasets and "PHQ Score" in the NHANES dataset. The mode, standard deviation, variance, and interquartile range were determined for all categorical variables to understand the distribution and dispersion. Skewness and kurtosis were calculated to identify outliers.

Two features were determined to cause large kurtosis. The data was examined, and it was not found to have outliers. The feature of no insurance only had one participant with a “yes.” This was not unexpected due to the large numbers of participants with other insurances and the changes in U.S. policy in recent years around insurance access for the uninsured. “No Provider” was the other feature with a large kurtosis. This feature was not a valuable feature as it was only for patients who visited the clinic without seeing a provider. Normalization of the data was not required for the categorical data therefore was not performed. even though we specified it in our proposal. The results of the descriptive statistics were compiled into a tabular description for easier comprehension (see appendix).

Among the two data sets, the common variables were age, gender, race, and insurance type. These shared variables were compared across the datasets (Figure 3), (Figure 4) and analyzed for differences in depression reporting (Figure 5), (Figure 6), (), (Figure)

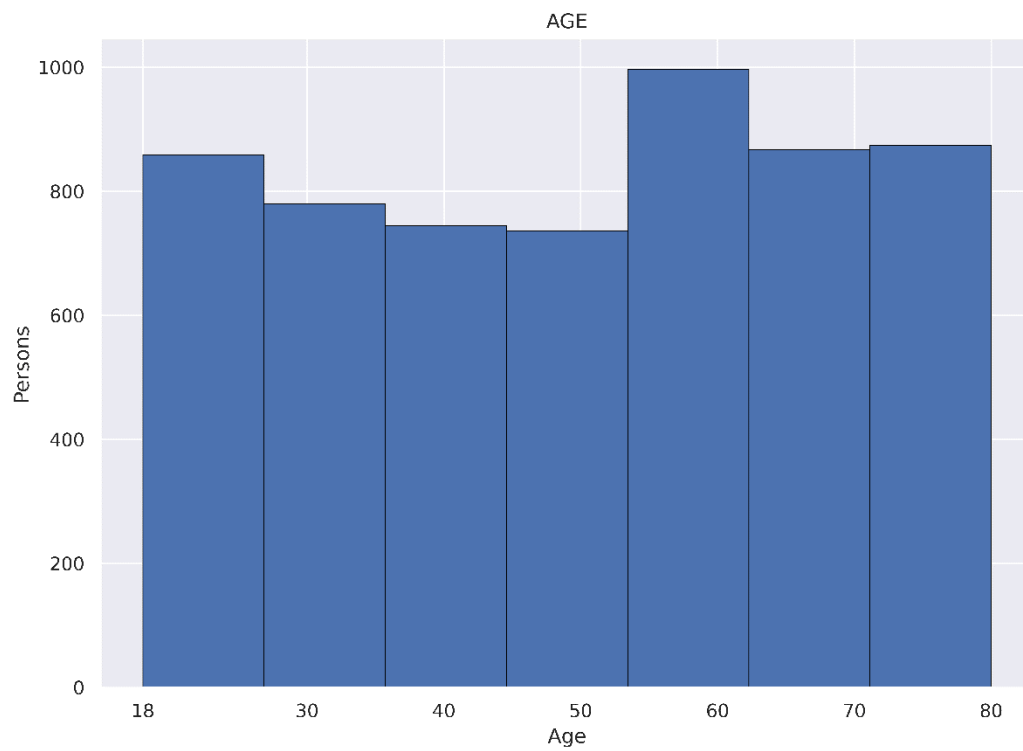


Figure 3. Age Distribution for NHANES

The largest group of persons with an indication of depression in the NAMCS group was those aged 60 to 70. with the distribution skewed to the older population. This can be seen in Figure 4.

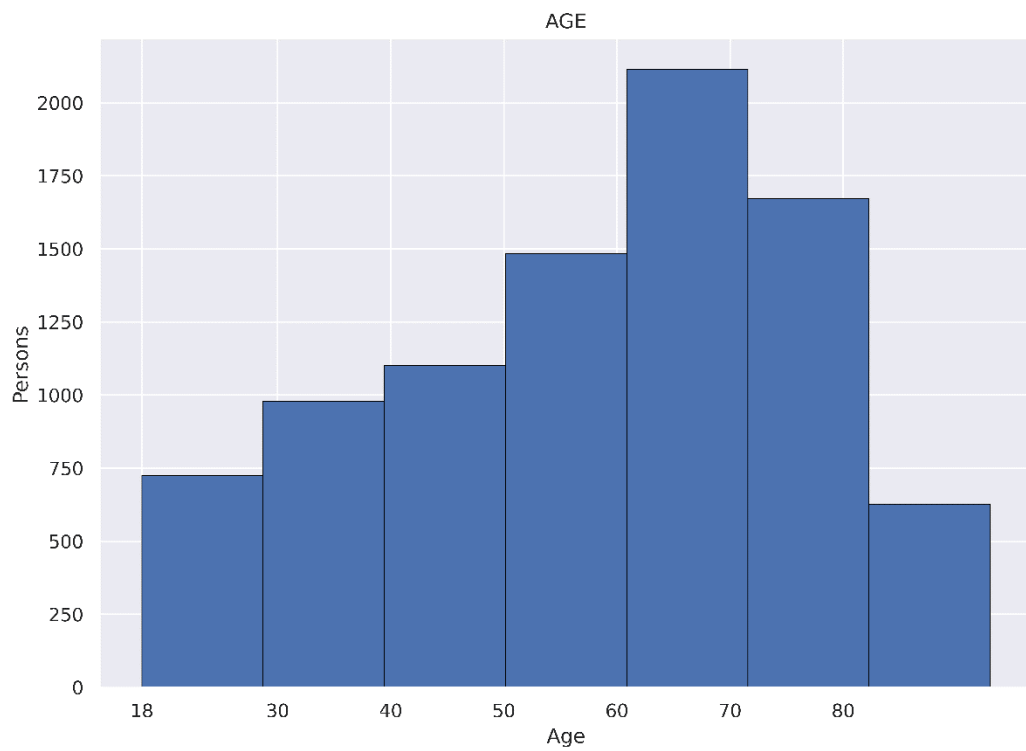
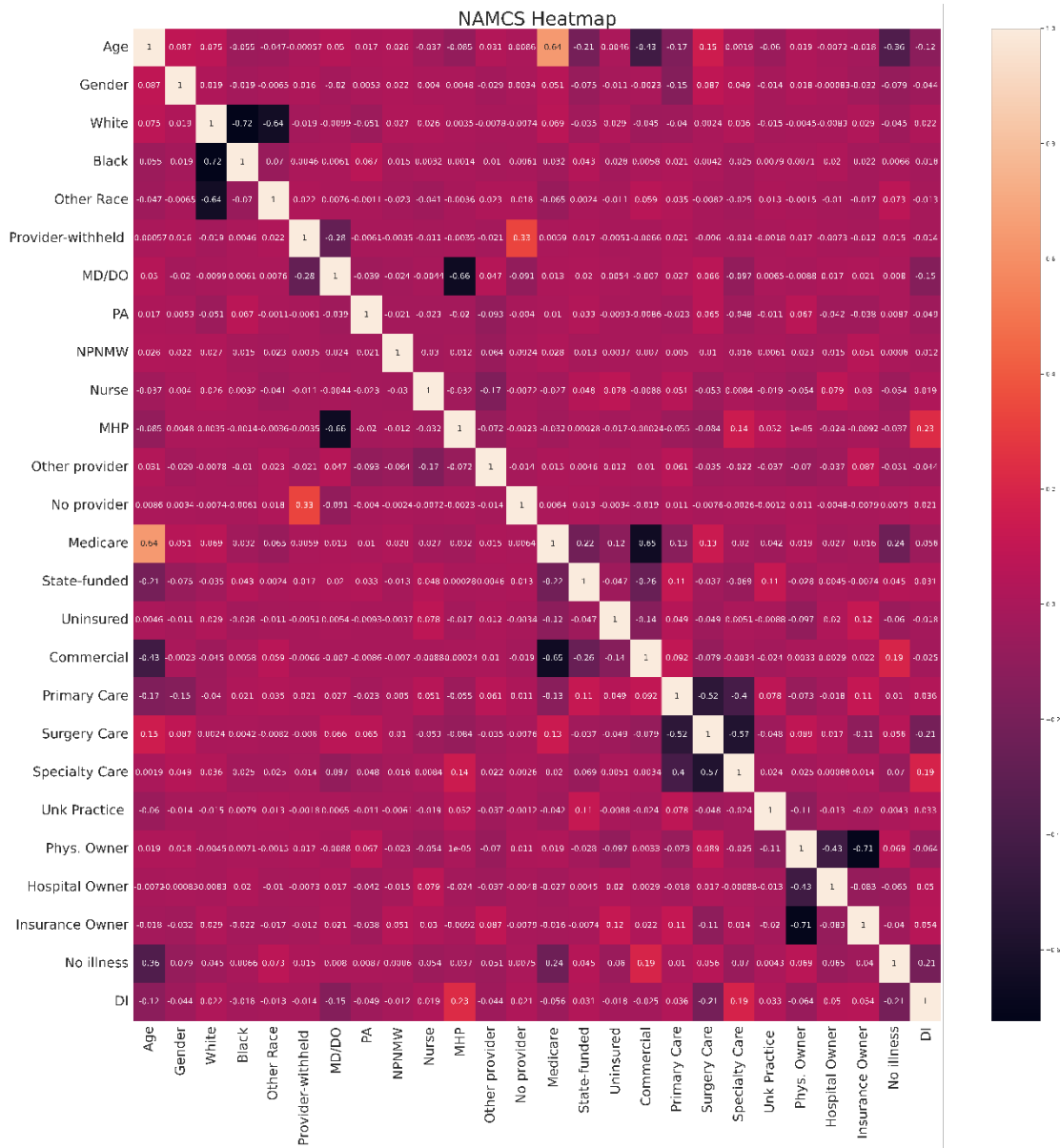


Figure 4. Age Distribution for NAMCS

Figure 5 shows Depression Indicator (DI) vs Race in NAMCS dataset. The 0 indicates 'Yes' and 1 as 'No' in the below graph.



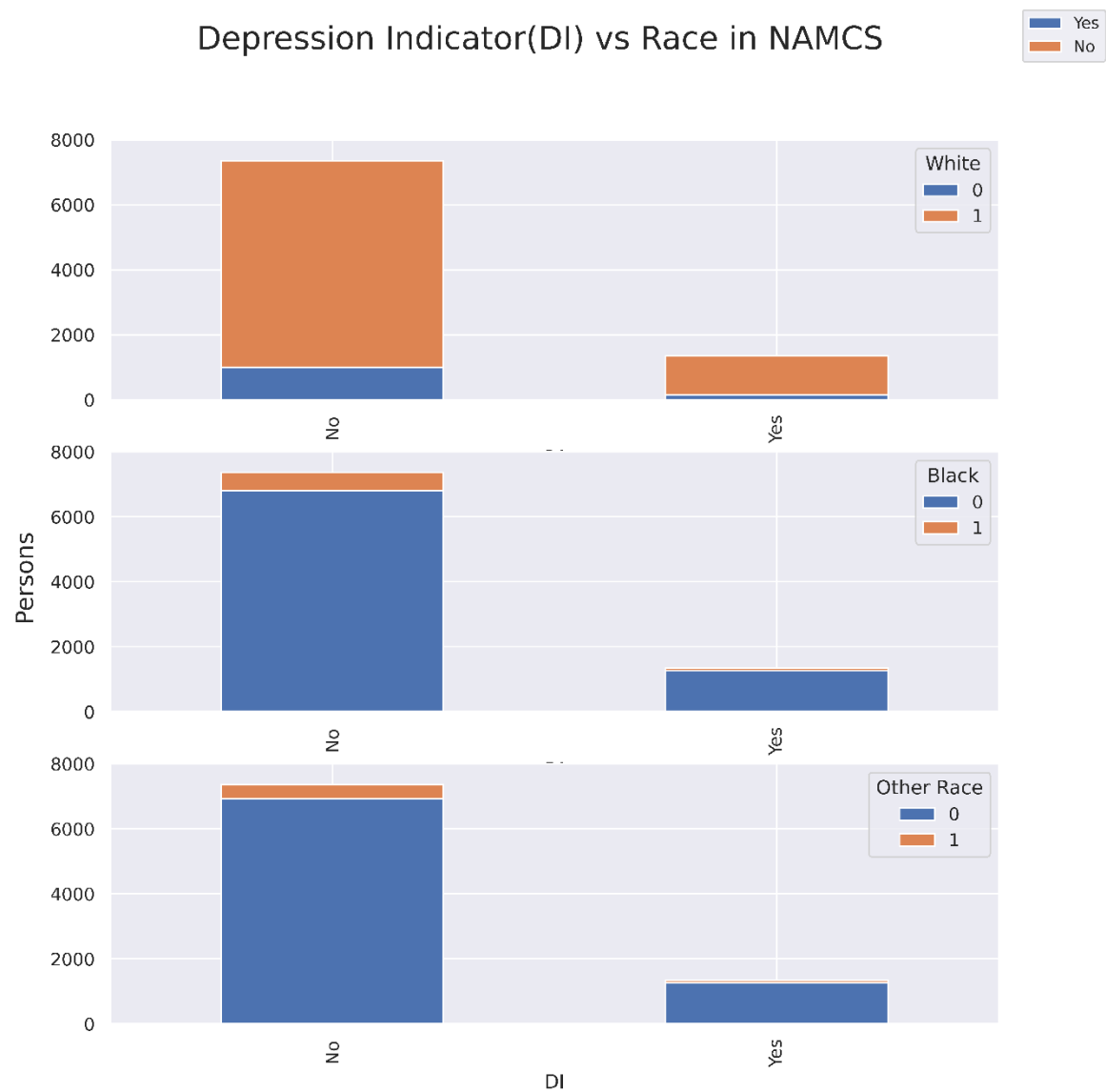


Figure 5. DI by Race in NAMCS

The below graph shows the Depression indicator (DI) by gender in NAMCS dataset. Both males and

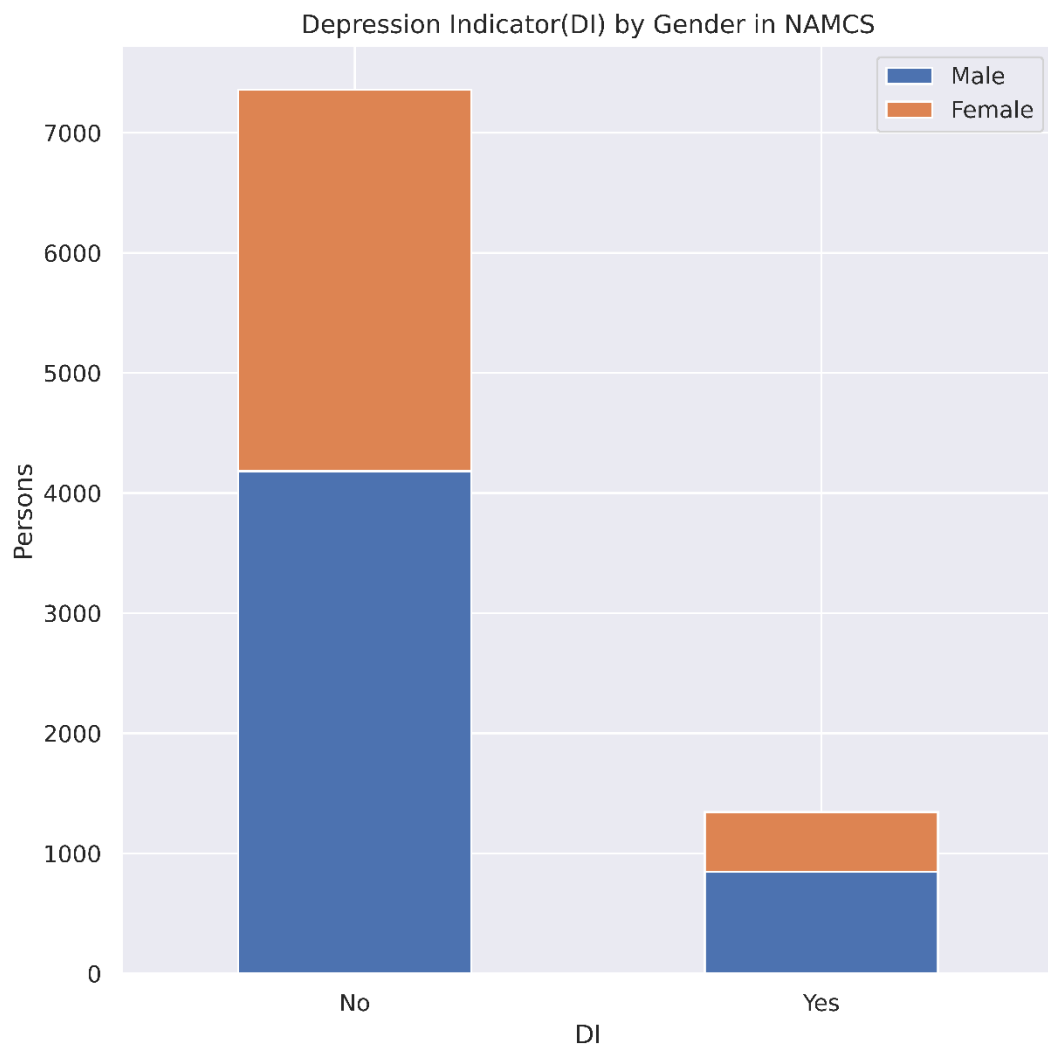


Figure 6. DI by Gender in NAMCS

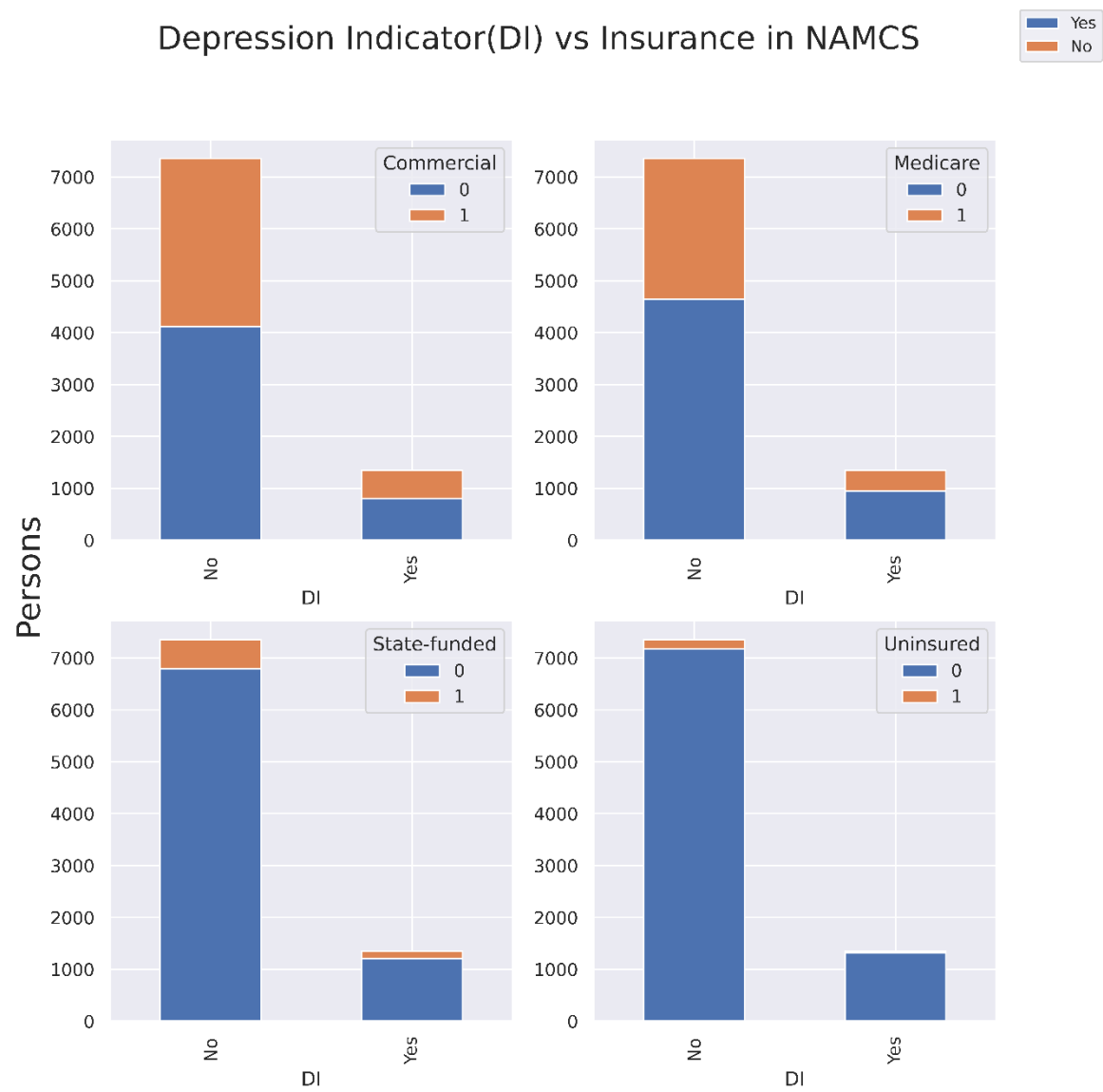
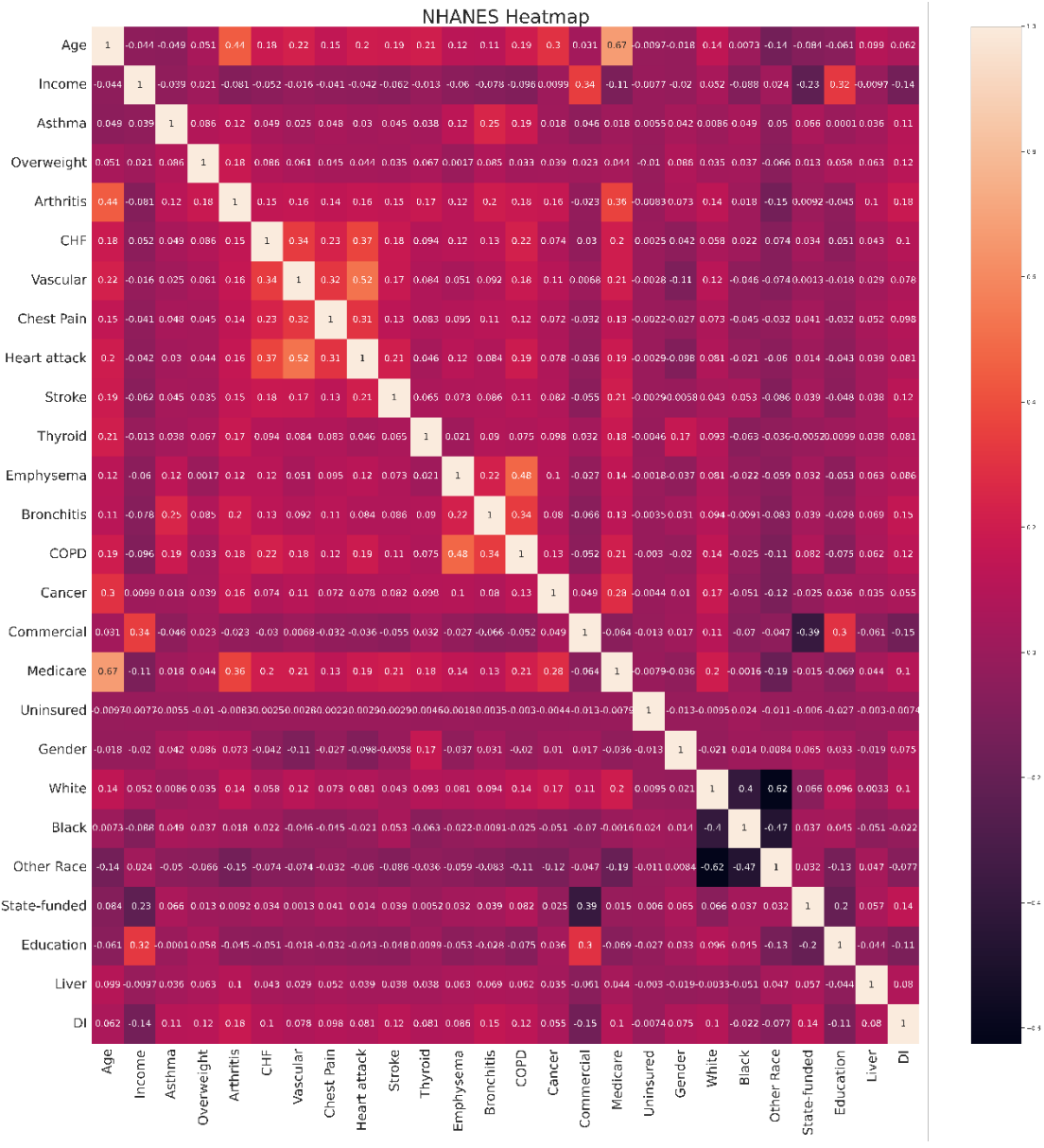
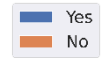


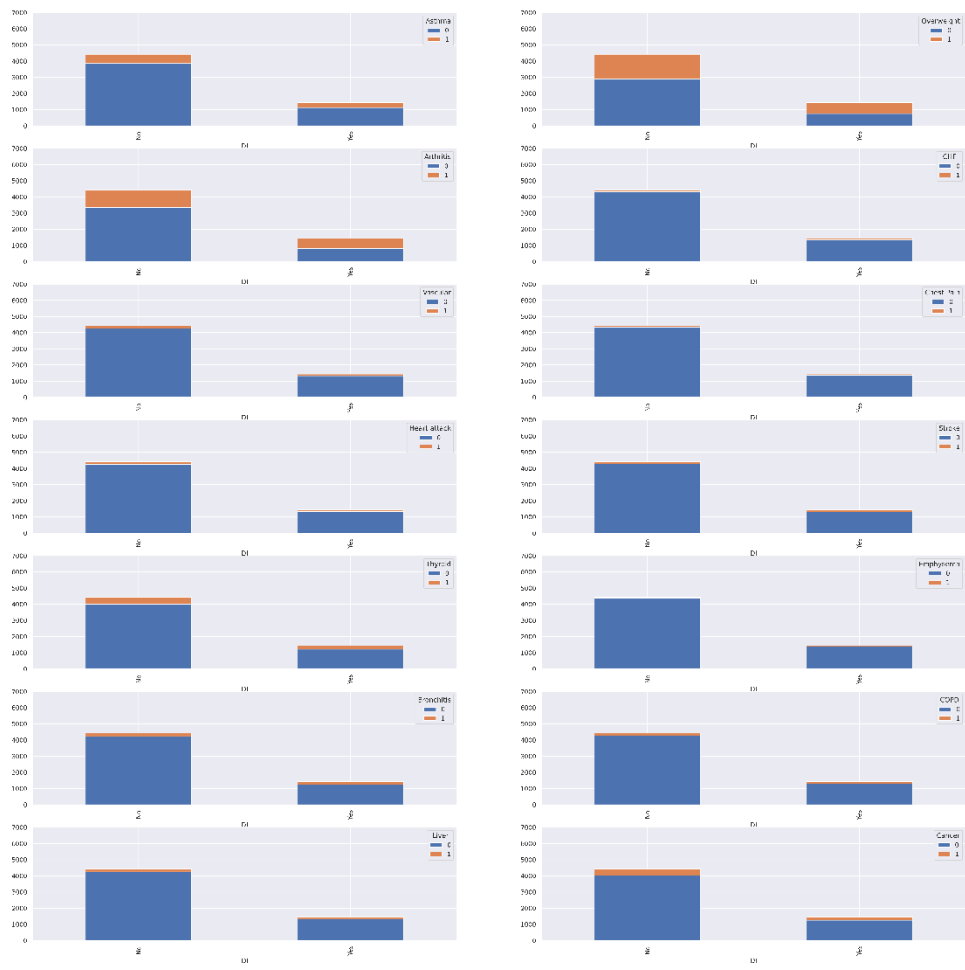
Figure 7. DI by Insurance Type in NAMCS



Depression Indicator(DI) vs Co-morbid in NHANES

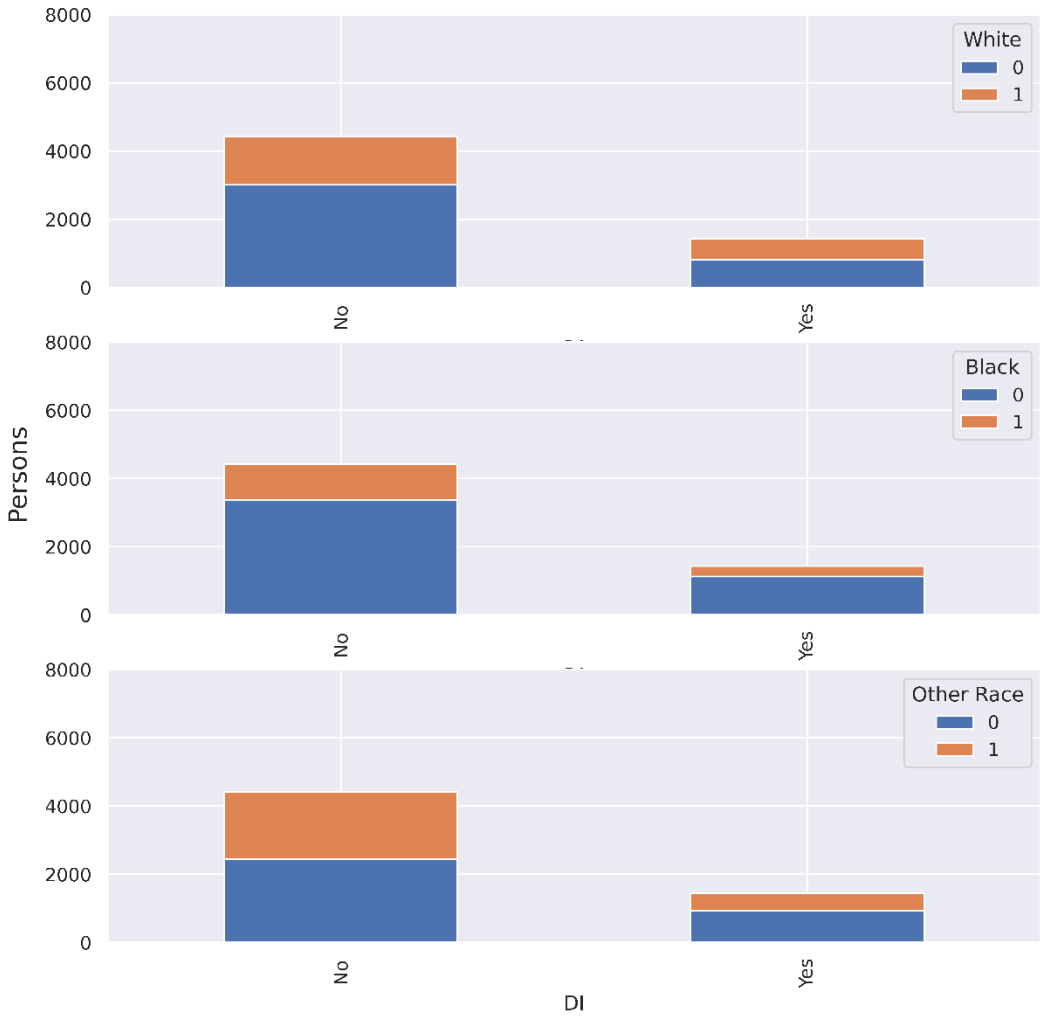


Persons



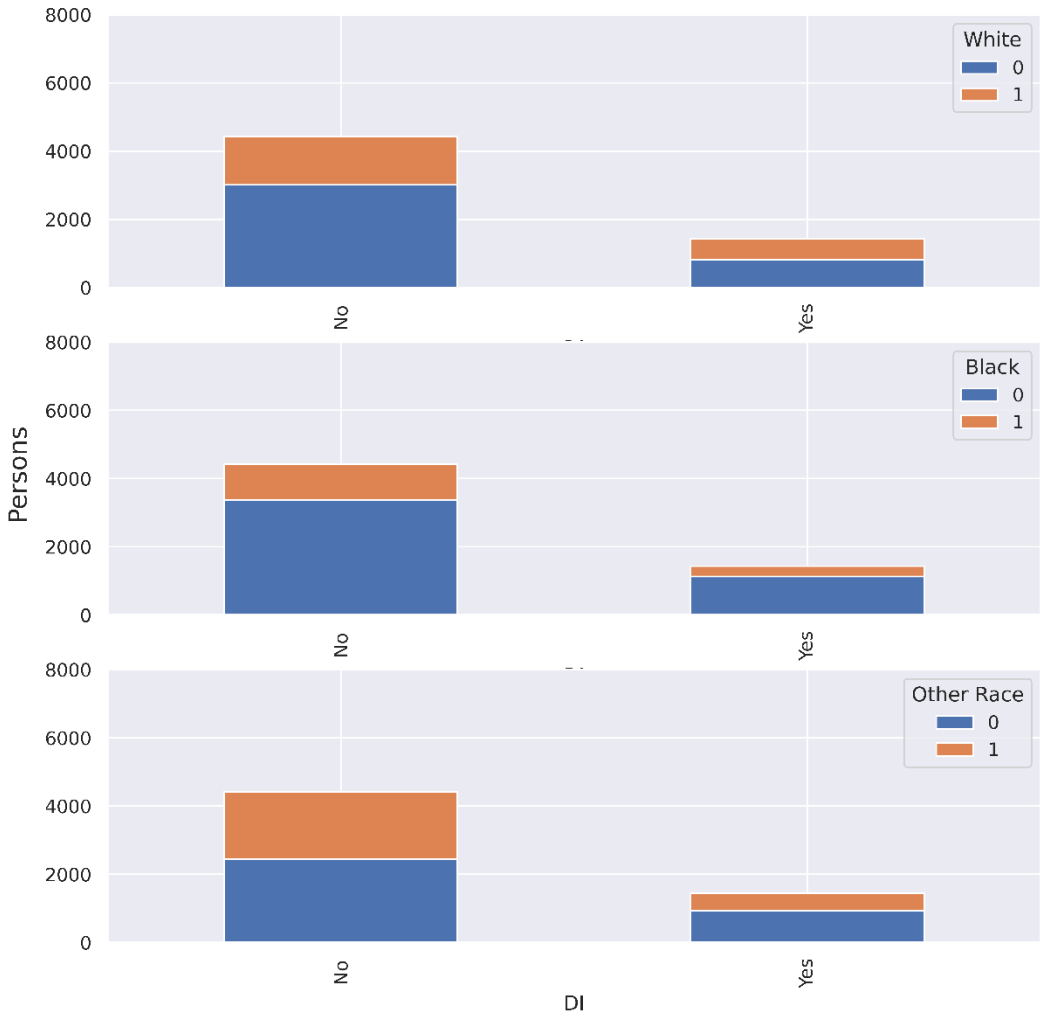
DI

Depression Indicator(DI) vs Race in NHANES





Depression Indicator(DI) vs Race in NHANES



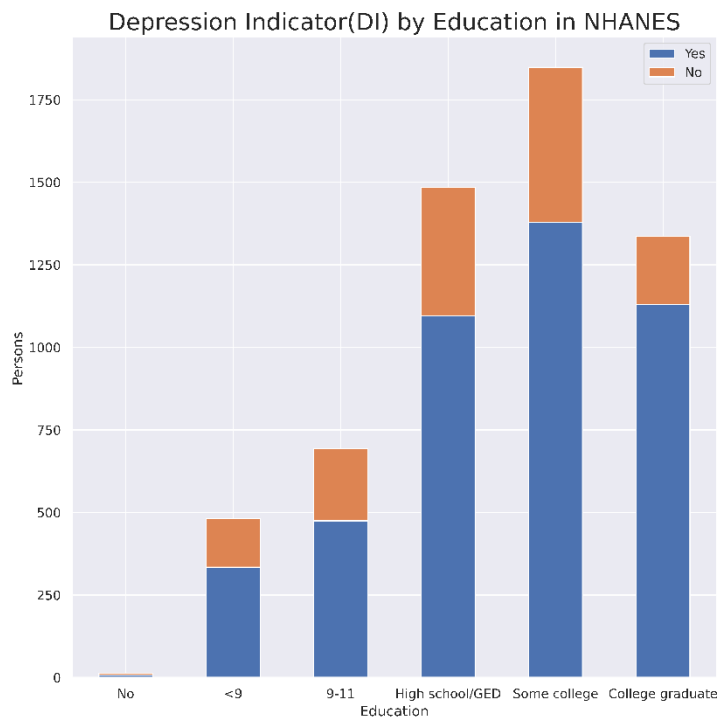


Figure 8 DI by Education for NHANES

The findings indicate a larger association among college going and High school age group depression than those of college graduates in the depression reported in the NHANES.

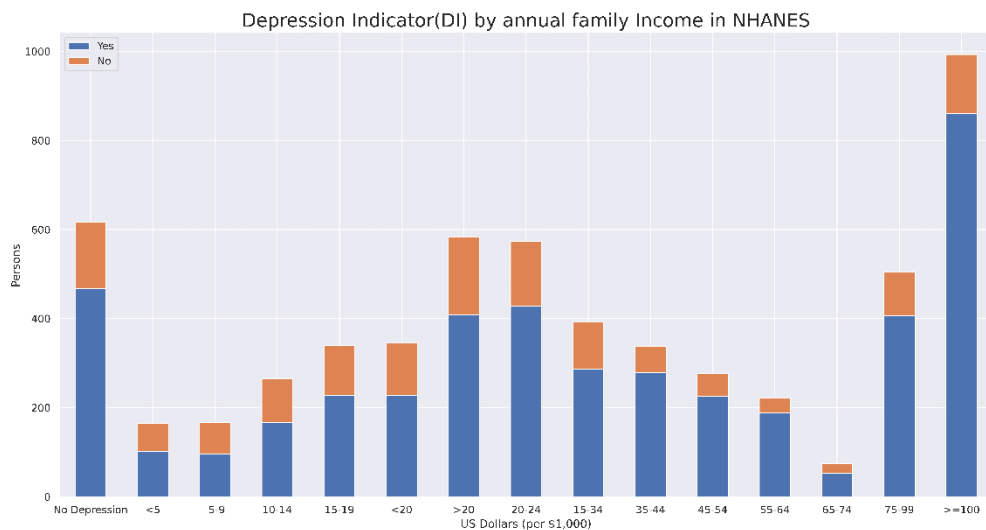


Figure 9 DI by Income for NAMCS

The findings indicate a larger association with ≥ 100 followed by >20 and 20-24 income groups in the NAMCS which reported depression.

Hypothesis testing. The Chi-square test was used to assess any significant associations between the Depression Indicator (DI) and of each independent feature for each dataset and to determine the significant features to be used in the predicative modeling. For the null hypothesis, a significance of $p < 0.05$ was used as an indication to reject the null and a significant association between the groups. The chi-square contingency test in Python was used for the chi-square analysis. The results of the chi-square results determined that 23 and 29 of the features were significant features in NHANES and NAMCS, respectively (Figure) (Figure).

NHANES VARIABLES CHI SQUARE WITH DEPRESSION INDICATOR					
VARIABLE	CODE	P- VALUE	CRITICAL	STATIC	REJECT NULL
Age	AGE	0.03117	81.381	84.336	YES
Income	INDHHIN2	0.00000	23.685	210.484	YES
Asthma	MCQ010	0.00000	3.841	65.169	YES
Overweight	MCQ080	0.00000	3.841	79.925	YES
Arthritis	MCQ160A	0.00000	3.841	182.783	YES
CHF	MCQ160B	0.00000	3.841	56.897	YES
Vascular	MCQ160C	0.00000	3.841	35.056	YES
Chestpain	MCQ160D	0.00000	3.841	55.266	YES
HeartAttack	MCQ160E	0.00000	3.841	37.52	YES
Stroke	MCQ160F	0.00000	3.841	83.849	YES
Thyroid	MCQ160M	0.00000	3.841	37.901	YES
Emphysema	MCQ160G	0.00000	3.841	42.184	YES
Bronchitis	MCQ160K	0.00000	3.841	136.981	YES
COPD	MCQ160O	0.00000	3.841	88.838	YES
Cancer	MCQ220	0.00003	3.841	17.432	YES
Commercial	INSPRVT	0.00000	3.841	130.371	YES
Medicare	INSMCARE	0.00000	3.841	62.943	YES
Uninsured	INSNOCH	1.00000	3.841	0	NO
Gender	SEXML	0.00000	3.841	32.584	YES
White	RACEWH	0.00000	3.841	57.87	YES
Black	RACEBL	0.09907	3.841	2.72	NO
Other Race	RACEOT	0.00000	3.841	34.741	YES
State-Funded	INSSTATE	0.00000	3.841	109.903	YES
Education	EDUC	0.00000	9.488	87.52	YES
Liver	MCQ700	0.00000	3.841	36.454	YES

Figure 10. Chi-Square Testing Results NHANES

NAMCS VARIABLES CHI SQUARE WITH DEPRESSION INDICATOR					
VARIABLE	CODE	P- VALUE	CRITICAL	STATIC	REJECT NULL
Age	AGE	0.00000	96.217	198.706	YES
Gender	SEXML	0.00004	3.841	16.681	YES
White	RACEWH	0.04018	3.841	4.21	YES
Black	RACEBL	0.11329	3.841	2.508	NO
Other	RACEOT	0.25653	3.841	1.287	NO
Provider-withheld	NOPROVID	0.41137	3.841	0.675	NO
MD/DO	PHYS	0.00000	3.841	184.963	YES
PA	PHYSASST	0.00001	3.841	20.164	YES
NPNMW	NPNMW	0.33057	3.841	0.947	NO
Nurse	RNLPN	0.09016	3.841	2.871	NO
MHP	MHP	0.00000	3.841	442.487	YES
Other Provider	OTHPROV	0.00005	3.841	16.417	YES
No provider	PROVNONE	0.22218	3.841	1.49	NO
Medicare	INSMCARE	0.00000	3.841	27.031	YES
State-funded	INSSTATE	0.00475	3.841	7.972	YES
Uninsured	INSNOCH	0.11259	3.841	2.517	NO
Commercial	INSPRVT	0.02384	3.841	5.106	YES
Primary Care	SPPRC	0.00085	3.841	11.126	YES
Surgery Care	SPSUR	0.00000	3.841	365.775	YES
Speciality Care	SPMEC	0.00000	3.841	298.753	YES
Owner-withheld	OWNUNKN	0.00448	3.841	8.08	YES
Phy.Owner	OWNPHYS	0.00000	3.841	35.709	YES
Hospital Owner	OWNHOSP	0.00000	3.841	21.41	YES
Insurance Owner	OWNINSR	0.00000	3.841	24.63	YES
No illness	NOCHRON	0.00000	3.841	373.763	YES
Infection	DGRP1	0.50118	3.841	0.452	NO
Cancer/Blood	DGRP2	0.00000	3.841	27.45	YES
Metabolic	DGRP3	0.01191	3.841	6.325	YES
Neurologic	DGRP4	0.02569	3.841	4.977	YES
Eyes/Ears	DGRP5	0.00000	3.841	148.936	YES
Heart/Lung	DGRP6	0.03875	3.841	4.272	YES
Digestive	DGRP7	0.01181	3.841	6.34	YES
Skin	DGRP8	0.00191	3.841	9.631	YES
Bones	DGRP9	0.27198	3.841	1.207	NO
Urinary	DGRP10	0.01331	3.841	6.127	YES
Women's Health	DGRP11	0.00273	3.841	8.983	YES
Genetic	DGRP12	1.00000	3.841	0	NO
Other Disorder	DGRP13	0.16058	3.841	1.969	NO
Injury	DGRP14	0.02574	3.841	4.973	YES
Public Health	DGRP16	0.00000	3.841	40.53	YES

Figure 11. Chi-Square Testing Results NAMCS

The proportion of the DI between NHANES (Figure) and NAMCS () was also used as a measure of hypothesis testing. The percentage of positive DI per dataset was compared. If the NHANES proportion of DI was higher than the NAMCS proportion, the null hypothesis would be rejected.

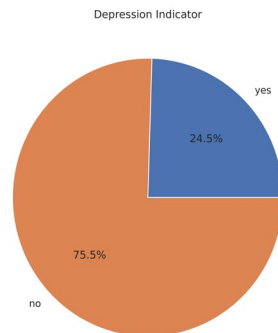


Figure 12. Proportion of DI in NHANES

4.2 Machine Learning and Model Testing.

Binary classification model. XGBoost and Random Tree Forest

With the significant features determined for each dataset, the datasets, a model was be built for future predictions about the risk of depression and depression underreporting will be used as a supervised machine learning model to predict between the classification of depression diagnosis or no depression diagnosis. Scikit-Learn in Python will be used to build and test the model.

4.3 Performance Analysis. Python scikit-learn and a Precision-Recall (PR) Curve [14] will be used to evaluate the model's performance. The PR Curve will produce precision and recall values for probabilities at set thresholds. The PR area under the curve (AUC) will be the metric for evaluating the model's performance.

Binary Classification Models:

With the significant variables from both the datasets we build two models for the diagnosis of depression among the two datasets. The models that we employed for the diagnosis of depression for the two datasets are

- XGBOOST CLASSIFIER
- RANDOM FOREST CLASSIFIER

The two machine learning algorithms were performed on both the datasets. We used different functions for Cross-Validation, Determining Model Metrics, Feature importance, Confusion Matrix, PR-Curves, and ROC curves of the models for comparison and performance analysis between the models.

XGBOOST CLASSIFIER

NAMCS DATASET

As 70% of the data contains information for the N Depression reported and 30% for the Depression reported, we performed a function called **SMOTE** (Synthetic Minority Oversampling Technique) for the balancing of the data we have among the two datasets. Then we performed the testing and training of the NAMCS data set.

The cross-validation score of the NAMCS dataset after SMOTE with XGBOOST classifier using F1 macro is 0.89 with a standard deviation of 0.1. we can see the significant difference for score before and after using the SMOTE.

Cross Validation Score for namcs before smote

```
cross(xgb_cl, Xnam_train, ynam_train, 'f1_macro')
```

Mean f1_macro of 0.68 with a standard deviation of 0.02

Cross Validation Score for namcs after smote

```
cross(xgb_cl, Xnam_train, ynam_train, 'f1_macro')
```

Mean f1_macro of 0.89 with a standard deviation of 0.10

Figure 13 Cross Validation Score before smote NAMCS

The model Performance Scores for the NAMCS after SMOTE are

Accuracy: 85%

Precision: 54.9%

Sensitivity recall: 51%

Specificity: 91.8%

F1_score: 0.5292

Model scores for namcs after smote

```
In [29]: skmets(ynam_test, prednam)
```

```
Out[29]: {'Accuracy': 0.851809304997128,  
          'Precision': 0.5492424242424242,  
          'Sensitivity_recall': 0.5105633802816901,  
          'Specificity': 0.9183253260123542,  
          'F1_score': 0.5291970802919708}
```

Figure 14 Model Scores for NAMCS after smote

After using the function SMOTE the top three features that contributed for the reporting of the depression are:

No Chronic Conditions, patients with diagnosis of heart or lung diseases, and patients with diagnosis of cancer or blood related diseases.

Feature Importance for predicting Depression for namcs after smote

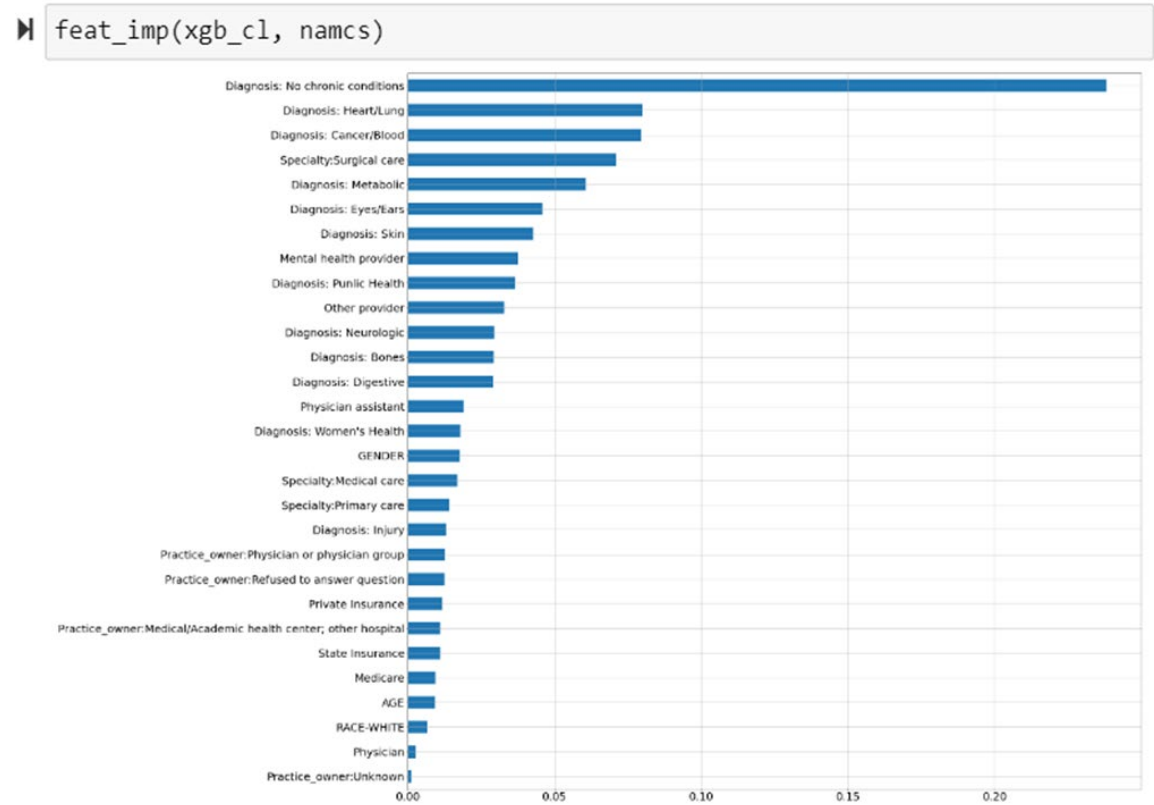


Figure 15 Feature Importance for NAMCS

Confusion Matrix of XGBOOST classifier for NAMCS dataset: The confusion matrix for the XGBOOST classifier after using the SMOTE, we can see that model predictions. Of all the positive cases form the dataset, the model predicted 51% of them as True positives and 49% as False Negatives. But, of all the negative cases from the dataset, the model predicted 91.8% of them as True Negative and 8.2% as False Positives. The model has high accuracy for the prediction of patients with no depression.

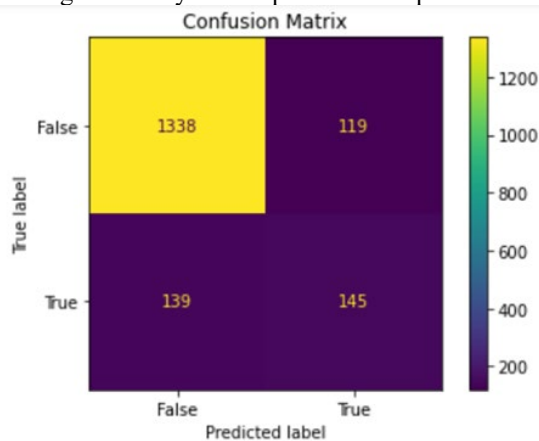


Figure 16 Confusion Matrix of NAMCS

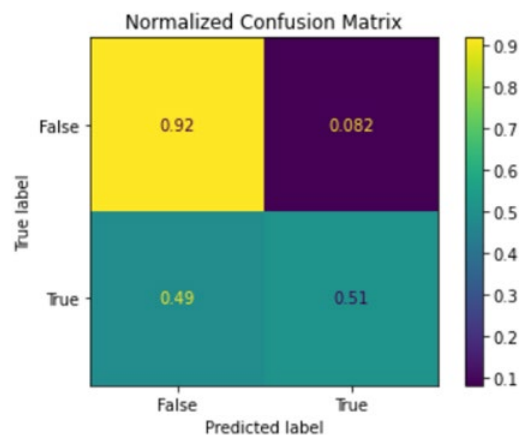


Figure 17 Normalized Confusion Matrix NAMCS

PR curve:

The precision-recall curve for the NAMCS dataset after applying SMOTE, AUC is 0.5698

PR Curve for namcs after smote

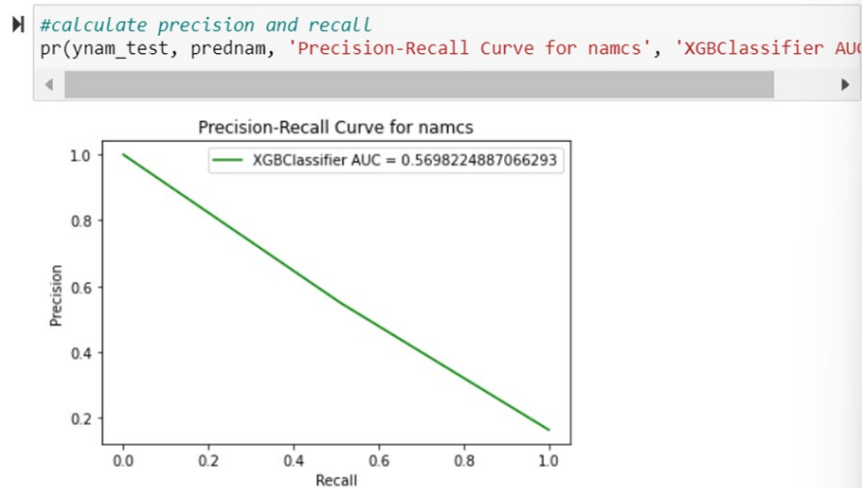


Figure 18 PR Curve NAMCS after Smote

ROC Curve:

The ROC curve for the XGBOOST classifier performed on the NAMCS dataset after SMOTE, AUC is 0.83

ROC Curve for namcs after smote

```
roc(xgb_cl, Xnam_test, ynam_test)
```

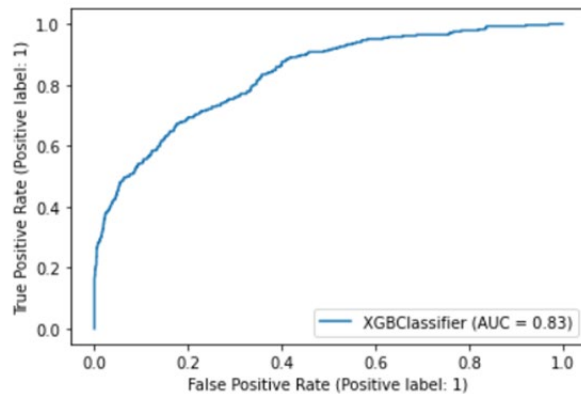


Figure 19 ROC Curve XGBOOST

NHANES DATASET

After performing the XGBOOST classifier on the NAMCS dataset, we performed the same on the NHANES dataset. The SMOTE function is applied to the NHANES dataset for the balancing of the data. The XGBOOST classifier is performed on the NHANES dataset by splitting the data into training and testing data. The cross-validation score(f1_macro) for the NHANES after SMOTE is 0.79 with a standard deviation of 0.17.

Cross Validation Score for nhanes before smote

```
cross(xgb_cl, Xnh_train, ynh_train, 'f1_macro')
```

Mean f1_macro of 0.59 with a standard deviation of 0.02

Cross Validation Score for nhanes after smote

```
cross(xgb_cl, Xnh_train, ynh_train, 'f1_macro')
```

Mean f1_macro of 0.79 with a standard deviation of 0.17

The model performance scores for NHANES are:

Accuracy: 73.8%
Precision: 44.9%
Sensitivity recall: 30.66%
Specificity: 87.8%
F1_score: 0.3644

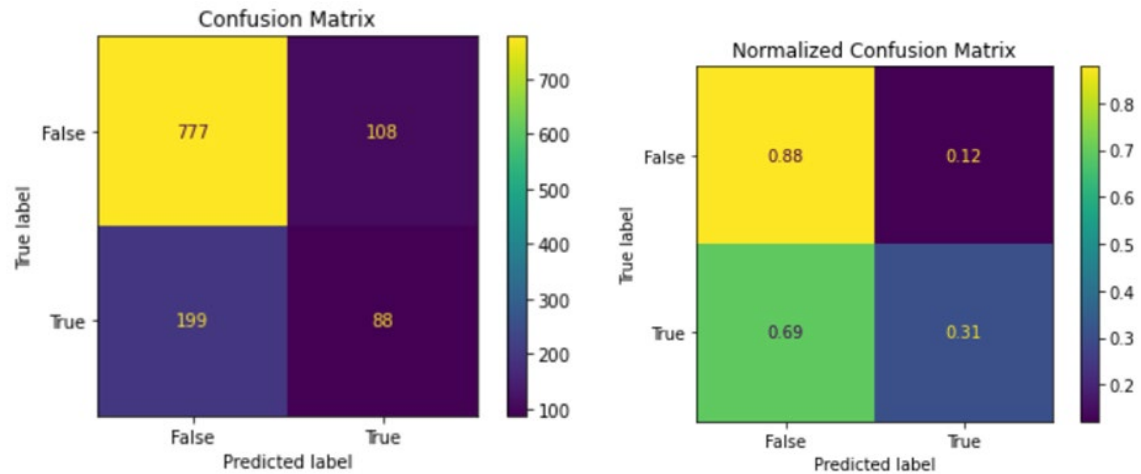
Model scores for nhanes after smote

```
In [46]: skmets(ynh_test, prednh)
```

```
Out[46]: {'Accuracy': 0.7380546075085325,  
          'Precision': 0.4489795918367347,  
          'Sensitivity_recall': 0.30662020905923343,  
          'Specificity': 0.8779661016949153,  
          'F1_score': 0.36438923395445133}
```

Confusion Matrix of XGBOOST classifier for NHANES dataset:

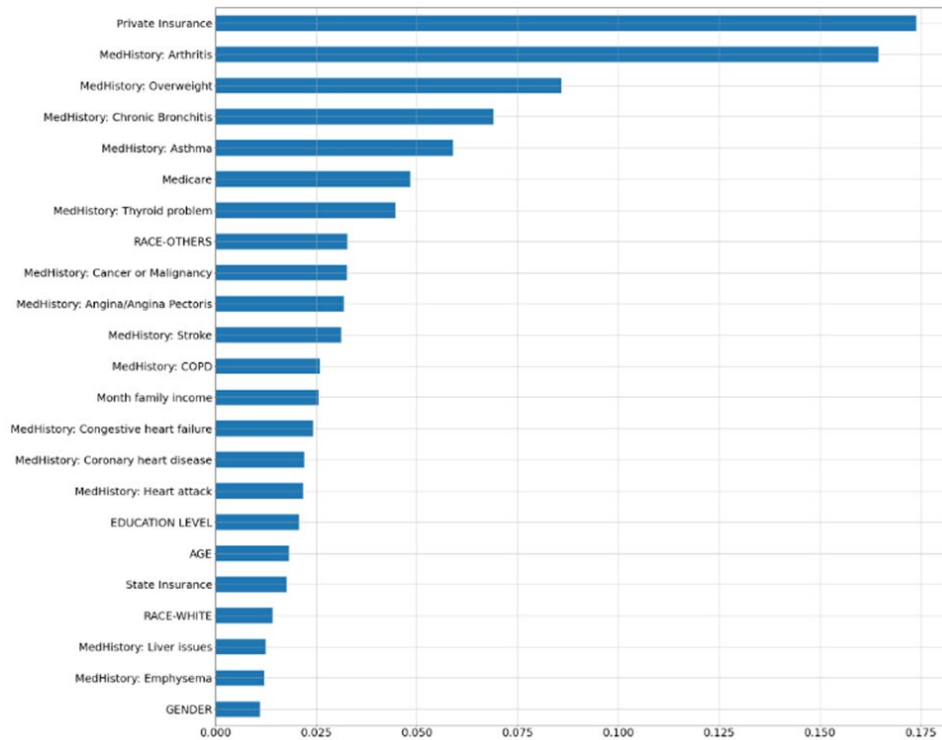
From the confusion matrix, we can see the model predictions. Of all the positive cases from the dataset, the classifier predicted 31% of them as True Positives and 69% of them as False Negatives. Of all the negatives for the Depression from the dataset, the classifier predicted 88% of them as True Negatives and with 12% as False Positives.



After performing the Feature importance function on the NHANES dataset, the top three factors that contributed for the diagnosis of depression are Private Insurance, people suffering from arthritis, and people with overweight in medical history.

Feature Importance for predicting Depression for nhanes after smote

```
feat_imp(xgb_cl, nhanes)
```

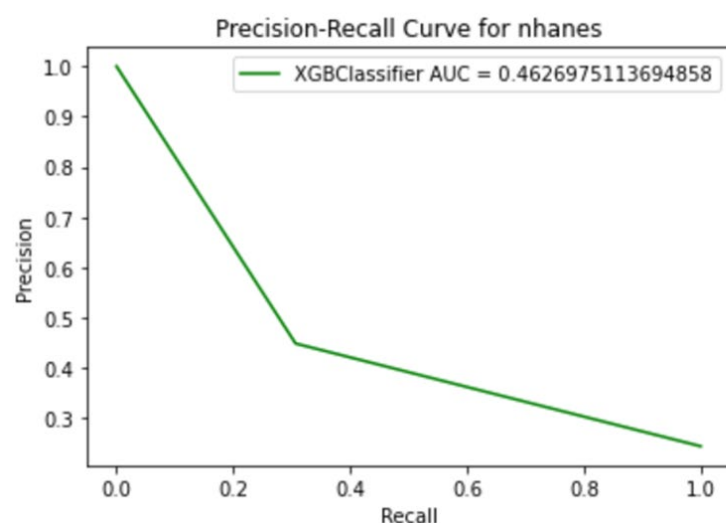


PR curve:

The precision- recall curve for the XGBOOST classifier performed on the NHANES dataset after applying SMOTE, AUC is 0.4627

PR Curve for nhanes after smote

```
#calculate precision and recall  
pr(ynh_test, prednh, 'Precision-Recall Curve for nhanes', 'XGBClassifier
```

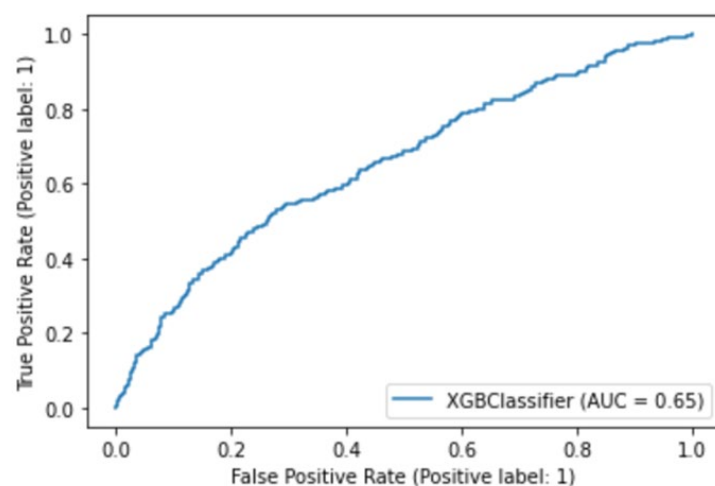


ROC Curve:

The ROC curve for the XGBOOST classifier performed on the NHANES dataset after SMOTE, AUC is 0.65

ROC Curve for nhanes after smote

```
roc(xgb_cl, Xnh_test, ynh_test)
```



RANDOM FOREST CLASSIFIER

After performing the XGBOOST classifier on both datasets, we employed the Random Forest Classifier to classification on both datasets. We fine-tuned the hyperparameters of the Random Forest Classifier using the the RandomizedSearchCV and found that the optimal parameters were:

```
n_estimators = 89,  
min_samples_split = 12,
```

```
min_samples_leaf = 4,  
max_features = 'auto',  
max_depth = 28,  
bootstrap = True
```

NAMCS DATASET

The Random Forest Classifier was performed on the NAMCS dataset by dividing it into train and test data. The SMOTE function was applied to this classifier for balancing the data. The cross-validation score (f1_macro) of Random Forest Classifier for the NAMCS dataset is 0.87 with a standard deviation of 0.08.

```
Mean f1_macro of 0.63 with a standard deviation of 0.02
```

```
Mean f1_macro of 0.87 with a standard deviation of 0.08
```

The model performance score of Random Forest Classifier for NAMCS dataset are:

Accuracy: 84.55%

Precision: 52.59%

Sensitivity recall: 53.52%

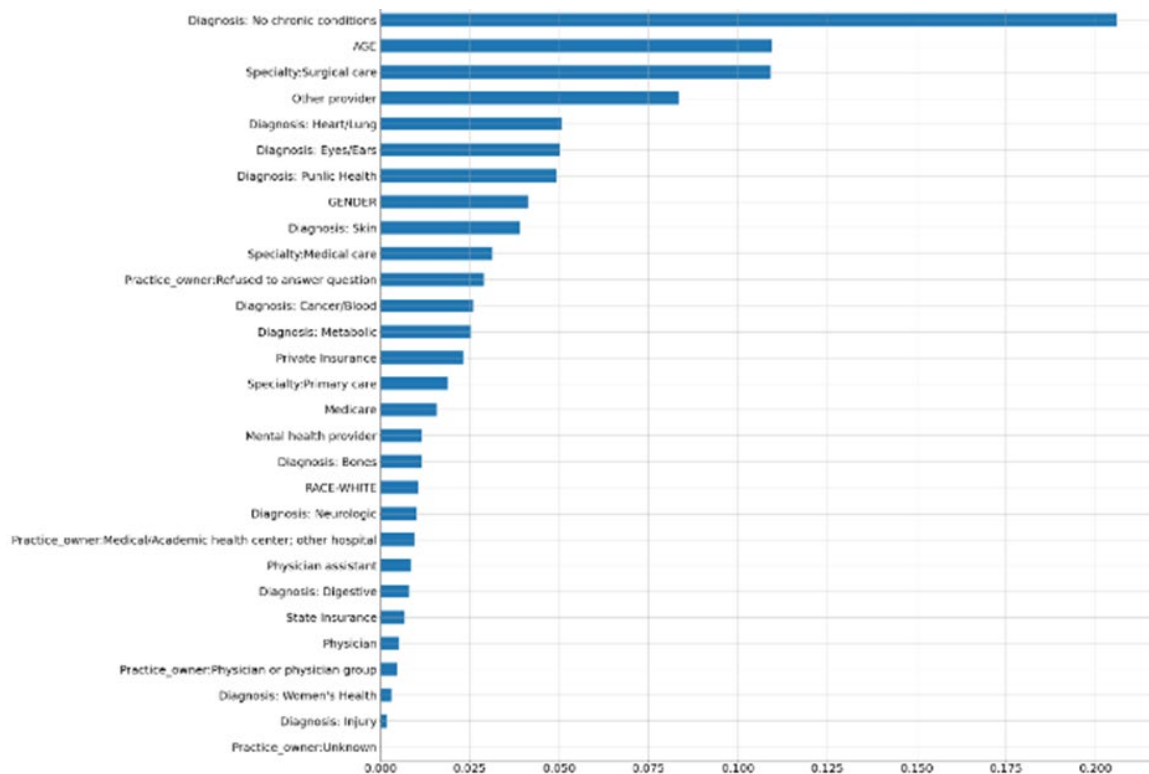
Specificity: 90.6%

F1_score: 0.53

Model scores for namcs after smote

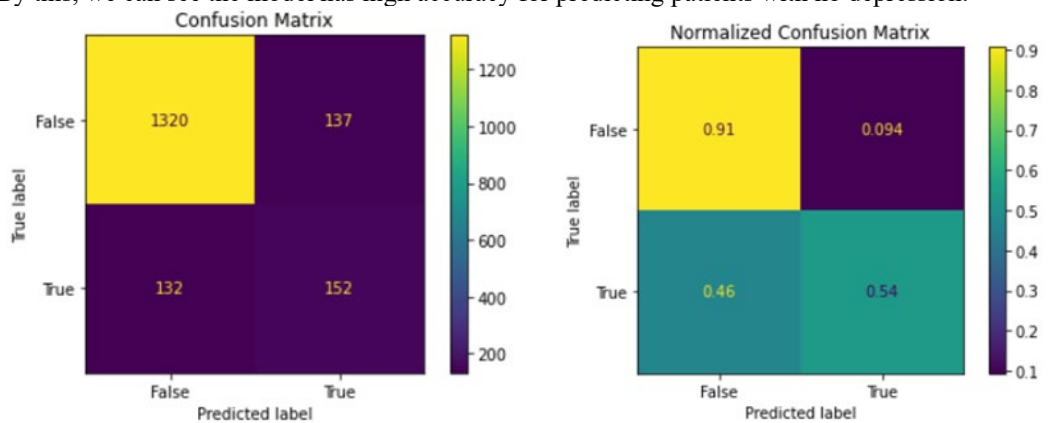
```
skmets(ynam_test, prednam)  
9]: {'Accuracy': 0.8454910970706491,  
     'Precision': 0.5259515570934256,  
     'Sensitivity_recall': 0.5352112676056338,  
     'Specificity': 0.905971173644475,  
     'F1_score': 0.5305410122164049}
```

After performing the Feature importance function on the NAMCS dataset, the top three features that contributed for the reporting of the depression are No Chronic Conditions, Age and Specialty: Surgical Care.



Confusion Matrix of Random Forest Classifier for NAMCS dataset:

From the confusion matrix, we can see the model predictions. Of all the positive cases from the dataset, the classifier predicted 54% of them as True Positives and 46% as False Negatives. Of all the negatives for the Depression from the dataset, the classifier predicted 90.6% of them as True Negatives and 9.4% as False Positives. By this, we can see the model has high accuracy for predicting patients with no depression.

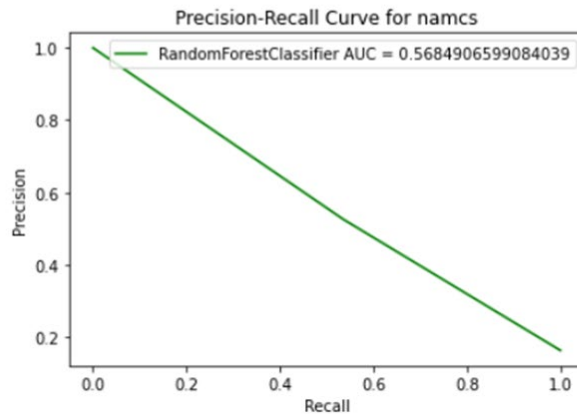


PR curve:

The precision- recall curve for the Random Forest classifier performed on the NAMCS dataset after applying SMOTE, AUC is 0.5685

PR Curve for namcs after smote

```
#calculate precision and recall  
pr(yname_test, predname, 'Precision-Recall Curve for namcs', 'RandomForestClassifier')
```

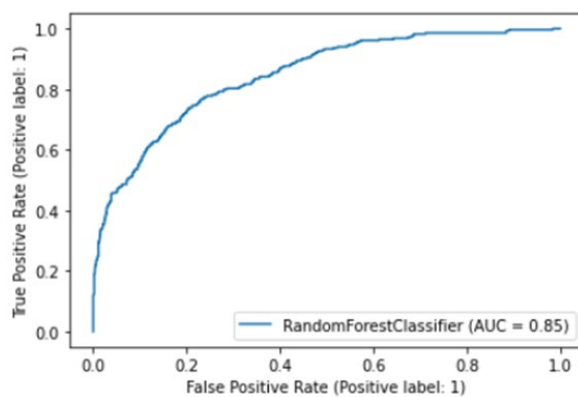


ROC Curve:

The ROC curve for the Random Forest classifier performed on the NAMCS dataset after SMOTE, AUC is 0.85

ROC Curve for namcs after smote

```
roc(clf, Xname_test, yname_test)
```



NHANES DATASET

Here, we performed the Random Forest Classifier on the NHANES dataset by splitting the data into training and testing data. Even here, we used the SMOTE function for balancing the data from the datasets. The cross-validation score (f1_score) of the Random Forest Classifier for the NHANES dataset is 0.79 with a standard deviation of 0.15.

Mean f1_macro of 0.54 with a standard deviation of 0.00

Mean f1_macro of 0.79 with a standard deviation of 0.15

The model performance score of Random Forest Classifier for NHANES dataset are:

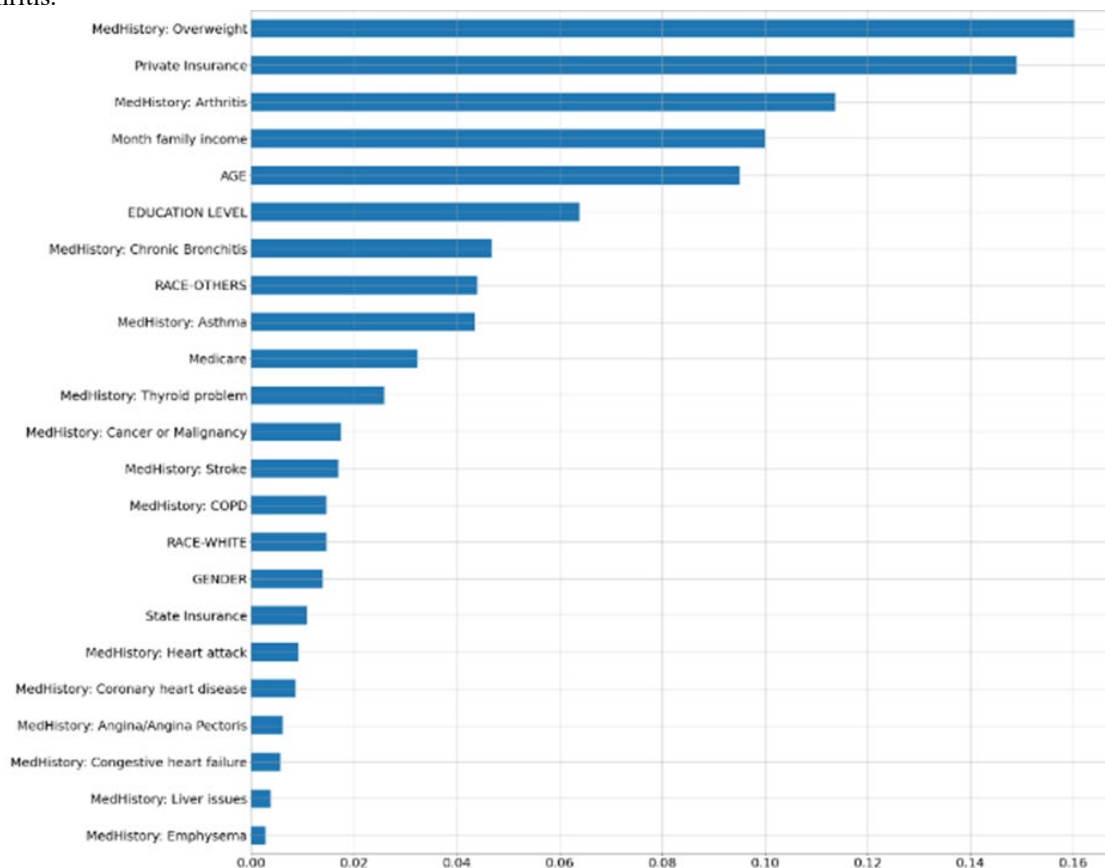
Accuracy: 72.87%
Precision: 42.36%
Sensitivity recall: 29.97%
Specificity: 86.78%
F1_score: 0.3510

Model scores for nhanes after smote

```
skmets(ynh_test, prednh)
```

```
1]: {'Accuracy': 0.7286689419795221,  
     'Precision': 0.4236453201970443,  
     'Sensitivity_recall': 0.29965156794425085,  
     'Specificity': 0.8677966101694915,  
     'F1_score': 0.3510204081632653}
```

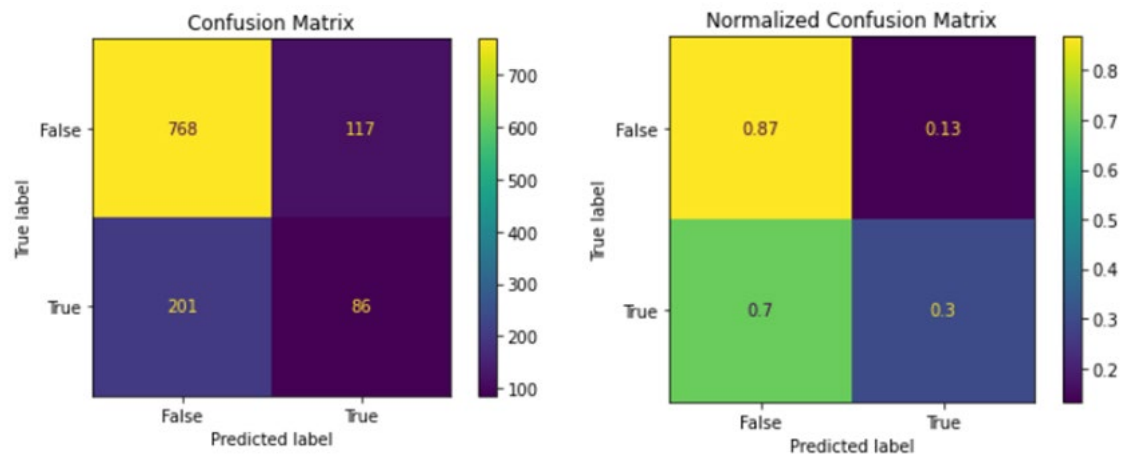
After performing the Feature importance function on the NHANES dataset, the top three features that contributed for the reporting of depression are Medical History of Overweight, Private Insurance and Medical History of Arthritis.



Confusion Matrix of Random Forest Classifier for NHANES dataset:

From the confusion matrix, we can see the model predictions. Of all the positive cases from the dataset, the classifier predicted 30% of them as True Positives and 70% of them as False Negatives. Of all the negatives for the Depression from the dataset, the classifier predicted 87% of them as True Negatives and with 13% as False Positives.

The model has high accuracy for predicting patients with no depression.



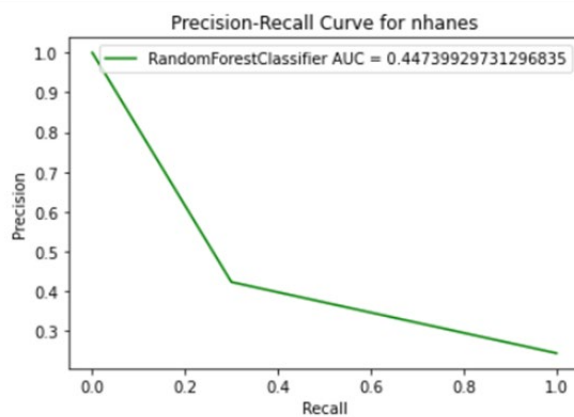
PR curve:

The precision- recall curve for the Random Forest classifier performed on the NHANES dataset after applying SMOTE, AUC is 0.4474

PR Curve for nhanes after smote

```
#calculate precision and recall
pr(ynh_test, prednh, 'Precision-Recall Curve for nhanes', 'RandomForestClass:

```

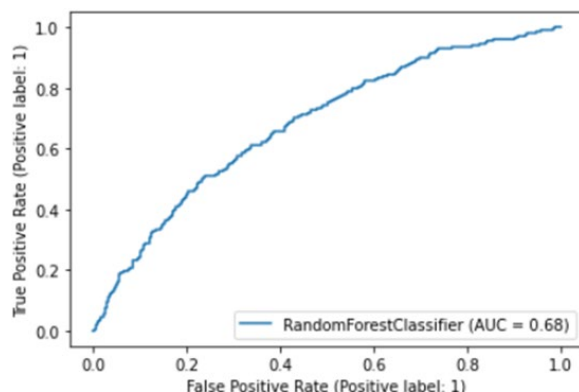


ROC Curve:

The ROC curve for the Random Forest classifier performed on the NHANES dataset after SMOTE, AUC is 0.68

ROC Curve for nhanes after smote

```
roc(clf, Xnh_test, ynh_test)
```



5 Results

Future research should investigate the consequences of depression reporting patterns for provider-reported outcomes. A surprising finding of this study was the higher prevalence of depression reported by healthcare providers compared to self-reporting. Although there were discrepancies in depression reporting, additional research is required to investigate the influence of this reporting on adults and the steps that should be taken on a specific age group for better screening and diagnosis. More research is also required to determine how providers, patients, and members of various health professions report depression.

5.1 Hypothesis Testing

- The chi-square test was utilized for hypothesis testing because both of the data sets contained categorical data.
- Even though we mentioned in the draft proposal using a two-sample t test, we employed the chi square test because the data we had was categorical. So we chose Chi Square over the two sample tests.
- With the use of the chi-square contingency test, we identified the significant factors from both datasets have an association with the depression indicator.
- The significant factors for the depression indicator for both datasets was used to forecast depression using XGBoost, Random Forest and SVM models.

Both models were able to provide good classification for the NHANES and names datasets. The names data was able to provide better classification for the Depression as compared to NHANES survey data. The diagnosis codes were able to provide a good deal of information to the model to enable the Depression Indicator. There was found to be an increase in the depression proportion in the NHANES dataset as compared to names dataset and significant associations between the depression indicator and multiple factors that were also found to predict depression. Hence, we reject the null hypothesis as there is substantiating evidence to do so. The predictive model found that a person in the NAMCS dataset has a higher chance of being detected for depression compared to a person who was surveyed in NHANES. The analysis provided information for the specific groups most in need of screening for depression and for those providers and can be used in future study to develop a more precise depression screening tool for adults.

The team was meticulous about details and had to rerun the codes several times to attain the required results. After cleaning and running the codes, we saw anomalies in our methods and considered several approaches to ensure we were doing the project correctly and on time. It would have been far better if we had chosen a dataset from the same population that self-reported depression and where providers reported depression, which would have provided more accurate results about depression reporting.

6 Conclusions

- This project was able to demonstrate multiple factors impacting depression and the providers who report depression.
- Hypothesis testing supported the rejection of the null hypothesis. Depression cases were observed to be lower in the NHANES dataset compared to the NAMCS dataset. As a result, we reject the null hypothesis since there is sufficient evidence to support it.
- A person in the NAMCS dataset has a higher chance of being detected for depression compared to a person who was surveyed in NHANES.
- A model was established to support the further development of a survey that can be used to further develop a depression indicator.
- Both models were able to provide good classification for the NHANES and NAMCS datasets. The NAMCS data was able to provide better classification for the Depression as compared to NHANES survey data.
- We conclude that a person in the NAMCS dataset has a higher chance of being detected for depression compared to a person who was surveyed in NHANES.

7 Limitations

Although we made every effort to minimize limitations, there inevitably are limitations in any research study. The use of 2018 survey data for both datasets due to constraints from the Covid pandemic's ban on in-person contact limits the possible relevancy of the data. The data is recent data, however data changes rapidly in all data science, especially medical data. There is a chance that depression rates and factors leading to depression have been altered from the compounding effect of the Covid pandemic on depression.

Findings from the models related to direct comparisons of the two datasets are limited because the two samples are from two separate populations. To compare the datasets, we ran the model predictions for the diagnosis of depression on the two datasets.

8 Project Challenges and Successes

Initially, the team was meant to do a comparative analysis by identifying the common variables in both datasets and analyze the mean differences in depression reporting by providers compared to the lay public. The project intended to identify any discrepancies in identification of patients with depression by providers. Nonetheless, after consultation with our advisor and further investigation into the data, the populations are not the same and cannot be compared directly. Also, after getting full access to the data, the team realized that most of the variables were categorical, and t-testing was not appropriate for hypothesis testing. To obtain pertinent findings, chi-square analyses were required to identify significant variables, and the modeling was possible to conclude the hypothesis testing.

9 Appendix
(Figure)

CONTINUOUS DATA										
DATASET	n	VARIABLE	MEAN	MEDIAN	MODE	STD	VARIANCE	IQR	SKEWNESS	KURTOSIS
NAMCS	8701	Age	58	61	71	18.44	340.1	27	0	-0.74
NHANES	5856	Age	50	51	80	18.78	352.54	32	-0.06	-1.17
NHANES	5856	PHQ Score	2.82	1	0	4.11	16.89	4	2.08	4.67

Figure 20. Descriptive Statistics-Continuous Variables

(Figure)

CATEGORICAL DATA													
		NHANES						NAMCS					
VARIABLE	MODE	STD	VARI	IQR	SKEWNESS	KURTOSIS		VARIABLE	MODE	STD	VARIANCE	IQR	SKEWNESS
Gender	1	0.50	0.25	1	-0.06	-2.00		Gender	0	0.49	0.24	1	0.31
Black	0	0.42	0.18	0	1.29	-0.34		Black	0	0.26	0.07	0	3.25
Other Race	0	0.49	0.24	1	0.31	-1.91		Other Race	0	0.23	0.05	0	3.77
White	0	0.48	0.23	1	0.64	-1.59		White	1	0.34	0.11	0	-2.17
Education	4	1.55	2.40	3	-0.44	-1.33		Speciality Care	0	0.46	0.21	1	0.84
Income	15	4.98	24.81	10	0.04	-1.17		Primary Care	0	0.44	0.20	1	1.05
Medicare	0	0.44	0.20	1	1.04	-0.91		Surgery Care	0	0.49	0.24	1	0.30
Uninsured	0	0.01	0.00	0	76.52	5856.00		Nurse	0	0.30	0.09	0	2.65
Commercial	0	0.50	0.25	1	0.05	-2.00		Provider-withheld	0	0.03	0.00	0	31.05
State-funded	0	0.38	0.14	0	1.71	0.93		NPNMW	0	0.11	0.01	0	8.98
Asthma	0	0.36	0.13	0	1.95	1.79		MHP	0	0.11	0.01	0	9.22
Overweight	0	0.49	0.24	1	0.49	-1.76		Other provider	0	0.46	0.21	1	0.83
Arthritis	0	0.45	0.21	1	0.93	-1.14		MD/DO	1	0.11	0.01	0	-8.49
CHF	0	0.18	0.03	0	5.12	24.19		PA	0	0.18	0.03	0	5.12
Vascular	0	0.21	0.04	0	4.38	17.16		No provider	0	0.02	0.00	0	46.62
Chest Pain	0	0.16	0.03	0	5.78	31.43		Hospital Owner	0	0.21	0.05	0	4.21
Heart attack	0	0.21	0.04	0	4.33	16.75		Insurance Owner	0	0.32	0.10	0	2.36
Stroke	0	0.21	0.04	0	4.30	16.51		Phys.Owner	1	0.41	0.17	0	-1.41
Emphysema	0	0.13	0.02	0	7.23	50.31		Owner-withheld	0	0.06	0.00	0	17.87
Bronchitis	0	0.25	0.06	0	3.45	9.91		Medicare	0	0.48	0.23	1	0.60
Thyroid	0	0.32	0.10	0	2.46	4.07		Uninsured	0	0.15	0.02	0	6.17
COPD	0	0.22	0.05	0	4.13	15.05		Commercial	0	0.50	0.25	1	0.26
Cancer	0	0.30	0.09	0	2.66	5.08		State-funded	0	0.27	0.07	0	3.09
Liver	0	0.22	0.05	0	4.12	14.99		Infection	0	0.04	0.00	0	23.26
DPQ1	0	0.72	0.52	0	2.31	4.74		Urinary	0	0.23	0.05	0	3.88
DPQ2	0	0.68	0.46	0	2.48	5.84		Womens Health	0	0.12	0.01	0	8.10
DPQ3	0	0.91	0.83	1	1.60	1.44		Genetic	0	0.05	0.00	0	19.82
DPQ4	0	0.91	0.83	1	1.33	0.83		Other Disorder	0	0.28	0.08	0	3.01
DPQ5	0	0.74	0.55	0	2.32	4.68		Injury	0	0.15	0.02	0	6.38
DPQ6	0	0.59	0.34	0	3.18	10.30		Morbidity	0	0.00	0.00	0	0.00
DPQ7	0	0.64	0.41	0	3.10	9.27		Public health	0	0.37	0.13	0	1.87
DPQ8	0	0.51	0.26	0	4.01	16.56		Cancer/Blood	0	0.18	0.03	0	5.16
DPQ9	0	0.28	0.08	0	7.38	61.53		Metabolic	0	0.21	0.05	0	4.27
PHQ Score	0	4.11	16.89	4	2.08	4.67		Neurologic	0	0.17	0.03	0	5.49
Feels DPRN	5	1.27	1.61	1	-1.40	1.19		Eyes/Ears	0	0.35	0.12	0	2.07
DPRN Meds	2	0.39	0.15	0	-2.99	8.73		Heart/Lung	0	0.26	0.07	0	3.20
DPRN Level	0	1.15	1.31	2	0.80	-0.86		Digestive	0	0.17	0.03	0	5.37
DI	0	0.43	0.19	0	1.18	-0.60		Skin	0	0.28	0.08	0	2.89
								Bones	0	0.25	0.06	0	3.47
								No illness	0	0.47	0.22	1	0.70
								Chronic DPRN	0	0.30	0.09	0	2.61
								Therapy Referral	0	0.14	0.02	0	6.97
								MHP Referral	0	0.14	0.02	0	6.76
								DPRN Screen	0	0.20	0.04	0	4.58
								DI	0	0.36	0.13	0	1.91

Figure 21. Descriptive Statistics-Categorical Variables

2. World Health Organization: Mental Health and COVID-19: Early evidence of the pandemic's impact (2022)
- 3.
4. Knaak, S., Mantler, E., Szeto, A.: Mental illness-related stigma in healthcare. *Healthcare Management Forum* 30, 111-116 (2017)
5. Hansson, L., Jormfeldt, H., Svedberg, P., Svensson, B.: Mental health professionals' attitudes towards people with mental illness: do they differ from attitudes held by people with mental illness? *Int J Soc Psychiatry* 59, 48-54 (2013)
6. Garcia, M.E., Hinton, L., Neuhaus, J., Feldman, M., Livaudais-Toman, J., Karliner, L.S.: Equitability of Depression Screening After Implementation of General Adult Screening in Primary Care. *JAMA Network Open* 5, e2227658-e2227658 (2022)
7. National Center for Health Statistics: Summary of Current Surveys and Data Collection Systems. factsheet, Centers for Disease Control and Prevention (2020)
8. National Center for Health Statistics (NCHS): National Health and Nutrition Examination Survey. In: Centers for Disease Control and Prevention (CDC) (ed.). U.S. Department of Health and Human Services, Centers for Disease Control and Prevention,, Hyattsville, MD (2018)
9. National Center for Health Statistics: Unweighted Response Rates for NHANES 2017-2018 by Age and Gender. In: *NHANES-2017-2018-Response-Rates-508* (ed.).
10. National Center for Health Statistics: National Ambulatory Medical Care Survey. Center for Disease Control, (2018)
11. National Center for Health Statistics: National Ambulatory Medical Care Survey: 2018 National Summary Tables. Centers for Disease Control and Prevention, (2018)
12. National Center for Health Statistics (NCHS): 2018 NAMCS Micro-Data File Documentation. In: *Ambulatory and Hospital Care Statistics* (ed.), pp. 155. Division of Health Care Statistics, (2018)
- 13.
14. Boyd, K., Eng, K.H., Page, C.D.: Area under the Precision-Recall Curve: Point Estimates and Confidence Intervals. In: *Machine Learning and Knowledge Discovery in Databases*, pp. 451-466. Springer Berlin Heidelberg, (Year)

```
In [1]: import pandas as pd
import numpy as np
import re
```

NAMCS

```
In [2]: df = pd.read_sas('namcs2018_sas.sas7bdat', format = 'sas7bdat', encoding='Latin 1')
```

C:\Users\alanv\AppData\Roaming\Python\Python39\site-packages\pandas\io\sas\sas7bdat.py:800: PerformanceWarning: DataFrame is highly fragmented. This is usually the result of calling `frame.insert` many times, which has poor performance. Consider joining all columns at once using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe = frame.copy()`

rslt[name] = self._byte_chunk[jb, :].view(dtype=self.byte_order + "d")

C:\Users\alanv\AppData\Roaming\Python\Python39\site-packages\pandas\io\sas\sas7bdat.py:809: PerformanceWarning: DataFrame is highly fragmented. This is usually the result of calling `frame.insert` many times, which has poor performance. Consider joining all columns at once using pd.concat(axis=1) instead. To get a de-fragmented frame, use `newframe = frame.copy()`

rslt[name] = self._string_chunk[js, :]

```
In [3]: DIAG= ['DIAG1', 'DIAG2', 'DIAG3', 'DIAG4', 'DIAG5']
```

```
In [4]: df.shape
```

```
Out[4]: (9953, 1038)
```

```
In [5]: df.to_csv('namcs_.csv', encoding='utf-8', index=False)
```

```
In [6]: df1 = pd.read_csv('namcs_.csv',
    usecols=['AGE', 'SEX', 'RACER', 'PAYTYPER', 'DIAG1', 'DIAG2', 'DIAG3', 'DIAG4', 'DIAG5', 'DEPRN', 'NOCHRON', 'DEPRESS',
    'MENTAL', 'PSYCHOTH', 'NOPROVID', 'PHYS', 'PHYSASST', 'NPNMW', 'RNLPN', 'MHP', 'OTHPROV', 'PROVNONE', 'SPECCA',
    'OWNSR'], low_memory=False)
```

```
In [7]: print(df1.columns)
```

```
Index(['AGE', 'SEX', 'RACER', 'PAYTYPER', 'DIAG1', 'DIAG2', 'DIAG3', 'DIAG4',
      'DIAG5', 'DEPRN', 'NOCHRON', 'DEPRESS', 'MENTAL', 'PSYCHOTH',
      'NOPROVID', 'PHYS', 'PHYSASST', 'NPNMW', 'RNLPN', 'MHP', 'OTHPROV',
      'PROVNONE', 'SPECCAT', 'OWNSR'],
      dtype='object')
```

```
In [8]: df2 = df1[df1['AGE'] > 17]
```

```
In [9]: df2.head()
```

```
Out[9]:
```

	AGE	SEX	RACER	PAYTYPER	DIAG1	DIAG2	DIAG3	DIAG4	DIAG5	DEPRN	...	NOPROVID	PHYS	PHYSASST	NPNMW	RNLPN	MHP
0	69.0	2.0	1.0	1.0	R918	J439	R599	J449	R090	0.0	...	0.0	1.0	0.0	0.0	0.0	0.0
1	64.0	1.0	1.0	1.0	J449	Z720	R918	R911	-9	0.0	...	0.0	1.0	0.0	0.0	0.0	0.0
2	81.0	1.0	1.0	2.0	G473	I489	I10-	-9	-9	0.0	...	0.0	1.0	0.0	0.0	0.0	0.0
3	66.0	1.0	1.0	2.0	J449	J961	-9	-9	-9	0.0	...	0.0	1.0	0.0	0.0	0.0	0.0
4	44.0	1.0	1.0	1.0	J449	R911	F172	I503	I340	0.0	...	0.0	1.0	0.0	0.0	0.0	0.0

5 rows × 24 columns

```
In [10]: df2.to_csv('_NAMCS_.csv', encoding='utf-8', index=False)
```

```
In [11]: df=pd.read_csv('_NAMCS_.csv', usecols=["AGE","SEX","RACER","PAYTYPER","DIAG1","DIAG2","DIAG3","DIAG4","DIAG5","DEPRN","
        "PSYCHOTH","NOPROVID","PHYS","PHYSASST","NPNMW","RNLPN","MHP","OTHPROV","PROVNO
        na_values = ['na', '-', '.', '', 'NaN'])

df.head()
```

```
Out[11]:
```

	AGE	SEX	RACER	PAYTYPER	DIAG1	DIAG2	DIAG3	DIAG4	DIAG5	DEPRN	...	NOPROVID	PHYS	PHYSASST	NPNMW	RNLPN	MHP
0	69.0	2.0	1.0	1.0	R918	J439	R599	J449	R090	0.0	...	0.0	1.0	0.0	0.0	0.0	0.0
1	64.0	1.0	1.0	1.0	J449	Z720	R918	R911	-9	0.0	...	0.0	1.0	0.0	0.0	0.0	0.0
2	81.0	1.0	1.0	2.0	G473	I489	I10-	-9	-9	0.0	...	0.0	1.0	0.0	0.0	0.0	0.0
3	66.0	1.0	1.0	2.0	J449	J961	-9	-9	-9	0.0	...	0.0	1.0	0.0	0.0	0.0	0.0
4	44.0	1.0	1.0	1.0	J449	R911	F172	I503	I340	0.0	...	0.0	1.0	0.0	0.0	0.0	0.0

5 rows × 24 columns

```
In [12]: df['SEQN'] = df.index + 11000
df.columns
```

```
Out[12]: Index(['AGE', 'SEX', 'RACER', 'PAYTYPER', 'DIAG1', 'DIAG2', 'DIAG3', 'DIAG4',
        'DIAG5', 'DEPRN', 'NOCHRON', 'DEPRESS', 'MENTAL', 'PSYCHOTH',
        'NOPROVID', 'PHYS', 'PHYSASST', 'NPNMW', 'RNLPN', 'MHP', 'OTHPROV',
        'PROVNONE', 'SPECCAT', 'OWNSR', 'SEQN'],
        dtype='object')
```

```
In [13]: df.reindex(columns=['SEQN', 'AGE', 'SEX', 'RACER', 'PAYTYPER', 'DIAG1', 'DIAG2', 'DIAG3', 'DIAG4',
        'DIAG5', 'DEPRN', 'NOCHRON', 'DEPRESS', 'MENTAL', 'PSYCHOTH',
        'NOPROVID', 'PHYS', 'PHYSASST', 'NPNMW', 'RNLPN', 'MHP', 'OTHPROV',
        'PROVNONE', 'SPECCAT', 'OWNSR' ])
```

```
Out[13]:
```

	SEQN	AGE	SEX	RACER	PAYTYPER	DIAG1	DIAG2	DIAG3	DIAG4	DIAG5	...	NOPROVID	PHYS	PHYSASST	NPNMW	RNLPN	M
0	11000	69.0	2.0	1.0	1.0	R918	J439	R599	J449	R090	...	0.0	1.0	0.0	0.0	0.0	
1	11001	64.0	1.0	1.0	1.0	J449	Z720	R918	R911	-9	...	0.0	1.0	0.0	0.0	0.0	
2	11002	81.0	1.0	1.0	2.0	G473	I489	I10-	-9	-9	...	0.0	1.0	0.0	0.0	0.0	
3	11003	66.0	1.0	1.0	2.0	J449	J961	-9	-9	-9	...	0.0	1.0	0.0	0.0	0.0	
4	11004	44.0	1.0	1.0	1.0	J449	R911	F172	I503	I340	...	0.0	1.0	0.0	0.0	0.0	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
8696	19696	19.0	1.0	3.0	1.0	R109	-9	-9	-9	-9	...	0.0	1.0	0.0	0.0	0.0	
8697	19697	93.0	1.0	1.0	2.0	K589	-9	-9	-9	-9	...	0.0	1.0	0.0	0.0	0.0	
8698	19698	32.0	2.0	1.0	1.0	K509	-9	-9	-9	-9	...	0.0	1.0	0.0	0.0	0.0	
8699	19699	65.0	1.0	1.0	2.0	K219	R05-	-9	-9	-9	...	0.0	1.0	0.0	0.0	0.0	
8700	19700	27.0	2.0	1.0	1.0	R101	-9	-9	-9	-9	...	0.0	1.0	0.0	0.0	0.0	

8701 rows × 25 columns

```
In [14]: # use of 'AGE' code for NAMCS Limits dataset to adults over 17 years old. No need to filter for adults.
df.count()
```

```
Out[14]: AGE      8701
SEX      8701
RACER    8701
PAYTYPER 8701
DIAG1    8701
DIAG2    8701
DIAG3    8701
DIAG4    8701
DIAG5    8701
DEPRN    8701
NOCHRON  8701
DEPRESS  8701
MENTAL   8701
PSYCHOTH 8701
NOPROVID 8701
PHYS     8701
PHYSASST 8701
NPNMW    8701
RNLPN    8701
MHP      8701
OTHPROV  8701
PROVNONE 8701
SPECCAT  8701
OWNSR    8701
SEQN     8701
dtype: int64
```

```
In [15]: df.isna().sum().sum()
```

```
Out[15]: 0
```

```
In [16]: df=pd.get_dummies(df, columns=['SEX'], drop_first=True)
```

```
In [17]: df=df.fillna(0)
```

```
In [18]: df.rename(columns = {'SEX_2.0':'SEXML'}, inplace = True)
df.columns
```

```
Out[18]: Index(['AGE', 'RACER', 'PAYTYPER', 'DIAG1', 'DIAG2', 'DIAG3', 'DIAG4', 'DIAG5',
        'DEPRN', 'NOCHRON', 'DEPRESS', 'MENTAL', 'PSYCHOTH', 'NOPROVID', 'PHYS',
        'PHYSASST', 'NPNMW', 'RNLPN', 'MHP', 'OTHPROV', 'PROVNONE', 'SPECCAT',
        'OWNSR', 'SEQN', 'SEXML'],
        dtype='object')
```

```
In [19]: df=pd.get_dummies(df, columns=['NOCHRON'], drop_first=False)
```

```
In [20]: df2=df.fillna(0)
df2.columns
```

```
Out[20]: Index(['AGE', 'RACER', 'PAYTYPER', 'DIAG1', 'DIAG2', 'DIAG3', 'DIAG4', 'DIAG5',
              'DEPRN', 'DEPRESS', 'MENTAL', 'PSYCHOTH', 'NOPROVID', 'PHYS',
              'PHYSASST', 'NPNMW', 'RNLPN', 'MHP', 'OTHPROV', 'PROVNONE', 'SPECCAT',
              'OWNSR', 'SEQN', 'SEXML', 'NOCHRON_0.0', 'NOCHRON_1.0', 'NOCHRON_2.0'],
              dtype='object')
```

```
In [21]: s=df2.pop('NOCHRON_0.0')
s=df2.pop('NOCHRON_2.0')
df2.columns
```

```
Out[21]: Index(['AGE', 'RACER', 'PAYTYPER', 'DIAG1', 'DIAG2', 'DIAG3', 'DIAG4', 'DIAG5',
              'DEPRN', 'DEPRESS', 'MENTAL', 'PSYCHOTH', 'NOPROVID', 'PHYS',
              'PHYSASST', 'NPNMW', 'RNLPN', 'MHP', 'OTHPROV', 'PROVNONE', 'SPECCAT',
              'OWNSR', 'SEQN', 'SEXML', 'NOCHRON_1.0'],
              dtype='object')
```

```
In [22]: df2=pd.get_dummies(df2, columns=['RACER'], drop_first=False)
df2.columns
```

```
Out[22]: Index(['AGE', 'PAYTYPER', 'DIAG1', 'DIAG2', 'DIAG3', 'DIAG4', 'DIAG5', 'DEPRN',
              'DEPRESS', 'MENTAL', 'PSYCHOTH', 'NOPROVID', 'PHYS', 'PHYSASST',
              'NPNMW', 'RNLPN', 'MHP', 'OTHPROV', 'PROVNONE', 'SPECCAT', 'OWNSR',
              'SEQN', 'SEXML', 'NOCHRON_1.0', 'RACER_1.0', 'RACER_2.0', 'RACER_3.0'],
              dtype='object')
```

```
In [23]: df3=df2.fillna(0)
```

```
In [24]: df3.head()
```

Out[24]:

	AGE	PAYTYPER	DIAG1	DIAG2	DIAG3	DIAG4	DIAG5	DEPRN	DEPRESS	MENTAL	...	OTHPROV	PROVNONE	SPECCAT	OWNSR	SEQ
0	69.0	1.0	R918	J439	R599	J449	R090	0.0	0.0	0.0	...	0.0	0.0	3.0	1.0	1100
1	64.0	1.0	J449	Z720	R918	R911	-9	0.0	0.0	0.0	...	0.0	0.0	3.0	1.0	1100
2	81.0	2.0	G473	I489	I10-	-9	-9	0.0	0.0	0.0	...	0.0	0.0	3.0	1.0	1100
3	66.0	2.0	J449	J961	-9	-9	-9	0.0	0.0	0.0	...	0.0	0.0	3.0	1.0	1100
4	44.0	1.0	J449	R911	F172	I503	I340	0.0	0.0	0.0	...	0.0	0.0	3.0	1.0	1100

5 rows × 27 columns

In [25]: `df3.rename(columns = { 'NOCHRON_1.0': 'NOCHRON', 'RACER_1.0': 'RACEWH', 'RACER_2.0': 'RACEBL', 'RACER_3.0': 'RACEOT' }, inplace = True)`

In [26]: `df3.columns`

Out[26]: Index(['AGE', 'PAYTYPER', 'DIAG1', 'DIAG2', 'DIAG3', 'DIAG4', 'DIAG5', 'DEPRN', 'DEPRESS', 'MENTAL', 'PSYCHOTH', 'NOPROVID', 'PHYS', 'PHYSASST', 'NPNMW', 'RNLPN', 'MHP', 'OTHPROV', 'PROVNONE', 'SPECCAT', 'OWNSR', 'SEQN', 'SEXML', 'NOCHRON', 'RACEWH', 'RACEBL', 'RACEOT'], dtype='object')

In [27]: `df4=pd.get_dummies(df3, columns=['PAYTYPER'], drop_first=False)`
`df4.columns`

Out[27]: Index(['AGE', 'DIAG1', 'DIAG2', 'DIAG3', 'DIAG4', 'DIAG5', 'DEPRN', 'DEPRESS', 'MENTAL', 'PSYCHOTH', 'NOPROVID', 'PHYS', 'PHYSASST', 'NPNMW', 'RNLPN', 'MHP', 'OTHPROV', 'PROVNONE', 'SPECCAT', 'OWNSR', 'SEQN', 'SEXML', 'NOCHRON', 'RACEWH', 'RACEBL', 'RACEOT', 'PAYTYPER_-9.0', 'PAYTYPER_-8.0', 'PAYTYPER_1.0', 'PAYTYPER_2.0', 'PAYTYPER_3.0', 'PAYTYPER_4.0', 'PAYTYPER_5.0', 'PAYTYPER_6.0', 'PAYTYPER_7.0'], dtype='object')

In [28]: `df4=df4.replace(r'^\s*$', np.nan, regex=True)`
`df4.columns`

```
Out[28]: Index(['AGE', 'DIAG1', 'DIAG2', 'DIAG3', 'DIAG4', 'DIAG5', 'DEPRN', 'DEPRESS',
        'MENTAL', 'PSYCHOTH', 'NOPROVID', 'PHYS', 'PHYSASST', 'NPNMW', 'RNLPN',
        'MHP', 'OTHPROV', 'PROVNONE', 'SPECCAT', 'OWNSR', 'SEQN', 'SEXML',
        'NOCHRON', 'RACEWH', 'RACEBL', 'RACEOT', 'PAYTYPER_-9.0',
        'PAYTYPER_-8.0', 'PAYTYPER_1.0', 'PAYTYPER_2.0', 'PAYTYPER_3.0',
        'PAYTYPER_4.0', 'PAYTYPER_5.0', 'PAYTYPER_6.0', 'PAYTYPER_7.0'],
        dtype='object')
```

```
In [29]: df5=df4.fillna(0)
        df5.columns
```

```
Out[29]: Index(['AGE', 'DIAG1', 'DIAG2', 'DIAG3', 'DIAG4', 'DIAG5', 'DEPRN', 'DEPRESS',
        'MENTAL', 'PSYCHOTH', 'NOPROVID', 'PHYS', 'PHYSASST', 'NPNMW', 'RNLPN',
        'MHP', 'OTHPROV', 'PROVNONE', 'SPECCAT', 'OWNSR', 'SEQN', 'SEXML',
        'NOCHRON', 'RACEWH', 'RACEBL', 'RACEOT', 'PAYTYPER_-9.0',
        'PAYTYPER_-8.0', 'PAYTYPER_1.0', 'PAYTYPER_2.0', 'PAYTYPER_3.0',
        'PAYTYPER_4.0', 'PAYTYPER_5.0', 'PAYTYPER_6.0', 'PAYTYPER_7.0'],
        dtype='object')
```

```
In [30]: df5.rename(columns = {'PAYTYPER_2.0': "INSMCARE", 'PAYTYPER_3.0': 'INSSTATE'}, inplace = True)
        df5.columns
```

```
Out[30]: Index(['AGE', 'DIAG1', 'DIAG2', 'DIAG3', 'DIAG4', 'DIAG5', 'DEPRN', 'DEPRESS',
        'MENTAL', 'PSYCHOTH', 'NOPROVID', 'PHYS', 'PHYSASST', 'NPNMW', 'RNLPN',
        'MHP', 'OTHPROV', 'PROVNONE', 'SPECCAT', 'OWNSR', 'SEQN', 'SEXML',
        'NOCHRON', 'RACEWH', 'RACEBL', 'RACEOT', 'PAYTYPER_-9.0',
        'PAYTYPER_-8.0', 'PAYTYPER_1.0', 'INSMCARE', 'INSSTATE', 'PAYTYPER_4.0',
        'PAYTYPER_5.0', 'PAYTYPER_6.0', 'PAYTYPER_7.0'],
        dtype='object')
```

```
In [31]: def combinePayColumns(row):

        pay = ['PAYTYPER_-9.0', 'PAYTYPER_-8.0', 'PAYTYPER_5.0', 'PAYTYPER_6.0', 'PAYTYPER_7.0']
        for i in pay:
            if row[i] == 0:
                return '0'
            else:
                return '1'
```

```
In [32]: df5['INSNOCH'] =df5.apply(lambda row: combinePayColumns(row), axis =1)
```

```
In [33]: df5.columns
```

```
Out[33]: Index(['AGE', 'DIAG1', 'DIAG2', 'DIAG3', 'DIAG4', 'DIAG5', 'DEPRN', 'DEPRESS',
        'MENTAL', 'PSYCHOTH', 'NOPROVID', 'PHYS', 'PHYSASST', 'NPNMW', 'RNLPN',
        'MHP', 'OTHPROV', 'PROVNONE', 'SPECCAT', 'OWNSR', 'SEQN', 'SEXML',
        'NOCHRON', 'RACEWH', 'RACEBL', 'RACEOT', 'PAYTYPER_-9.0',
        'PAYTYPER_-8.0', 'PAYTYPER_1.0', 'INSMCARE', 'INSSTATE', 'PAYTYPER_4.0',
        'PAYTYPER_5.0', 'PAYTYPER_6.0', 'PAYTYPER_7.0', 'INSNOCH'],
        dtype='object')
```

```
In [34]: s=df5.pop('PAYTYPER_-9.0')
s=df5.pop('PAYTYPER_-8.0')
s=df5.pop('PAYTYPER_5.0')
s=df5.pop('PAYTYPER_6.0')
s=df5.pop('PAYTYPER_7.0')
#dropping individual groups for other payers
df5.columns
```

```
Out[34]: Index(['AGE', 'DIAG1', 'DIAG2', 'DIAG3', 'DIAG4', 'DIAG5', 'DEPRN', 'DEPRESS',
        'MENTAL', 'PSYCHOTH', 'NOPROVID', 'PHYS', 'PHYSASST', 'NPNMW', 'RNLPN',
        'MHP', 'OTHPROV', 'PROVNONE', 'SPECCAT', 'OWNSR', 'SEQN', 'SEXML',
        'NOCHRON', 'RACEWH', 'RACEBL', 'RACEOT', 'PAYTYPER_1.0', 'INSMCARE',
        'INSSTATE', 'PAYTYPER_4.0', 'INSNOCH'],
        dtype='object')
```

```
In [35]: def combinePrvtColumns(row):

    pay1 = ['PAYTYPER_1.0', 'PAYTYPER_4.0']
    for i in pay1:
        if row[i] ==0:
            return '0'
        else:
            return '1'
```

```
In [36]: df5['INSPRV'] =df5.apply(lambda row: combinePrvtColumns(row), axis =1)
```

```
In [37]: s=df5.pop('PAYTYPER_1.0')
s=df5.pop('PAYTYPER_4.0')
```

```
In [38]: df5.head()
```

Out[38]:

	AGE	DIAG1	DIAG2	DIAG3	DIAG4	DIAG5	DEPRN	DEPRESS	MENTAL	PSYCHOTH	...	SEQN	SEXML	NOCHRON	RACEWH	RACEBL
0	69.0	R918	J439	R599	J449	R090	0.0	0.0	0.0	0.0	...	11000	1	0	1	0
1	64.0	J449	Z720	R918	R911	-9	0.0	0.0	0.0	0.0	...	11001	0	0	1	0
2	81.0	G473	I489	I10-	-9	-9	0.0	0.0	0.0	0.0	...	11002	0	0	1	0
3	66.0	J449	J961	-9	-9	-9	0.0	0.0	0.0	0.0	...	11003	0	0	1	0
4	44.0	J449	R911	F172	I503	I340	0.0	0.0	0.0	0.0	...	11004	0	0	1	0

5 rows × 30 columns

In [39]: `df6=pd.get_dummies(df5, columns=['SPECCAT', 'OWNSR'], drop_first=False)`
`df6.columns`

Out[39]: Index(['AGE', 'DIAG1', 'DIAG2', 'DIAG3', 'DIAG4', 'DIAG5', 'DEPRN', 'DEPRESS',
'MENTAL', 'PSYCHOTH', 'NOPROVID', 'PHYS', 'PHYSASST', 'NPNMW', 'RNLPN',
'MHP', 'OTHPROV', 'PROVNONE', 'SEQN', 'SEXML', 'NOCHRON', 'RACEWH',
'RACEBL', 'RACEOT', 'INSMCARE', 'INSSTATE', 'INSNOCH', 'INSPRVT',
'SPECCAT_1.0', 'SPECCAT_2.0', 'SPECCAT_3.0', 'OWNSR_-9.0', 'OWNSR_-8.0',
'OWNSR_1.0', 'OWNSR_2.0', 'OWNSR_3.0'],
dtype='object')

In [40]: `df6.rename(columns = {'SPECCAT_1.0': 'SPPRC', 'SPECCAT_2.0': 'SPSUR', 'SPECCAT_3.0': 'SPMEC', 'OWNSR_-9.0': "OWNMISS",
, 'OWNSR_-8.0': 'OWNUNKN', 'OWNSR_1.0': 'OWNPHYS', 'OWNSR_2.0': 'OWNHOSP', 'OWNSR_3.0': 'OWNINSR'
, inplace = True)`
`#OWNMISS = missing response`

In [41]: `df6.columns`

Out[41]: Index(['AGE', 'DIAG1', 'DIAG2', 'DIAG3', 'DIAG4', 'DIAG5', 'DEPRN', 'DEPRESS',
'MENTAL', 'PSYCHOTH', 'NOPROVID', 'PHYS', 'PHYSASST', 'NPNMW', 'RNLPN',
'MHP', 'OTHPROV', 'PROVNONE', 'SEQN', 'SEXML', 'NOCHRON', 'RACEWH',
'RACEBL', 'RACEOT', 'INSMCARE', 'INSSTATE', 'INSNOCH', 'INSPRVT',
'SPPRC', 'SPSUR', 'SPMEC', 'OWNMISS', 'OWNUNKN', 'OWNPHYS', 'OWNHOSP',
'OWNINSR'],
dtype='object')

In [42]: `df6.head()`

Out[42]:

	AGE	DIAG1	DIAG2	DIAG3	DIAG4	DIAG5	DEPRN	DEPRESS	MENTAL	PSYCHOTH	...	INSNOCH	INSPRVT	SPPRC	SPSUR	SPMEC	C
0	69.0	R918	J439	R599	J449	R090	0.0	0.0	0.0	0.0	...	0	1	0	0	1	
1	64.0	J449	Z720	R918	R911	-9	0.0	0.0	0.0	0.0	...	0	1	0	0	1	
2	81.0	G473	I489	I10-	-9	-9	0.0	0.0	0.0	0.0	...	0	0	0	0	1	
3	66.0	J449	J961	-9	-9	-9	0.0	0.0	0.0	0.0	...	0	0	0	0	1	
4	44.0	J449	R911	F172	I503	I340	0.0	0.0	0.0	0.0	...	0	1	0	0	1	

5 rows × 36 columns

In [43]: df0=(df6['OWNMISS'].value_counts(normalize=True))
df0

Out[43]:

```
0    0.957936
1    0.042064
Name: OWNMISS, dtype: float64
```

In [44]: s2=df0.round(2)
print('Missing Provider Ownership Type df =', (s[1]*100), '%')

Missing Provider Ownership Type df = 0 %

In [45]: s3 = df6.pop('OWNMISS')
#dropping OWNMISS df since less than 10% missing

In [46]: df6.columns

Out[46]: Index(['AGE', 'DIAG1', 'DIAG2', 'DIAG3', 'DIAG4', 'DIAG5', 'DEPRN', 'DEPRESS',
'MENTAL', 'PSYCHOTH', 'NOPROVID', 'PHYS', 'PHYSASST', 'NPNMW', 'RNLPN',
'MHP', 'OTHPROV', 'PROVNONE', 'SEQN', 'SEXML', 'NOCHRON', 'RACEWH',
'RACEBL', 'RACEOT', 'INSMCARE', 'INSSTATE', 'INSNOCH', 'INSPRVT',
'SPPRC', 'SPSUR', 'SPMEC', 'OWNUNKN', 'OWNPHYS', 'OWNHOSP', 'OWNINSR'],
dtype='object')

In [47]: df6['DIAGALL'] = df6['DIAG1'] + " " + df6['DIAG2'] + " " + df6['DIAG3'] + " " + df6['DIAG4'] + " " + df6['DIAG5']

df6.head()

Out[47]:

	AGE	DIAG1	DIAG2	DIAG3	DIAG4	DIAG5	DEPRN	DEPRESS	MENTAL	PSYCHOTH	...	INSNOCH	INSPRVT	SPPRC	SPSUR	SPMEC	C
--	-----	-------	-------	-------	-------	-------	-------	---------	--------	----------	-----	---------	---------	-------	-------	-------	---

0	69.0	R918	J439	R599	J449	R090	0.0	0.0	0.0	0.0	...	0	1	0	0	1
---	------	------	------	------	------	------	-----	-----	-----	-----	-----	---	---	---	---	---

1	64.0	J449	Z720	R918	R911	-9	0.0	0.0	0.0	0.0	...	0	1	0	0	1
---	------	------	------	------	------	----	-----	-----	-----	-----	-----	---	---	---	---	---

2	81.0	G473	I489	I10-	-9	-9	0.0	0.0	0.0	0.0	...	0	0	0	0	1
---	------	------	------	------	----	----	-----	-----	-----	-----	-----	---	---	---	---	---

3	66.0	J449	J961	-9	-9	-9	0.0	0.0	0.0	0.0	...	0	0	0	0	1
---	------	------	------	----	----	----	-----	-----	-----	-----	-----	---	---	---	---	---

4	44.0	J449	R911	F172	I503	I340	0.0	0.0	0.0	0.0	...	0	1	0	0	1
---	------	------	------	------	------	------	-----	-----	-----	-----	-----	---	---	---	---	---

5 rows × 36 columns

```
In [48]: s4=df6.pop('DIAG1')
s4=df6.pop('DIAG2')
s4=df6.pop('DIAG3')
s4=df6.pop('DIAG4')
s4=df6.pop('DIAG5')
```

```
In [49]: df6['DGRP1']=df6['DIAGALL'].str.extract(r'^([A,B]){1}')
df6['DGRP2']=df6['DIAGALL'].str.extract(r'^([C,D]){1}')
df6['DGRP3']=df6['DIAGALL'].str.extract(r'^([E]){1}')
df6['DGRP4']=df6['DIAGALL'].str.extract(r'^([G]){1}')
df6['DGRP5']=df6['DIAGALL'].str.extract(r'^([H]){1}')
df6['DGRP6']=df6['DIAGALL'].str.extract(r'^([I,J]){1}')
df6['DGRP7']=df6['DIAGALL'].str.extract(r'^([K]){1}')
df6['DGRP8']=df6['DIAGALL'].str.extract(r'^([L]){1}')
df6['DGRP9']=df6['DIAGALL'].str.extract(r'^([M]){1}')
```

```
df6['DGRP10']=df6['DIAGALL'].str.extract(r'^{N}[1]')
df6['DGRP11']=df6['DIAGALL'].str.extract(r'^{O,P}[1]')
df6['DGRP12']=df6['DIAGALL'].str.extract(r'^{Q}[1]')
df6['DGRP13']=df6['DIAGALL'].str.extract(r'^{R}[1]')
df6['DGRP14']=df6['DIAGALL'].str.extract(r'^{S,T}[1]')
df6['DGRP15']=df6['DIAGALL'].str.extract(r'^{V,W,X,Y}[1]')
df6['DGRP16']=df6['DIAGALL'].str.extract(r'^{Z}[1]')
```

```
In [50]: df7=pd.get_dummies(df6, columns=['DGRP1', 'DGRP2', 'DGRP3', 'DGRP4', 'DGRP5',
      'DGRP6', 'DGRP7', 'DGRP8', 'DGRP9', 'DGRP10', 'DGRP11',
      'DGRP12', 'DGRP13', 'DGRP14','DGRP16'], drop_first=False)
```

```
In [51]: s5=df7.pop('DGRP1_B')
s5=df7.pop('DGRP2_D')
s5=df7.pop('DGRP6_J')
s5=df7.pop('DGRP14_T')
#dropping first columns for each group
df7.columns
```

```
Out[51]: Index(['AGE', 'DEPRN', 'DEPRESS', 'MENTAL', 'PSYCHOTH', 'NOPROVID', 'PHYS',
      'PHYSASST', 'NPNMW', 'RNLPN', 'MHP', 'OTHPROV', 'PROVNONE', 'SEQN',
      'SEXML', 'NOCHRON', 'RACEWH', 'RACEBL', 'RACEOT', 'INSMCARE',
      'INSSTATE', 'INSNOCH', 'INSPRVT', 'SPPRC', 'SPSUR', 'SPMEC', 'OWNUNKN',
      'OWNPHYS', 'OWNHOSP', 'OWNINSR', 'DIAGALL', 'DGRP15', 'DGRP1_A',
      'DGRP2_C', 'DGRP3_E', 'DGRP4_G', 'DGRP5_H', 'DGRP6_I', 'DGRP7_K',
      'DGRP8_L', 'DGRP9_M', 'DGRP10_N', 'DGRP11_O', 'DGRP12_Q', 'DGRP13_R',
      'DGRP14_S', 'DGRP16_Z'],
      dtype='object')
```

```
In [52]: df7['temp']=df7['DIAGALL'].str.extract(r'^{F32*,F33*,F43*}')
df7.columns
```

```
Out[52]: Index(['AGE', 'DEPRN', 'DEPRESS', 'MENTAL', 'PSYCHOTH', 'NOPROVID', 'PHYS',
      'PHYSASST', 'NPNMW', 'RNLPN', 'MHP', 'OTHPROV', 'PROVNONE', 'SEQN',
      'SEXML', 'NOCHRON', 'RACEWH', 'RACEBL', 'RACEOT', 'INSMCARE',
      'INSSTATE', 'INSNOCH', 'INSPRVT', 'SPPRC', 'SPSUR', 'SPMEC', 'OWNUNKN',
      'OWNPHYS', 'OWNHOSP', 'OWNINSR', 'DIAGALL', 'DGRP15', 'DGRP1_A',
      'DGRP2_C', 'DGRP3_E', 'DGRP4_G', 'DGRP5_H', 'DGRP6_I', 'DGRP7_K',
      'DGRP8_L', 'DGRP9_M', 'DGRP10_N', 'DGRP11_O', 'DGRP12_Q', 'DGRP13_R',
      'DGRP14_S', 'DGRP16_Z', 'temp'],
      dtype='object')
```

```
In [53]: df8=df7.fillna(0)
```

```
In [54]: def DepInd(row):

    if row['temp'] == 'F':
        return '1'

    elif row['MENTAL'] != 0:
        return '1'

    elif row['DEPRN'] != 0:
        return '1'

    elif row['DEPRESS'] != 0:
        return '1'

    elif row['PSYCHOTH'] != 0:
        return '1'
    else:
        return '0'
```

```
In [55]: df8['DI'] =df8.apply(lambda row: DepInd(row), axis =1)
```

```
In [56]: df8.rename(columns = {'DGRP1_A': 'DGRP1', 'DGRP2_C': 'DGRP2', 'DGRP3_E': 'DGRP3', 'DGRP4_G': 'DGRP4', 'DGRP5_H': 'DGRP5'},
df8.head()
```

```
Out[56]:
```

	AGE	DEPRN	DEPRESS	MENTAL	PSYCHOTH	NOPROVID	PHYS	PHYSASST	NPNMW	RNLPN	...	DGRP8	DGRP9	DGRP10	DGRP11	DGRP12
0	69.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	...	0	0	0	0	0
1	64.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	...	0	0	0	0	0
2	81.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	...	0	0	0	0	0
3	66.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	...	0	0	0	0	0
4	44.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	...	0	0	0	0	0

5 rows × 49 columns

```
In [57]: df8.value_counts('DI')
```

```
Out[57]: DI
0      7357
1      1344
dtype: int64
```

```
In [58]: s6=df8.pop('temp')
```

```
In [59]: df8.columns
```

```
Out[59]: Index(['AGE', 'DEPRN', 'DEPRESS', 'MENTAL', 'PSYCHOTH', 'NOPROVID', 'PHYS',
               'PHYSASST', 'NPNMW', 'RNLPN', 'MHP', 'OTHPROV', 'PROVNONE', 'SEQN',
               'SEXML', 'NOCHRON', 'RACEWH', 'RACEBL', 'RACEOT', 'INSMCARE',
               'INSSTATE', 'INSNOCH', 'INSPRVT', 'SPPRC', 'SPSUR', 'SPMEC', 'OWNUNKN',
               'OWNPHYS', 'OWNHOSP', 'OWNINSR', 'DIAGALL', 'DGRP15', 'DGRP1', 'DGRP2',
               'DGRP3', 'DGRP4', 'DGRP5', 'DGRP6', 'DGRP7', 'DGRP8', 'DGRP9', 'DGRP10',
               'DGRP11', 'DGRP12', 'DGRP13', 'DGRP14', 'DGRP16', 'DI'],
              dtype='object')
```

```
In [60]: df9=df8.reindex(columns=['SEQN', 'AGE', 'SEXML', 'RACEWH', 'RACEBL', 'RACEOT', 'NOPROVID', 'PHYS',
               'PHYSASST', 'NPNMW', 'RNLPN', 'MHP', 'OTHPROV', 'PROVNONE', 'INSMCARE',
               'INSSTATE', 'INSNOCH', 'INSPRVT', 'SPPRC', 'SPSUR', 'SPMEC', 'OWNUNKN',
               'OWNPHYS', 'OWNHOSP', 'OWNINSR', 'DIAGALL', 'NOCHRON', 'DGRP1', 'DGRP2',
               'DGRP3', 'DGRP4', 'DGRP5', 'DGRP6', 'DGRP7', 'DGRP8', 'DGRP9', 'DGRP10',
               'DGRP11', 'DGRP12', 'DGRP13', 'DGRP14', 'DGRP15', 'DGRP16', 'DEPRN', 'DEPRESS', 'MENTAL', 'PSYCHOTH', 'DI'])
```

```
In [61]: df9.columns
```

```
Out[61]: Index(['SEQN', 'AGE', 'SEXML', 'RACEWH', 'RACEBL', 'RACEOT', 'NOPROVID',
               'PHYS', 'PHYSASST', 'NPNMW', 'RNLPN', 'MHP', 'OTHPROV', 'PROVNONE',
               'INSMCARE', 'INSSTATE', 'INSNOCH', 'INSPRVT', 'SPPRC', 'SPSUR', 'SPMEC',
               'OWNUNKN', 'OWNPHYS', 'OWNHOSP', 'OWNINSR', 'DIAGALL', 'NOCHRON',
               'DGRP1', 'DGRP2', 'DGRP3', 'DGRP4', 'DGRP5', 'DGRP6', 'DGRP7', 'DGRP8',
               'DGRP9', 'DGRP10', 'DGRP11', 'DGRP12', 'DGRP13', 'DGRP14', 'DGRP15',
               'DGRP16', 'DEPRN', 'DEPRESS', 'MENTAL', 'PSYCHOTH', 'DI'],
              dtype='object')
```

```
In [62]: df9.to_csv('namcs_features_FINAL.csv', encoding='utf-8', index=False)
```

NHANES

```
In [63]: #read SAS(xport) file
df = pd.read_sas('DEMO_J.XPT')
df.head()
```

```
Out[63]:
```

	SEQN	SDDSRVYR	RIDSTATR	RIAGENDR	RIDAGEYR	RIDAGEMN	RIDRETH1	RIDRETH3	RIDEXMON	RIDEXAGM	...	DMDHREDZ	DMI
0	93703.0	10.0	2.0	2.0	2.0	NaN	5.0	6.0	2.0	27.0	...	3.0	
1	93704.0	10.0	2.0	1.0	2.0	NaN	3.0	3.0	1.0	33.0	...	3.0	
2	93705.0	10.0	2.0	2.0	66.0	NaN	4.0	4.0	2.0	NaN	...	1.0	
3	93706.0	10.0	2.0	1.0	18.0	NaN	5.0	6.0	2.0	222.0	...	3.0	
4	93707.0	10.0	2.0	1.0	13.0	NaN	5.0	7.0	2.0	158.0	...	2.0	

5 rows × 46 columns

```
In [64]: #convert SAS file to CSV with scientific values in float format
df.to_csv('nhanes_demo.csv', float_format='{:f}'.format, encoding='utf-8', index=False)
```

```
In [65]: df1 = pd.read_sas('DLQ_J.XPT')
df1.head()
```

```
Out[65]:
```

	SEQN	DLQ010	DLQ020	DLQ040	DLQ050	DLQ060	DLQ080	DLQ100	DLQ110	DLQ130	DLQ140	DLQ150	DLQ170
0	93703.0	2.0	2.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
1	93704.0	2.0	2.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
2	93705.0	2.0	2.0	2.0	2.0	2.0	2.0	4.0	2.0	1.0	4.0	2.0	1.0
3	93706.0	2.0	2.0	2.0	2.0	2.0	2.0	5.0	2.0	NaN	5.0	2.0	NaN
4	93707.0	2.0	2.0	2.0	2.0	2.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN

```
In [66]: df1.to_csv('nhanes_disability.csv', float_format='{:f}'.format, encoding='utf-8', index=False)
```

```
In [67]: df2 = pd.read_sas('DPQ_J.XPT')
```

```
In [68]: df2.head()
```

Out[68]:

	SEQN	DPQ010	DPQ020	DPQ030	DPQ040	DPQ050	DPQ060	DPQ070	DPQ080	DPQ090	DPQ100
0	93705.0	5.397605e-79	5.397605e-79	5.397605e-79	5.397605e-79	5.397605e-79	5.397605e-79	5.397605e-79	5.397605e-79	5.397605e-79	NaN
1	93706.0	5.397605e-79	5.397605e-79	5.397605e-79	5.397605e-79	5.397605e-79	5.397605e-79	5.397605e-79	5.397605e-79	5.397605e-79	NaN
2	93708.0	5.397605e-79	5.397605e-79	5.397605e-79	5.397605e-79	5.397605e-79	5.397605e-79	5.397605e-79	5.397605e-79	5.397605e-79	NaN
3	93709.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
4	93711.0	1.000000e+00	5.397605e-79	1.000000e+00	5.397605e-79	5.397605e-79	5.397605e-79	5.397605e-79	5.397605e-79	5.397605e-79	5.397605e-79

In [69]: `df2.to_csv('nhanes_depression_screensers.csv', float_format='{:.f}'.format, encoding='utf-8', index=False)`

In [70]: `df3 = pd.read_sas('HIQ_J.XPT')`
`df3.head()`

Out[70]:

	SEQN	HIQ011	HIQ031A	HIQ031B	HIQ031C	HIQ031D	HIQ031E	HIQ031F	HIQ031H	HIQ031I	HIQ031J	HIQ031AA	HIQ260	HIQ105
0	93703.0	1.0	14.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
1	93704.0	1.0	14.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
2	93705.0	1.0	NaN	15.0	NaN	17.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	1.0
3	93706.0	1.0	14.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
4	93707.0	1.0	NaN	NaN	NaN	17.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

In [71]: `df3.to_csv('nhanes_health_insurance.csv', float_format='{:.f}'.format, encoding='utf-8', index=False)`

In [72]: `df4 = pd.read_sas('MCQ_J.XPT')`
`df4.head()`

Out[72]:

	SEQN	MCQ010	MCQ025	MCQ035	MCQ040	MCQ050	AGQ030	MCQ053	MCQ080	MCQ092	...	MCQ300A	MCQ366A	MCQ366B	M
0	93703.0	2.0	NaN	NaN	NaN	NaN	NaN	2.0	NaN	NaN	...	NaN	NaN	NaN	
1	93704.0	2.0	NaN	NaN	NaN	NaN	NaN	2.0	NaN	NaN	...	NaN	NaN	NaN	
2	93705.0	1.0	10.0	2.0	NaN	NaN	NaN	2.0	2.0	2.0	...	2.0	2.0	2.0	
3	93706.0	2.0	NaN	NaN	NaN	NaN	NaN	2.0	2.0	2.0	...	NaN	2.0	2.0	
4	93707.0	2.0	NaN	NaN	NaN	NaN	NaN	2.0	NaN	2.0	...	NaN	NaN	NaN	

5 rows × 76 columns

```
In [73]: df4.to_csv('nhanes_medical_conditions.csv', float_format='{:.f}'.format, encoding='utf-8', index=False)
```

```
In [74]: from functools import reduce
df5=pd.read_csv('nhanes_depression_screeners.csv')
df6=pd.read_csv('nhanes_demo.csv')
df7=pd.read_csv('nhanes_medical_conditions.csv')
df8=pd.read_csv('nhanes_disability.csv')
df9=pd.read_csv('nhanes_health_insurance.csv')
#define List of DataFrames
dfs = [df5, df6, df7, df8, df9]

#merge all DataFrames into one
final_df = reduce(lambda left,right: pd.merge(left,right,on=['SEQN'],
                                              how='outer'), dfs)
```

```
In [75]: #view merged DataFrame
final_df.head()
```

Out[75]:

	SEQN	DPQ010	DPQ020	DPQ030	DPQ040	DPQ050	DPQ060	DPQ070	DPQ080	DPQ090	...	HIQ031E	HIQ031F	HIQ031H	HIQ031I
0	93705.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	NaN	NaN	NaN	NaN
1	93706.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	NaN	NaN	NaN	NaN
2	93708.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	NaN	NaN	NaN	NaN
3	93709.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	NaN	NaN	NaN
4	93711.0	1.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	...	NaN	NaN	21.0	NaN

5 rows × 158 columns



In [76]: final_df.drop_duplicates(keep=False)

Out[76]:

	SEQN	DPQ010	DPQ020	DPQ030	DPQ040	DPQ050	DPQ060	DPQ070	DPQ080	DPQ090	...	HIQ031E	HIQ031F	HIQ031H	HIQ
0	93705.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	NaN	NaN	NaN	
1	93706.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	NaN	NaN	NaN	
2	93708.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	NaN	NaN	NaN	
3	93709.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	NaN	NaN	
4	93711.0	1.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	...	NaN	NaN	21.0	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
9249	102942.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	NaN	NaN	
9250	102945.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	NaN	21.0	
9251	102950.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	19.0	21.0	
9252	102951.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	NaN	NaN	
9253	102955.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	NaN	NaN	NaN	

9254 rows × 158 columns



```
In [77]: final_df.to_csv('NHANES_merged.csv', encoding='utf-8', index=False)
```

```
In [78]: df=pd.read_csv('NHANES_merged.csv', usecols=['SEQN', 'DPQ010', 'DPQ020', 'DPQ030', 'DPQ040', 'DPQ050', 'DPQ060', 'DPQ070',
'DMDEDUC2', 'DLQ140', 'DLQ150', 'DLQ170', 'MCQ010', 'MCQ080', 'MCQ160A', 'M
'MCQ160F', 'MCQ160M', 'MCQ160G', 'MCQ160K', 'MCQ160O', 'MCQ160L', 'MCQ170L
'MCQ510C', 'MCQ510D', 'MCQ510E', 'MCQ510F', 'MCQ220', 'INDHHIN2', 'HIQ031A
na_values = ['na', '-', '.', '', 'NaN'])
```

```
In [79]: df.head()
```

```
Out[79]:
```

	SEQN	DPQ010	DPQ020	DPQ030	DPQ040	DPQ050	DPQ060	DPQ070	DPQ080	DPQ090	...	DLQ140	DLQ150	DLQ170	HIQ031A
0	93705.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	4.0	2.0	1.0	NaN
1	93706.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	5.0	2.0	NaN	14.0
2	93708.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	5.0	2.0	NaN	NaN
3	93709.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	1.0	1.0	1.0	NaN
4	93711.0	1.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	...	5.0	2.0	NaN	NaN

5 rows × 48 columns

```
In [80]: #limiting dataset to age adults only per proposal
df=df[df['RIDAGEYR'] > 17]
df.count()
```

```

Out[80]:
SEQN      5856
DPQ010    5094
DPQ020    5093
DPQ030    5093
DPQ040    5092
DPQ050    5092
DPQ060    5091
DPQ070    5091
DPQ080    5091
DPQ090    5090
RIAGENDR  5856
RIDAGEYR  5856
RIDRETH3  5856
DMEDEDUC3    287
DMEDEDUC2  5569
INDHHIN2  5507
MCQ010      5856
MCQ080      5856
MCQ160A     5569
MCQ160B     5569
MCQ160C     5569
MCQ160D     5569
MCQ160E     5569
MCQ160F     5569
MCQ160M     5569
MCQ160G     5569
MCQ160K     5569
MCQ160O     5569
MCQ160L     5569
MCQ170L     294
MCQ500      287
MCQ510A     139
MCQ510B      6
MCQ510C     24
MCQ510D     50
MCQ510E     12
MCQ510F     78
MCQ220     5569
DLQ140     5762
DLQ150     5762
DLQ170     3037
HIQ031A     2892
HIQ031B     1573
HIQ031D     1025
HIQ031E      2

```

```
HIQ031H      233  
HIQ031I      220  
HIQ031AA       1  
dtype: int64
```

```
In [81]: df.isna().sum()
```

```

Out[81]:
SEQN      0
DPQ010    762
DPQ020    763
DPQ030    763
DPQ040    764
DPQ050    764
DPQ060    765
DPQ070    765
DPQ080    765
DPQ090    766
RIAGENDR   0
RIDAGEYR   0
RIDRETH3   0
DMDEDUC3  5569
DMDEDUC2   287
INDHHIN2   349
MCQ010     0
MCQ080     0
MCQ160A    287
MCQ160B    287
MCQ160C    287
MCQ160D    287
MCQ160E    287
MCQ160F    287
MCQ160M    287
MCQ160G    287
MCQ160K    287
MCQ160O    287
MCQ160L    287
MCQ170L    5562
MCQ500     5569
MCQ510A    5717
MCQ510B    5850
MCQ510C    5832
MCQ510D    5806
MCQ510E    5844
MCQ510F    5778
MCQ220     287
DLQ140     94
DLQ150     94
DLQ170    2819
HIQ031A    2964
HIQ031B    4283
HIQ031D    4831
HIQ031E    5854

```

```
HIQ031H    5623
HIQ031I    5636
HIQ031AA   5855
dtype: int64
```

```
In [82]: #replacing blank values with nan
df.replace(r'^\s*$', np.nan, regex=True)
```

```
Out[82]:
```

	SEQN	DPQ010	DPQ020	DPQ030	DPQ040	DPQ050	DPQ060	DPQ070	DPQ080	DPQ090	...	DLQ140	DLQ150	DLQ170	HIQ031
0	93705.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	4.0	2.0	1.0	Na
1	93706.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	5.0	2.0	NaN	14
2	93708.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	5.0	2.0	NaN	Na
3	93709.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	1.0	1.0	1.0	Na
4	93711.0	1.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	...	5.0	2.0	NaN	Na
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
9175	102751.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	5.0	2.0	NaN	14
9193	102804.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	4.0	2.0	1.0	Na
9203	102825.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	3.0	1.0	3.0	Na
9247	102940.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	4.0	2.0	1.0	Na
9251	102950.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	3.0	2.0	1.0	Na

5856 rows × 48 columns

```
In [83]: df2=df.fillna(0)
```

```
In [84]: df2.isna().sum()
```

```
Out[84]: SEQN      0
          DPQ010    0
          DPQ020    0
          DPQ030    0
          DPQ040    0
          DPQ050    0
          DPQ060    0
          DPQ070    0
          DPQ080    0
          DPQ090    0
          RIAGENDR   0
          RIDAGEYR   0
          RIDRETH3   0
          DMDEDUC3   0
          DMDEDUC2   0
          INDHHIN2   0
          MCQ010     0
          MCQ080     0
          MCQ160A    0
          MCQ160B    0
          MCQ160C    0
          MCQ160D    0
          MCQ160E    0
          MCQ160F    0
          MCQ160M    0
          MCQ160G    0
          MCQ160K    0
          MCQ160O    0
          MCQ160L    0
          MCQ170L    0
          MCQ500     0
          MCQ510A    0
          MCQ510B    0
          MCQ510C    0
          MCQ510D    0
          MCQ510E    0
          MCQ510F    0
          MCQ220     0
          DLQ140     0
          DLQ150     0
          DLQ170     0
          HIQ031A    0
          HIQ031B    0
          HIQ031D    0
          HIQ031E    0
```

```

HIQ031H    0
HIQ031I    0
HIQ031AA   0
dtype: int64

```

In [85]: df2.head()

```

Out[85]:
   SEQN  DPQ010  DPQ020  DPQ030  DPQ040  DPQ050  DPQ060  DPQ070  DPQ080  DPQ090  ...  DLQ140  DLQ150  DLQ170  HIQ031A  I
0  93705.0     0.0     0.0     0.0     0.0     0.0     0.0     0.0     0.0     0.0  ...     4.0     2.0     1.0     0.0
1  93706.0     0.0     0.0     0.0     0.0     0.0     0.0     0.0     0.0     0.0  ...     5.0     2.0     0.0    14.0
2  93708.0     0.0     0.0     0.0     0.0     0.0     0.0     0.0     0.0     0.0  ...     5.0     2.0     0.0     0.0
3  93709.0     0.0     0.0     0.0     0.0     0.0     0.0     0.0     0.0     0.0  ...     1.0     1.0     1.0     0.0
4  93711.0     1.0     0.0     1.0     0.0     0.0     0.0     0.0     0.0     0.0  ...     5.0     2.0     0.0     0.0

```

5 rows × 48 columns

In [86]: df2=pd.get_dummies(df2, columns=['RIAGENDR'], drop_first=True)

In [87]: df3=df2.fillna(0)

In [88]: df3.rename(columns = {'RIAGENDR_2.0': 'SEXML', 'RIDAGEYR': 'AGE'}, inplace = True)
df3.columns

```

Out[88]:
Index(['SEQN', 'DPQ010', 'DPQ020', 'DPQ030', 'DPQ040', 'DPQ050', 'DPQ060',
      'DPQ070', 'DPQ080', 'DPQ090', 'AGE', 'RIDRETH3', 'DMDEDUC3', 'DMDEDUC2',
      'INDHHIN2', 'MCQ010', 'MCQ080', 'MCQ160A', 'MCQ160B', 'MCQ160C',
      'MCQ160D', 'MCQ160E', 'MCQ160F', 'MCQ160M', 'MCQ160G', 'MCQ160K',
      'MCQ1600', 'MCQ160L', 'MCQ170L', 'MCQ500', 'MCQ510A', 'MCQ510B',
      'MCQ510C', 'MCQ510D', 'MCQ510E', 'MCQ510F', 'MCQ220', 'DLQ140',
      'DLQ150', 'DLQ170', 'HIQ031A', 'HIQ031B', 'HIQ031D', 'HIQ031E',
      'HIQ031H', 'HIQ031I', 'HIQ031AA', 'SEXML'],
      dtype='object')

```

In [89]: df4=pd.get_dummies(df3, columns=['RIDRETH3'], drop_first=False, dtype=str)
df4.columns

```
Out[89]: Index(['SEQN', 'DPQ010', 'DPQ020', 'DPQ030', 'DPQ040', 'DPQ050', 'DPQ060',
        'DPQ070', 'DPQ080', 'DPQ090', 'AGE', 'DMDEDUC3', 'DMDEDUC2', 'INDHHIN2',
        'MCQ010', 'MCQ080', 'MCQ160A', 'MCQ160B', 'MCQ160C', 'MCQ160D',
        'MCQ160E', 'MCQ160F', 'MCQ160M', 'MCQ160G', 'MCQ160K', 'MCQ160O',
        'MCQ160L', 'MCQ170L', 'MCQ500', 'MCQ510A', 'MCQ510B', 'MCQ510C',
        'MCQ510D', 'MCQ510E', 'MCQ510F', 'MCQ220', 'DLQ140', 'DLQ150', 'DLQ170',
        'HIQ031A', 'HIQ031B', 'HIQ031D', 'HIQ031E', 'HIQ031H', 'HIQ031I',
        'HIQ031AA', 'SEXML', 'RIDRETH3_1.0', 'RIDRETH3_2.0', 'RIDRETH3_3.0',
        'RIDRETH3_4.0', 'RIDRETH3_6.0', 'RIDRETH3_7.0'],
        dtype='object')
```

```
In [90]: df4['RACEOT']=df4['RIDRETH3_1.0'] + " " + df4['RIDRETH3_2.0'] + " " + df4['RIDRETH3_6.0'] + " " + df4['RIDRETH3_7.0']
```

```
In [91]: s2=df4.pop('RIDRETH3_1.0')
s2=df4.pop('RIDRETH3_2.0')
s2=df4.pop('RIDRETH3_6.0')
s2=df4.pop('RIDRETH3_7.0')
#dropping individual groups for other races
df4.columns
```

```
Out[91]: Index(['SEQN', 'DPQ010', 'DPQ020', 'DPQ030', 'DPQ040', 'DPQ050', 'DPQ060',
        'DPQ070', 'DPQ080', 'DPQ090', 'AGE', 'DMDEDUC3', 'DMDEDUC2', 'INDHHIN2',
        'MCQ010', 'MCQ080', 'MCQ160A', 'MCQ160B', 'MCQ160C', 'MCQ160D',
        'MCQ160E', 'MCQ160F', 'MCQ160M', 'MCQ160G', 'MCQ160K', 'MCQ160O',
        'MCQ160L', 'MCQ170L', 'MCQ500', 'MCQ510A', 'MCQ510B', 'MCQ510C',
        'MCQ510D', 'MCQ510E', 'MCQ510F', 'MCQ220', 'DLQ140', 'DLQ150', 'DLQ170',
        'HIQ031A', 'HIQ031B', 'HIQ031D', 'HIQ031E', 'HIQ031H', 'HIQ031I',
        'HIQ031AA', 'SEXML', 'RIDRETH3_3.0', 'RIDRETH3_4.0', 'RACEOT'],
        dtype='object')
```

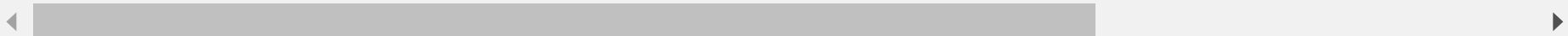
```
In [92]: df4.rename(columns = {'RIDRETH3_3.0':'RACEWH', 'RIDRETH3_4.0':'RACEBL'}, inplace = True)
```

```
In [93]: df4.head()
```

Out[93]:

	SEQN	DPQ010	DPQ020	DPQ030	DPQ040	DPQ050	DPQ060	DPQ070	DPQ080	DPQ090	...	HIQ031B	HIQ031D	HIQ031E	HIQ031F
0	93705.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	15.0	17.0	0.0	0.0
1	93706.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0
2	93708.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	15.0	0.0	0.0	0.0
3	93709.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	15.0	17.0	0.0	0.0
4	93711.0	1.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	21.0

5 rows × 50 columns



In [94]: `df5=df4.replace(r'^\s*$', np.nan, regex=True)`

In [95]: `df5=df5.fillna(0)`

In [96]: `dfh0=(df5['INDHHIN2'].value_counts(normalize=True))`
`dfh0`

Out[96]:

15.0	0.169570
6.0	0.099727
7.0	0.097848
14.0	0.086236
8.0	0.066940
0.0	0.059597
5.0	0.059085
4.0	0.057889
9.0	0.057719
10.0	0.047302
3.0	0.045253
12.0	0.037910
2.0	0.028347
1.0	0.028176
99.0	0.024761
77.0	0.020833
13.0	0.012807

Name: INDHHIN2, dtype: float64

In [97]: `# missing (2%) and refused to answer(2.5%); values <10%; will convert to 0 according to proposal`

```
In [98]: #missing not calculated for the co-morbid conditions since these would not be a required field for all responders
df6= df5.replace({'MCQ010':{7:0, 9:0, 2:0}, 'MCQ080':{7:0, 9:0, 2:0}, 'MCQ160A':{7:0, 9:0, 2:0}, 'MCQ160B':{7:0, 9:0, 2:0}, 'MCQ160F':{7:0, 9:0, 2:0}, 'MCQ160M':{7:0, 9:0, 2:0}, 'MCQ160G':{7:0, 9:0, 2:0}, 'MCQ510C':{3:1}, 'MCQ510D':{3:1}, 'MCQ510E':{5:1}, 'MCQ510F':{6:1}, 'MCQ22
```

```
In [99]: dfI=(df6[['HIQ031A', 'HIQ031B', 'HIQ031D', 'HIQ031E', 'HIQ031H', 'HIQ031I', 'HIQ031AA']].apply(pd.Series.value_counts,n
dfI
```

```
Out[99]:
```

	HIQ031A	HIQ031B	HIQ031D	HIQ031E	HIQ031H	HIQ031I	HIQ031AA
0.0	0.506148	0.731387	0.824966	0.999658	0.960212	0.962432	0.999829
14.0	0.486339	NaN	NaN	NaN	NaN	NaN	NaN
15.0	NaN	0.268613	NaN	NaN	NaN	NaN	NaN
17.0	NaN	NaN	0.175034	NaN	NaN	NaN	NaN
18.0	NaN	NaN	NaN	0.000342	NaN	NaN	NaN
21.0	NaN	NaN	NaN	NaN	0.039788	NaN	NaN
22.0	NaN	NaN	NaN	NaN	NaN	0.037568	NaN
40.0	NaN	NaN	NaN	NaN	NaN	NaN	0.000171
77.0	0.000512	NaN	NaN	NaN	NaN	NaN	NaN
99.0	0.007001	NaN	NaN	NaN	NaN	NaN	NaN

```
In [100... # missing (0.7% and refused to answer(0.05%); values <10%; will convert to 0 according to proposal
```

```
In [101... df7=df6.replace({'HIQ031A':{14:1, 99:0, 77:0}, 'HIQ031B':{15:1}, 'HIQ031D':{17:1}, 'HIQ031AA':{40:1}, 'HIQ031E':{18:1},
```

```
In [102... df7.head()
```

Out[102]:

	SEQN	DPQ010	DPQ020	DPQ030	DPQ040	DPQ050	DPQ060	DPQ070	DPQ080	DPQ090	...	HIQ031B	HIQ031D	HIQ031E	HIQ031H
0	93705.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	1.0	1.0	0.0	0.0
1	93706.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0
2	93708.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	1.0	0.0	0.0	0.0
3	93709.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	1.0	1.0	0.0	0.0
4	93711.0	1.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	1.0

5 rows × 50 columns

In [103]:

```
def combineColumns(row):
    if row['HIQ031D'] ==0:
        return '0'
    else:
        return '1'

    if row['HIQ031E'] ==0:
        return '0'
    else:
        return '1'

    if row['HIQ031H'] ==0:
        return '0'
    else:
        return '1'

    if row['HIQ031I'] ==0:
        return '0'
    else:
        return '1'
```

In [104]:

```
df7['INSSTATE'] =df7.apply(lambda row: combineColumns(row), axis =1)
```

In [105]:

```
s3=df7.pop('HIQ031D')
s3=df7.pop('HIQ031E')
s3=df7.pop('HIQ031H')
s3=df7.pop('HIQ031I')
```

```
#dropping individual groups for other state insurance
df7.columns
```

```
Out[105]: Index(['SEQN', 'DPQ010', 'DPQ020', 'DPQ030', 'DPQ040', 'DPQ050', 'DPQ060',
        'DPQ070', 'DPQ080', 'DPQ090', 'AGE', 'DMDEDUC3', 'DMDEDUC2', 'INDHHIN2',
        'MCQ010', 'MCQ080', 'MCQ160A', 'MCQ160B', 'MCQ160C', 'MCQ160D',
        'MCQ160E', 'MCQ160F', 'MCQ160M', 'MCQ160G', 'MCQ160K', 'MCQ160O',
        'MCQ160L', 'MCQ170L', 'MCQ500', 'MCQ510A', 'MCQ510B', 'MCQ510C',
        'MCQ510D', 'MCQ510E', 'MCQ510F', 'MCQ220', 'DLQ140', 'DLQ150', 'DLQ170',
        'HIQ031A', 'HIQ031B', 'HIQ031AA', 'SEXML', 'RACEWH', 'RACEBL', 'RACEOT',
        'INSSTATE'],
        dtype='object')
```

```
In [106... df7.rename(columns = {'HIQ031A':'INSPRT', 'HIQ031B':'INSMCARE', 'HIQ031AA':'INSNOCH'}, inplace = True)
```

```
In [107... dfD=(df7['DMDEDUC2'].value_counts(normalize=True))
dfD
```

```
Out[107]: 4.0    0.303620
          5.0    0.228142
          3.0    0.226264
          2.0    0.108948
          1.0    0.081796
          0.0    0.049010
          9.0    0.001878
          7.0    0.000342
          Name: DMDEDUC2, dtype: float64
```

```
In [108... dfE=(df7['DMDEDUC3'].value_counts(normalize=True))
dfE
```

```
Out[108]: 0.0    0.950990
          13.0   0.024249
          15.0   0.011954
          11.0   0.007172
          12.0   0.002220
          10.0   0.001537
          14.0   0.000683
          9.0    0.000683
          66.0   0.000512
          Name: DMDEDUC3, dtype: float64
```

```
In [109... # missing, unknown or refused for Annual Income (INDHHIN2), Education Levels (DMDEDUC2, DMDEDUC3) values <10%, will con
```

```
In [110]: df8=df7.replace({'DMDEDUC3':{55.0:1, 66.0:1, 77.0:0, 99.0:0, 15.0:4, 14.0:3, 13.0:3, 12.0:3, 11.0:2, 10.0:2, 9.0:2, 8.0:2, 7.0:2, 6.0:2, 5.0:2, 4.0:2, 3.0:2, 2.0:2, 1.0:2, 0.0:2}})

dfM=(df8['DMDEDUC3'].value_counts(normalize=True))
dfM
```

```
Out[110]: 0.0    0.950990
          3.0    0.027152
          4.0    0.011954
          2.0    0.009392
          1.0    0.000512
          Name: DMDEDUC3, dtype: float64
```

```
In [111]: dfP=(df8[['DLQ140','DLQ150','DLQ170']].apply(pd.Series.value_counts,normalize=True))
dfP
```

```
Out[111]:
```

	DLQ140	DLQ150	DLQ170
0.0	0.016052	0.016052	0.481387
1.0	0.054474	0.099556	0.248805
2.0	0.070014	0.883538	0.080772
3.0	0.081113	NaN	0.186988
4.0	0.310451	NaN	NaN
5.0	0.465335	NaN	NaN
7.0	0.000171	NaN	NaN
9.0	0.002391	0.000854	0.002049

```
In [112]: # missing (0.2%) and refused to answer(2%); values <10%; will convert to 0 according to proposal
```

```
In [113]: dfP=(df8[['DPQ010','DPQ020','DPQ030','DPQ040','DPQ050','DPQ060','DPQ070','DPQ080','DPQ090']].apply(pd.Series.value_counts,normalize=True))
dfP
```

Out[113]:

	DPQ010	DPQ020	DPQ030	DPQ040	DPQ050	DPQ060	DPQ070	DPQ080	DPQ090
0.0	0.777664	0.790301	0.662910	0.571721	0.781421	0.854167	0.854850	0.906079	0.966359
1.0	0.139344	0.141906	0.194843	0.275273	0.132855	0.098190	0.088286	0.056011	0.023224
2.0	0.047814	0.038251	0.065403	0.078040	0.047131	0.025102	0.026639	0.020492	0.005464
3.0	0.033811	0.028518	0.075649	0.073770	0.037739	0.021175	0.029372	0.016223	0.004098
7.0	0.000683	0.000342	0.000512	0.000512	0.000342	0.000512	0.000342	0.000342	0.000342
9.0	0.000683	0.000683	0.000683	0.000683	0.000512	0.000854	0.000512	0.000854	0.000512

In [114... *# 'dont know'(9.0) values for each question range from 0.5-0.9% and refused (7.0) values for each question range from 0*

In [115... `df9=df8.replace({'DPQ010':{7:0, 9:0}, 'DPQ020':{7:0, 9:0}, 'DPQ030':{7:0, 9:0}, 'DPQ040':{7:0, 9:0}, 'DPQ050':{7:0, 9:0}, 'DPQ080':{7:0, 9:0}, 'DPQ090':{7:0, 9:0}, 'DLQ140':{7:0, 9:0}, 'DLQ150':{7:0, 9:0}, 'DLQ`

In [116... `def combineColumns(row):
 if row['DMDEDUC3'] ==1:
 return '1'

 if row['DMDEDUC3'] ==2:
 return '2'

 if row['DMDEDUC3'] ==3:
 return '3'

 if row['DMDEDUC3'] ==4:
 return '4'

 if row['DMDEDUC2'] ==1:
 return '1'

 if row['DMDEDUC2'] ==2:
 return '2'

 if row['DMDEDUC2'] ==3:
 return '3'

 if row['DMDEDUC2'] ==4:
 return '4'`

```

if row['DMDEDUC2'] ==5:
    return '5'

else:
    return '0'

```

In [117... `df9['EDUC'] =df9.apply(lambda row: combineColumns(row), axis =1)`
`df9.head()`

Out[117]:

	SEQN	DPQ010	DPQ020	DPQ030	DPQ040	DPQ050	DPQ060	DPQ070	DPQ080	DPQ090	...	DLQ170	INSPRVT	INSMCARE	INSNOC
0	93705.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	1.0	0.0	1.0	0.0
1	93706.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	1.0	0.0	0.0
2	93708.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	1.0	0.0
3	93709.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	1.0	0.0	1.0	0.0
4	93711.0	1.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0

5 rows × 48 columns

In [118... `def combineColumns(row):`
`if row['MCQ160L'] ==0:`
 `return '0'`
`else:`
 `return '1'`

`if row['MCQ170L'] ==0:`
 `return '0'`
`else:`
 `return '1'`

`if row['MCQ500'] ==0:`
 `return '0'`
`else:`
 `return '1'`

`if row['MCQ510A'] ==0:`
 `return '0'`
`else:`
 `return '1'`

```

if row['MCQ510B'] ==0:
    return '0'
else:
    return '1'

if row['MCQ510C'] ==0:
    return '0'
else:
    return '1'

if row['MCQ510D'] ==0:
    return '0'
else:
    return '1'

if row['MCQ510E'] ==0:
    return '0'
else:
    return '1'

```

```
In [119... df9['MCQ700'] =df9.apply(lambda row: combineColumns(row), axis =1)
```

```
In [120... s4=df9.pop('MCQ160L')
s4=df9.pop('MCQ170L')
s4=df9.pop('MCQ500')
s4=df9.pop('MCQ510A')
s4=df9.pop('MCQ510B')
s4=df9.pop('MCQ510C')
s4=df9.pop('MCQ510D')
s4=df9.pop('MCQ510E')
s4=df9.pop('MCQ510F')
s4=df9.pop('DMDDEDUC3')
s4=df9.pop('DMDDEDUC2')
#dropping individual groups for other education and liver conditions
df9.columns
```

```
Out[120]: Index(['SEQN', 'DPQ010', 'DPQ020', 'DPQ030', 'DPQ040', 'DPQ050', 'DPQ060',
      'DPQ070', 'DPQ080', 'DPQ090', 'AGE', 'INDHHIN2', 'MCQ010', 'MCQ080',
      'MCQ160A', 'MCQ160B', 'MCQ160C', 'MCQ160D', 'MCQ160E', 'MCQ160F',
      'MCQ160M', 'MCQ160G', 'MCQ160K', 'MCQ1600', 'MCQ220', 'DLQ140',
      'DLQ150', 'DLQ170', 'INSPRVT', 'INSMCARE', 'INSNOCH', 'SEXML', 'RACEWH',
      'RACEBL', 'RACEOT', 'INSSTATE', 'EDUC', 'MCQ700'],
      dtype='object')
```

In [121... *# Create new column for PHQ total*

```
df9['PHQSCR'] = df9[['DPQ010', 'DPQ020', 'DPQ030', 'DPQ040', 'DPQ050', 'DPQ060', 'DPQ070', 'DPQ080', 'DPQ090']].sum(axis=1)
df9.head()
```

Out[121]:

	SEQN	DPQ010	DPQ020	DPQ030	DPQ040	DPQ050	DPQ060	DPQ070	DPQ080	DPQ090	...	INSMCARE	INSNOCH	SEXML	RACEW
0	93705.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	1.0	0.0	1	
1	93706.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0	
2	93708.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	1.0	0.0	1	
3	93709.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	1.0	0.0	1	
4	93711.0	1.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0	

5 rows × 39 columns

```
def DepInd(row):
    if row['PHQSCR'] >= 10:
        return '1'
    elif row['DLQ140'] < 3:
        return '1'
    elif row['DLQ150'] == 1:
        return '1'
    elif row['DLQ170'] == 2:
        return '1'
    else:
        return '0'
```

In [123... df9['DI'] = df9.apply(lambda row: DepInd(row), axis =1)

In [124... df9.head()

Out[124]:

	SEQN	DPQ010	DPQ020	DPQ030	DPQ040	DPQ050	DPQ060	DPQ070	DPQ080	DPQ090	...	INSNOCH	SEXML	RACEWH	RACEBL
0	93705.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	1	0	1
1	93706.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0	0	0
2	93708.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	1	0	0
3	93709.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	1	0	1
4	93711.0	1.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0	0	0

5 rows × 40 columns



In [125...

df9.columns

Out[125]:

```
Index(['SEQN', 'DPQ010', 'DPQ020', 'DPQ030', 'DPQ040', 'DPQ050', 'DPQ060',
      'DPQ070', 'DPQ080', 'DPQ090', 'AGE', 'INDHHIN2', 'MCQ010', 'MCQ080',
      'MCQ160A', 'MCQ160B', 'MCQ160C', 'MCQ160D', 'MCQ160E', 'MCQ160F',
      'MCQ160M', 'MCQ160G', 'MCQ160K', 'MCQ1600', 'MCQ220', 'DLQ140',
      'DLQ150', 'DLQ170', 'INSPRVT', 'INSMCARE', 'INSNOCH', 'SEXML', 'RACEWH',
      'RACEBL', 'RACEOT', 'INSSTATE', 'EDUC', 'MCQ700', 'PHQSCR', 'DI'],
      dtype='object')
```

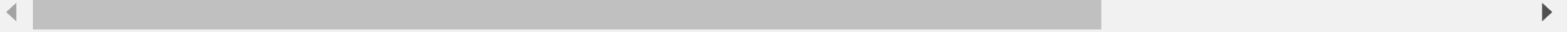
In [126...

```
df9.reindex(columns=['SEQN', 'AGE', 'SEXML', 'RACEWH', 'RACEBL', 'RACEOT', 'INDHHIN2', 'INSPRVT', 'INSMCARE',
                    'INSNOCH', 'INSSTATE', 'EDUC', 'MCQ010', 'MCQ080', 'MCQ160A', 'MCQ160B', 'MCQ160C', 'MCQ160D', 'MCQ160E', 'MCQ160F', 'MCQ160M', 'MCQ160G', 'MCQ160K', 'MCQ1600', 'MCQ220', 'DLQ140', 'DLQ150', 'DLQ170', 'DPQ010', 'DPQ020', 'DPQ030', 'DPQ040', 'DPQ050', 'DPQ060', 'DPQ070', 'D
```

Out[126]:

	SEQN	AGE	SEXML	RACEWH	RACEBL	RACEOT	INDHHIN2	INSPRVT	INSMCARE	INSNOCH	...	DPQ020	DPQ030	DPQ040	DPQ050
0	93705.0	66.0	1	0	1	0	3.0	0.0	1.0	0.0	...	0.0	0.0	0.0	0.0
1	93706.0	18.0	0	0	0	1	0.0	1.0	0.0	0.0	...	0.0	0.0	0.0	0.0
2	93708.0	66.0	1	0	0	1	6.0	0.0	1.0	0.0	...	0.0	0.0	0.0	0.0
3	93709.0	75.0	1	0	1	0	2.0	0.0	1.0	0.0	...	0.0	0.0	0.0	0.0
4	93711.0	56.0	0	0	0	1	15.0	0.0	0.0	0.0	...	0.0	1.0	0.0	0.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
9175	102751.0	19.0	1	1	0	0	12.0	1.0	0.0	0.0	...	0.0	0.0	0.0	0.0
9193	102804.0	30.0	0	0	0	1	8.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0
9203	102825.0	80.0	1	1	0	0	7.0	0.0	1.0	0.0	...	0.0	0.0	0.0	0.0
9247	102940.0	77.0	1	1	0	0	0.0	0.0	1.0	0.0	...	0.0	0.0	0.0	0.0
9251	102950.0	64.0	0	0	1	0	7.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0

5856 rows × 40 columns



```
In [127... df9.to_csv('nhanes_features_FINAL.csv', float_format='{:f}'.format, encoding='utf-8', index=False)
```

Hypothesis Testing and Classification Model Prediction

Importing Libraries

```
In [1]: import numpy as np
import pandas as pd
import xgboost as xgb
import matplotlib.pyplot as plt

from sklearn import metrics
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (confusion_matrix, ConfusionMatrixDisplay, accuracy_score, precision_score, f1_score
                             , recall_score, classification_report, mean_squared_error, auc
                             , precision_recall_curve, RocCurveDisplay)
from sklearn.model_selection import cross_val_score, train_test_split

#!pip install imbalanced-learn
from imblearn.over_sampling import SMOTE

import scipy.stats as stats
from scipy.stats import chi2
from scipy.stats import chi2_contingency
from colorama import Fore
```

Function to perform Chi-Squared Test on the dataset

```
In [2]: # STATISTICALLY SIGNIFICANT AND NON SIGNIFICANT VARIABLES
# CALCULATING AND INTERPRETTING BOTH P VALUE AND TEST STATISTIC VALUE
def Stat(d, t):

    # Lists that will hold the significant and non significant features
    Sig=[]
    Non_sig=[]

    # defining confidence level and probability for statistical testing
    alpha = 0.05
```

```

prob = 1-alpha

for i in d.columns:
    data_table = pd.crosstab(d[i],t)
    stat,p,dof,expected = chi2_contingency(data_table)
    critical = chi2.ppf(prob, dof)

    if p<= alpha and abs(stat) >= critical:
        print(Fore.GREEN+'p is statistically significant with',i,p)
        print('probability=%.3f, critical=%.3f, stat=%.3f' % (prob, critical, stat))
        print('statistic value is significant (reject H0)\n')
        Sig.append(i)

    if p> alpha and abs(stat) < critical:
        print(Fore.RED+'p is statistically not significant with',i,p)
        print('probability=%.3f, critical=%.3f, stat=%.3f' % (prob, critical, stat))
        print('statistic value is not significant (fail to reject H0)\n')
        Non_sig.append(i)

return Sig, Non_sig

```

NAMCS dataset for chi-squared test

```

In [3]: # namcs dataset
df = pd.DataFrame(pd.read_excel("namcs_features_FINAL.xlsx"))
# Depression Indicator
target = df['DI']

# WE ARE DROPPING THESE COLUMNS 'PSYCHOTH', 'DEPRN', 'DEPRESS', 'MENTAL', 'PSYCHOTH', 'DIAGALL'
# BECAUSE THEY ARE PART OF DEPRESSION INDICATOR CALCULATION - DI

df1 = df.drop(['SEQN','DI','PSYCHOTH', 'DEPRN', 'DEPRESS', 'MENTAL', 'PSYCHOTH','DIAGALL'], axis=1)

```

Retrieving Statistically Significant and Non-significant Features for NAMCS

```

In [4]: namcs_sig, namcs_non = Stat(df1, target)

```

p is statistically significant with AGE 3.8945972750688357e-13
probability=0.950, critical=96.217, stat=198.706
statistic value is significant (reject H0)

p is statistically significant with SEXML 4.4211211300461036e-05
probability=0.950, critical=3.841, stat=16.681
statistic value is significant (reject H0)

p is statistically significant with RACEWH 0.04017824866140956
probability=0.950, critical=3.841, stat=4.210
statistic value is significant (reject H0)

p is statistically not significant with RACEBL 0.11328907163698385
probability=0.950, critical=3.841, stat=2.508
statistic value is not significant (fail to reject H0)

p is statistically not significant with RACEOT 0.2565276432678608
probability=0.950, critical=3.841, stat=1.287
statistic value is not significant (fail to reject H0)

p is statistically not significant with NOPROVID 0.41136653334410456
probability=0.950, critical=3.841, stat=0.675
statistic value is not significant (fail to reject H0)

p is statistically significant with PHYS 3.998714035925914e-42
probability=0.950, critical=3.841, stat=184.963
statistic value is significant (reject H0)

p is statistically significant with PHYSASST 7.105980341152718e-06
probability=0.950, critical=3.841, stat=20.164
statistic value is significant (reject H0)

p is statistically not significant with NPNMW 0.3305729223732823
probability=0.950, critical=3.841, stat=0.947
statistic value is not significant (fail to reject H0)

p is statistically not significant with RNLPN 0.09016299778450985
probability=0.950, critical=3.841, stat=2.871
statistic value is not significant (fail to reject H0)

p is statistically significant with MHP 3.113020125757064e-98
probability=0.950, critical=3.841, stat=442.487
statistic value is significant (reject H0)

p is statistically significant with OTHPROV 5.08368829865834e-05

probability=0.950, critical=3.841, stat=16.417
statistic value is significant (reject H0)

p is statistically not significant with PROVNONE 0.22218091983263052
probability=0.950, critical=3.841, stat=1.490
statistic value is not significant (fail to reject H0)

p is statistically significant with INSMCARE 2.0017526564417026e-07
probability=0.950, critical=3.841, stat=27.031
statistic value is significant (reject H0)

p is statistically significant with INSSTATE 0.00474994380966783
probability=0.950, critical=3.841, stat=7.972
statistic value is significant (reject H0)

p is statistically not significant with INSNOCH 0.11259166438589081
probability=0.950, critical=3.841, stat=2.517
statistic value is not significant (fail to reject H0)

p is statistically significant with INSPRVT 0.023844945698492357
probability=0.950, critical=3.841, stat=5.106
statistic value is significant (reject H0)

p is statistically significant with SPPRC 0.0008513085246070578
probability=0.950, critical=3.841, stat=11.126
statistic value is significant (reject H0)

p is statistically significant with SPSUR 1.556442799587782e-81
probability=0.950, critical=3.841, stat=365.775
statistic value is significant (reject H0)

p is statistically significant with SPMEC 6.157188178357062e-67
probability=0.950, critical=3.841, stat=298.753
statistic value is significant (reject H0)

p is statistically significant with OWNUNKN 0.004476706562447029
probability=0.950, critical=3.841, stat=8.080
statistic value is significant (reject H0)

p is statistically significant with OWNPHYS 2.2912221498138237e-09
probability=0.950, critical=3.841, stat=35.709
statistic value is significant (reject H0)

p is statistically significant with OWNHOSP 3.708714859835234e-06
probability=0.950, critical=3.841, stat=21.410

statistic value is significant (reject H0)

p is statistically significant with OWNINSR 6.947709289907554e-07
probability=0.950, critical=3.841, stat=24.630
statistic value is significant (reject H0)

p is statistically significant with NOCHRON 2.8370838129696516e-83
probability=0.950, critical=3.841, stat=373.763
statistic value is significant (reject H0)

p is statistically not significant with DGRP1 0.5011840757859196
probability=0.950, critical=3.841, stat=0.452
statistic value is not significant (fail to reject H0)

p is statistically significant with DGRP2 1.6118696526088586e-07
probability=0.950, critical=3.841, stat=27.450
statistic value is significant (reject H0)

p is statistically significant with DGRP3 0.011906986125497968
probability=0.950, critical=3.841, stat=6.325
statistic value is significant (reject H0)

p is statistically significant with DGRP4 0.025688780050553646
probability=0.950, critical=3.841, stat=4.977
statistic value is significant (reject H0)

p is statistically significant with DGRP5 2.9620100184455633e-34
probability=0.950, critical=3.841, stat=148.936
statistic value is significant (reject H0)

p is statistically significant with DGRP6 0.038752220716391235
probability=0.950, critical=3.841, stat=4.272
statistic value is significant (reject H0)

p is statistically significant with DGRP7 0.01180536517158687
probability=0.950, critical=3.841, stat=6.340
statistic value is significant (reject H0)

p is statistically significant with DGRP8 0.0019135221566321527
probability=0.950, critical=3.841, stat=9.631
statistic value is significant (reject H0)

p is statistically not significant with DGRP9 0.271980260034547
probability=0.950, critical=3.841, stat=1.207
statistic value is not significant (fail to reject H0)

p is statistically significant with DGRP10 0.013311876043146868
 probability=0.950, critical=3.841, stat=6.127
 statistic value is significant (reject H0)

p is statistically significant with DGRP11 0.0027252424204183548
 probability=0.950, critical=3.841, stat=8.983
 statistic value is significant (reject H0)

p is statistically not significant with DGRP12 1.0
 probability=0.950, critical=3.841, stat=0.000
 statistic value is not significant (fail to reject H0)

p is statistically not significant with DGRP13 0.1605828481477233
 probability=0.950, critical=3.841, stat=1.969
 statistic value is not significant (fail to reject H0)

p is statistically significant with DGRP14 0.025740106914315473
 probability=0.950, critical=3.841, stat=4.973
 statistic value is significant (reject H0)

p is statistically significant with DGRP16 1.9360736852910946e-10
 probability=0.950, critical=3.841, stat=40.530
 statistic value is significant (reject H0)

NHANES dataset for chi-squared test

```
In [5]: # nhanes dataset
df_ = pd.DataFrame(pd.read_excel("nhanes_features_FINAL.xlsx"))
# Depression Indicator
target_ = df_['DI']

# WE ARE DROPPING THESE COLUMNS 'DLQ140', 'DLQ150', 'DLQ170', 'PHQSCR', 'DPQ010', 'DPQ020', 'DPQ030', 'DPQ040', 'DPQ050',
# 'DPQ060', 'DPQ070', 'DPQ080', 'DPQ090'
# BECAUSE THEY ARE PART OF DEPRESSION INDICATOR CALCULATION - DI

df1_ = df_.drop(['SEQN', 'DI', 'DLQ140', 'DLQ150', 'DLQ170', 'PHQSCR', 'DPQ010', 'DPQ020', 'DPQ030', 'DPQ040', 'DPQ050',
                 'DPQ060', 'DPQ070', 'DPQ080', 'DPQ090'], axis=1)
```

Retrieving Statistically Significant and Non-significant Features for NHANES

```
In [6]: nhanes_sig, nhanes_non = Stat(df1_, target_)
```

p is statistically significant with AGE 0.031169344259373052
probability=0.950, critical=81.381, stat=84.336
statistic value is significant (reject H0)

p is statistically significant with INDHHIN2 3.936574817639932e-37
probability=0.950, critical=23.685, stat=210.484
statistic value is significant (reject H0)

p is statistically significant with MCQ010 6.873873609517555e-16
probability=0.950, critical=3.841, stat=65.169
statistic value is significant (reject H0)

p is statistically significant with MCQ080 3.889666938644342e-19
probability=0.950, critical=3.841, stat=79.925
statistic value is significant (reject H0)

p is statistically significant with MCQ160A 1.196153517956054e-41
probability=0.950, critical=3.841, stat=182.783
statistic value is significant (reject H0)

p is statistically significant with MCQ160B 4.593087316206057e-14
probability=0.950, critical=3.841, stat=56.897
statistic value is significant (reject H0)

p is statistically significant with MCQ160C 3.2029232196242906e-09
probability=0.950, critical=3.841, stat=35.056
statistic value is significant (reject H0)

p is statistically significant with MCQ160D 1.0528154935101815e-13
probability=0.950, critical=3.841, stat=55.266
statistic value is significant (reject H0)

p is statistically significant with MCQ160E 9.050230382001851e-10
probability=0.950, critical=3.841, stat=37.520
statistic value is significant (reject H0)

p is statistically significant with MCQ160F 5.339281731443861e-20
probability=0.950, critical=3.841, stat=83.849
statistic value is significant (reject H0)

p is statistically significant with MCQ160M 7.441916480690449e-10
probability=0.950, critical=3.841, stat=37.901
statistic value is significant (reject H0)

p is statistically significant with MCQ160G 8.306499142703344e-11

probability=0.950, critical=3.841, stat=42.184
statistic value is significant (reject H0)

p is statistically significant with MCQ160K 1.2176957144576653e-31
probability=0.950, critical=3.841, stat=136.981
statistic value is significant (reject H0)

p is statistically significant with MCQ1600 4.285528448474836e-21
probability=0.950, critical=3.841, stat=88.838
statistic value is significant (reject H0)

p is statistically significant with MCQ220 2.9778569461277994e-05
probability=0.950, critical=3.841, stat=17.432
statistic value is significant (reject H0)

p is statistically significant with INSPRVT 3.398993797463646e-30
probability=0.950, critical=3.841, stat=130.371
statistic value is significant (reject H0)

p is statistically significant with INSMCARE 2.1272674384860927e-15
probability=0.950, critical=3.841, stat=62.943
statistic value is significant (reject H0)

p is statistically not significant with INSNOCH 1.0
probability=0.950, critical=3.841, stat=0.000
statistic value is not significant (fail to reject H0)

p is statistically significant with SEXML 1.1416972446301036e-08
probability=0.950, critical=3.841, stat=32.584
statistic value is significant (reject H0)

p is statistically significant with RACEWH 2.8003668424917386e-14
probability=0.950, critical=3.841, stat=57.870
statistic value is significant (reject H0)

p is statistically not significant with RACEBL 0.09906964310052707
probability=0.950, critical=3.841, stat=2.720
statistic value is not significant (fail to reject H0)

p is statistically significant with RACEOT 3.765264250057065e-09
probability=0.950, critical=3.841, stat=34.741
statistic value is significant (reject H0)

p is statistically significant with INSSTATE 1.0291857776949414e-25
probability=0.950, critical=3.841, stat=109.903

statistic value is significant (reject H0)

p is statistically significant with EDUC 4.4285470792863274e-18
probability=0.950, critical=9.488, stat=87.520
statistic value is significant (reject H0)

p is statistically significant with MCQ700 1.5633036634299246e-09
probability=0.950, critical=3.841, stat=36.454
statistic value is significant (reject H0)

Displaying Statistically Significant and Non-significant Features for both datasets

```
In [7]: print("Significant Features in NAMCS: ", len(namcs_sig), "\n", namcs_sig, "\n")
print("Non-Significant Features in NAMCS: ", len(namcs_non), "\n", namcs_non, "\n")

print("Significant Features in NHANES: ", len(nhanes_sig), "\n", nhanes_sig, "\n")
print("Non-Significant Features in NHANES: ", len(nhanes_non), "\n", nhanes_non)
```

Significant Features in NAMCS: 29
['AGE', 'SEXML', 'RACEWH', 'PHYS', 'PHYSASST', 'MHP', 'OTHPROV', 'INSMCARE', 'INSSTATE', 'INSPRVT', 'SPPRC', 'SPSUR', 'SPMEC', 'OWNUNKN', 'OWNPHYS', 'OWNHOSP', 'OWNINSR', 'NOCHRON', 'DGRP2', 'DGRP3', 'DGRP4', 'DGRP5', 'DGRP6', 'DGRP7', 'DGRP8', 'DGRP10', 'DGRP11', 'DGRP14', 'DGRP16']

Non-Significant Features in NAMCS: 11
['RACEBL', 'RACEOT', 'NOPROVID', 'NPNMW', 'RNLPN', 'PROVNONE', 'INSNOCH', 'DGRP1', 'DGRP9', 'DGRP12', 'DGRP13']

Significant Features in NHANES: 23
['AGE', 'INDHHIN2', 'MCQ010', 'MCQ080', 'MCQ160A', 'MCQ160B', 'MCQ160C', 'MCQ160D', 'MCQ160E', 'MCQ160F', 'MCQ160M', 'MCQ160G', 'MCQ160K', 'MCQ160O', 'MCQ220', 'INSPRVT', 'INSMCARE', 'SEXML', 'RACEWH', 'RACEOT', 'INSSTATE', 'EDUC', 'MCQ700']

Non-Significant Features in NHANES: 2
['INSNOCH', 'RACEBL']

Labels for NAMCS and NHANES Statistically Significant Features

```
In [8]: namcs = ['AGE', 'GENDER', 'RACE-WHITE', 'Physician', 'Physician assistant', 'Mental health provider', 'Other provider',
               'Medicare', 'State Insurance', 'Private Insurance', 'Specialty:Primary care', 'Specialty:Surgical care',
               'Specialty:Medical care', 'Practice_owner:Unknown', 'Practice_owner:Refused to answer question',
               'Practice_owner:Physician or physician group', 'Practice_owner:Medical/Academic health center; other hospita
```

```
, 'Diagnosis: No chronic conditions', 'Diagnosis: Cancer/Blood', 'Diagnosis: Metabolic', 'Diagnosis: Neurologi
, 'Diagnosis: Eyes/Ears', 'Diagnosis: Heart/Lung', 'Diagnosis: Digestive', 'Diagnosis: Skin', 'Diagnosis: Bone
, 'Diagnosis: Women\'s Health', 'Diagnosis: Injury', 'Diagnosis: Punlic Health']

nhanes = ['AGE', 'Month family income', 'MedHistory: Asthma', 'MedHistory: Overweight', 'MedHistory: Arthritis'
, 'MedHistory: Congestive heart failure', 'MedHistory: Coronary heart disease'
, 'MedHistory: Angina/Angina Pectoris', 'MedHistory: Heart attack', 'MedHistory: Stroke'
, 'MedHistory: Thyroid problem', 'MedHistory: Emphysema', 'MedHistory: Chronic Bronchitis'
, 'MedHistory: COPD', 'MedHistory: Cancer or Malignancy', 'Private Insurance', 'Medicare', 'GENDER'
, 'RACE-WHITE', 'RACE-OTHERS', 'State Insurance', 'EDUCATION LEVEL', 'MedHistory: Liver issues']
```

Function for Determining Model metrics: skmets(y_test, y_predicted)

```
In [9]: def skmets(y, p):

    Accuracy = accuracy_score(y, p)
    Precision = precision_score(y, p)
    Sensitivity_recall = recall_score(y, p)
    Specificity = recall_score(y, p, pos_label=0)
    F1_score = f1_score(y, p)
    M = {"Accuracy":Accuracy, "Precision":Precision, "Sensitivity_recall":Sensitivity_recall
        , "Specificity":Specificity, "F1_score":F1_score}

    return M
```

Function for Cross-Validation: cross(model, X_train, y_train, scoring_metric)

```
In [10]: def cross(m, X_tr, y_tr, s):

    scores = cross_val_score(m, X_tr, y_tr, cv=5, scoring=s)
    print("Mean "+s+" of %0.2f with a standard deviation of %0.2f" % (scores.mean(), scores.std()))
```

Function for Feature Importance: feat_imp(model, feature_labels)

```
In [11]: def feat_imp(m, i):

    feat_importances = pd.Series(m.feature_importances_, index=i)
    feat_importances.sort_values(ascending=True, inplace=True)
    feat_importances.plot(kind='barh', figsize = (30,30), fontsize=25, grid = True)
```

Function for Confusion Matrix: conf_mat(y_test, y_predicted)

```
In [12]: def conf_mat(y_ts, p):

    cn_matrix = metrics.confusion_matrix(y_ts, p)
    cm_display = ConfusionMatrixDisplay(confusion_matrix = cn_matrix , display_labels = [False, True])
    cm_display.plot()
    cm_display.ax_.set_title('Confusion Matrix')

    cn_matrix = metrics.confusion_matrix(y_ts, p, normalize='true')
    cm_display = ConfusionMatrixDisplay(confusion_matrix = cn_matrix , display_labels = [False, True])
    cm_display.plot()
    cm_display.ax_.set_title('Normalized Confusion Matrix')

    plt.show()
```

Function for Plotting Precision-Recall Curve: pr(y_test, y_predicted, plot_title, legend_text)

```
In [13]: def pr(y_ts, p, T, r):

    #calculate precision and recall
    precision, recall, thresholds = precision_recall_curve(y_ts, p)

    # Use AUC function to calculate the area under the curve of precision recall curve
    auc_precision_recall = auc(recall, precision)

    #create precision recall curve
    fig, ax = plt.subplots()
    ax.plot(recall, precision, color='green')

    #add axis labels to plot
    ax.set_title(T)
    ax.set_ylabel('Precision')
```

```
ax.set_xlabel('Recall')
ax.legend([r+str(auc_precision_recall)])

#display plot
plt.show()
```

Function for Plotting ROC Curve: roc(model, X_test, y_test)

```
In [14]: def roc(m, X_ts, y_ts):

        RocCurveDisplay.from_estimator(m, X_ts, y_ts)
        plt.show()
```

Assigning Statistically Significant Features and target feature for both datasets

```
In [15]: df_nam = df1[namcs_sig]
X_nam = df_nam
y_nam = target

df_nh = df1[nhanes_sig]
X_nh = df_nh
y_nh = target_
```

XGBOOST CLASSIFIER

```
In [16]: xgb_cl = xgb.XGBClassifier()
```

namcs dataset

```
In [17]: Xnam_train, Xnam_test, ynam_train, ynam_test = train_test_split(X_nam, y_nam, train_size=0.8, random_state=1000000000)
```

```
In [18]: # namcs Fit
xgb_cl.fit(Xnam_train, ynam_train)
modelnam_fit = xgb_cl.fit(Xnam_train, ynam_train)
```

```
# Predict  
prednam = xgb_cl.predict(Xnam_test)
```

```
In [19]: #metrics.get_scorer_names()
```

Cross Validation Score for namcs before smote

```
In [20]: cross(xgb_cl, Xnam_train, ynam_train, 'f1_macro')
```

Mean f1_macro of 0.68 with a standard deviation of 0.02

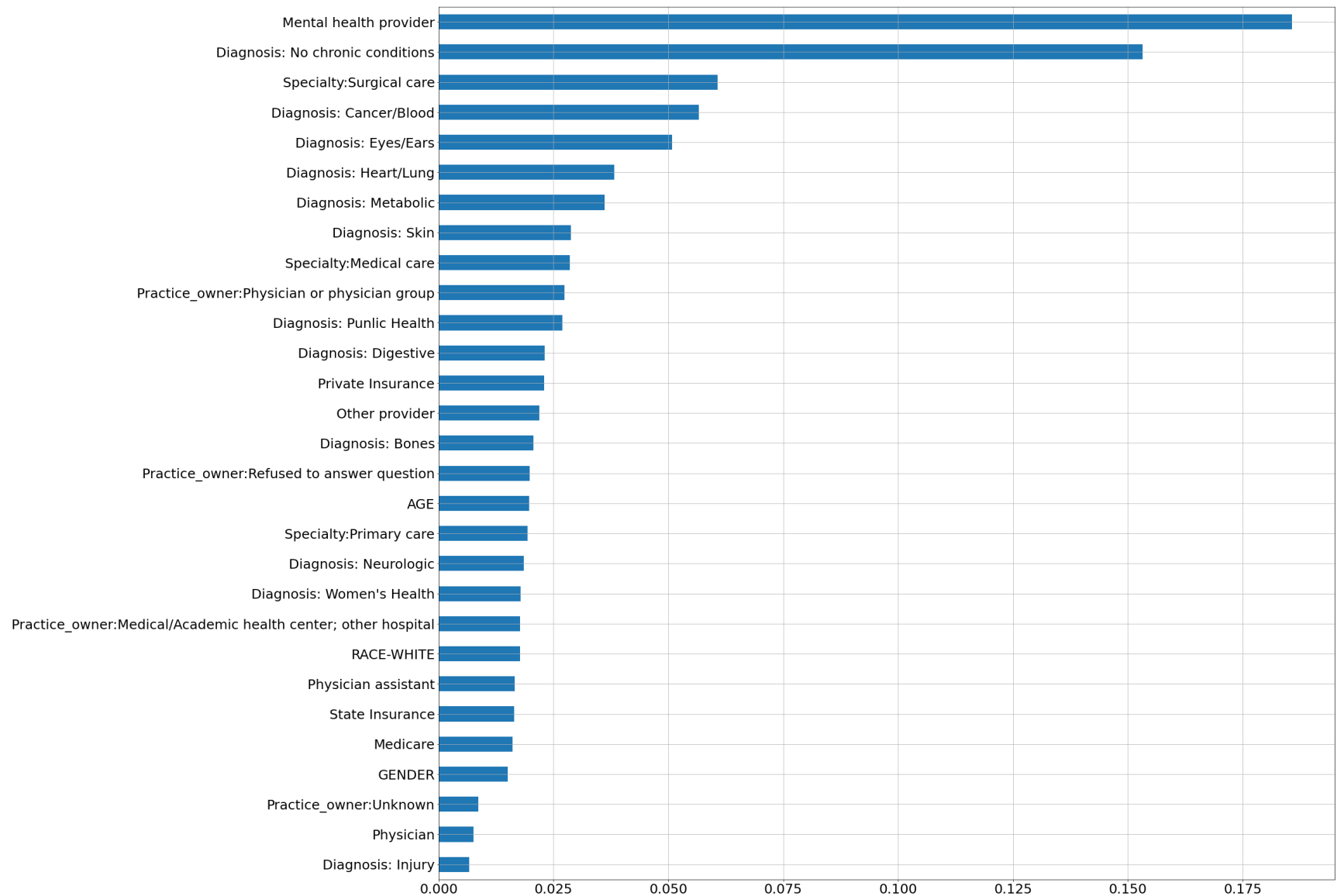
Model scores for namcs before smote

```
In [21]: skmets(ynam_test, prednam)
```

```
Out[21]: {'Accuracy': 0.8753589890867317,  
          'Precision': 0.6274509803921569,  
          'Sensitivity_recall': 0.375,  
          'Specificity': 0.9616161616161616,  
          'F1_score': 0.46943765281173594}
```

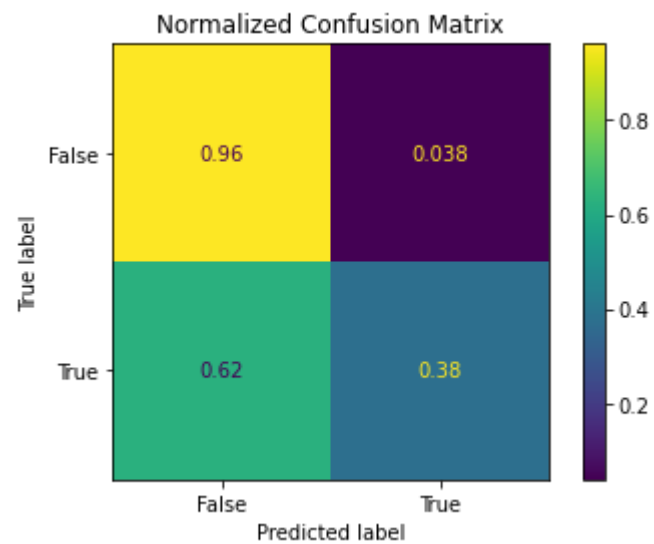
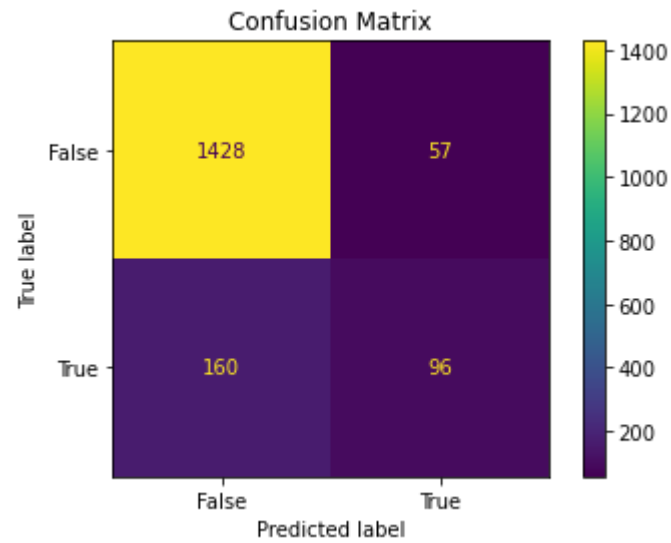
Feature Importance for predicting Depression for namcs before smote

```
In [22]: feat_imp(xgb_cl, namcs)
```



Confusion matrix for namcs before smote

```
In [23]: conf_mat(ynam_test, prednam)
```



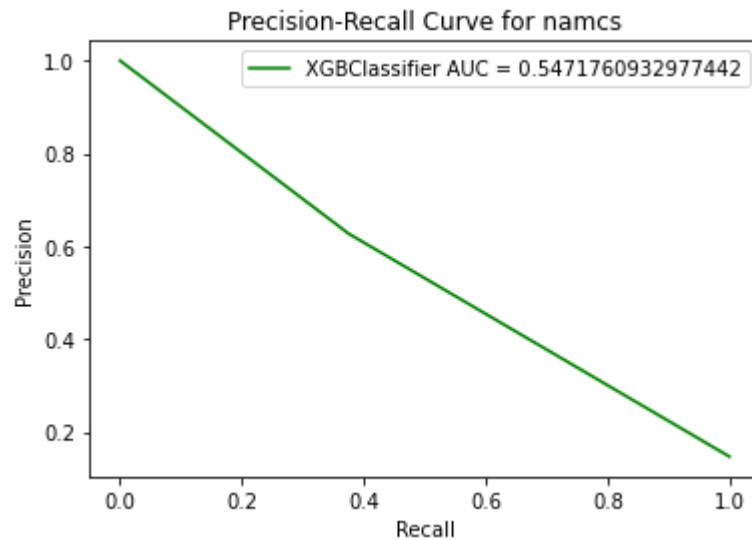
XG BOOST namcs dataset Report before smote

```
In [24]: print(classification_report(ynam_test,prednam))
```

	precision	recall	f1-score	support
0	0.90	0.96	0.93	1485
1	0.63	0.38	0.47	256
accuracy			0.88	1741
macro avg	0.76	0.67	0.70	1741
weighted avg	0.86	0.88	0.86	1741

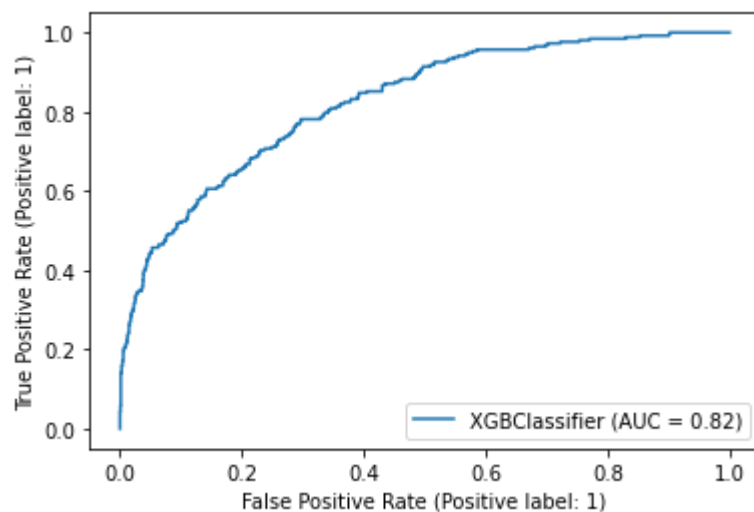
PR Curve for namcs before smote

```
In [25]: #calculate precision and recall
pr(yname_test, predname, 'Precision-Recall Curve for namcs', 'XGBClassifier AUC = ')
```



ROC Curve for namcs before smote

```
In [26]: roc(xgb_cl, Xname_test, yname_test)
```



Applying SMOTE for namcs dataset

```
In [27]: sm = SMOTE()
Xnam_train, ynam_train = sm.fit_resample(Xnam_train, ynam_train)
# namcs Fit
xgb_cl.fit(Xnam_train, ynam_train)
modelnam_fit = xgb_cl.fit(Xnam_train, ynam_train)
# Predict
prednam = xgb_cl.predict(Xnam_test)
```

Cross Validation Score for namcs after smote

```
In [28]: cross(xgb_cl, Xnam_train, ynam_train, 'f1_macro')
Mean f1_macro of 0.88 with a standard deviation of 0.08
```

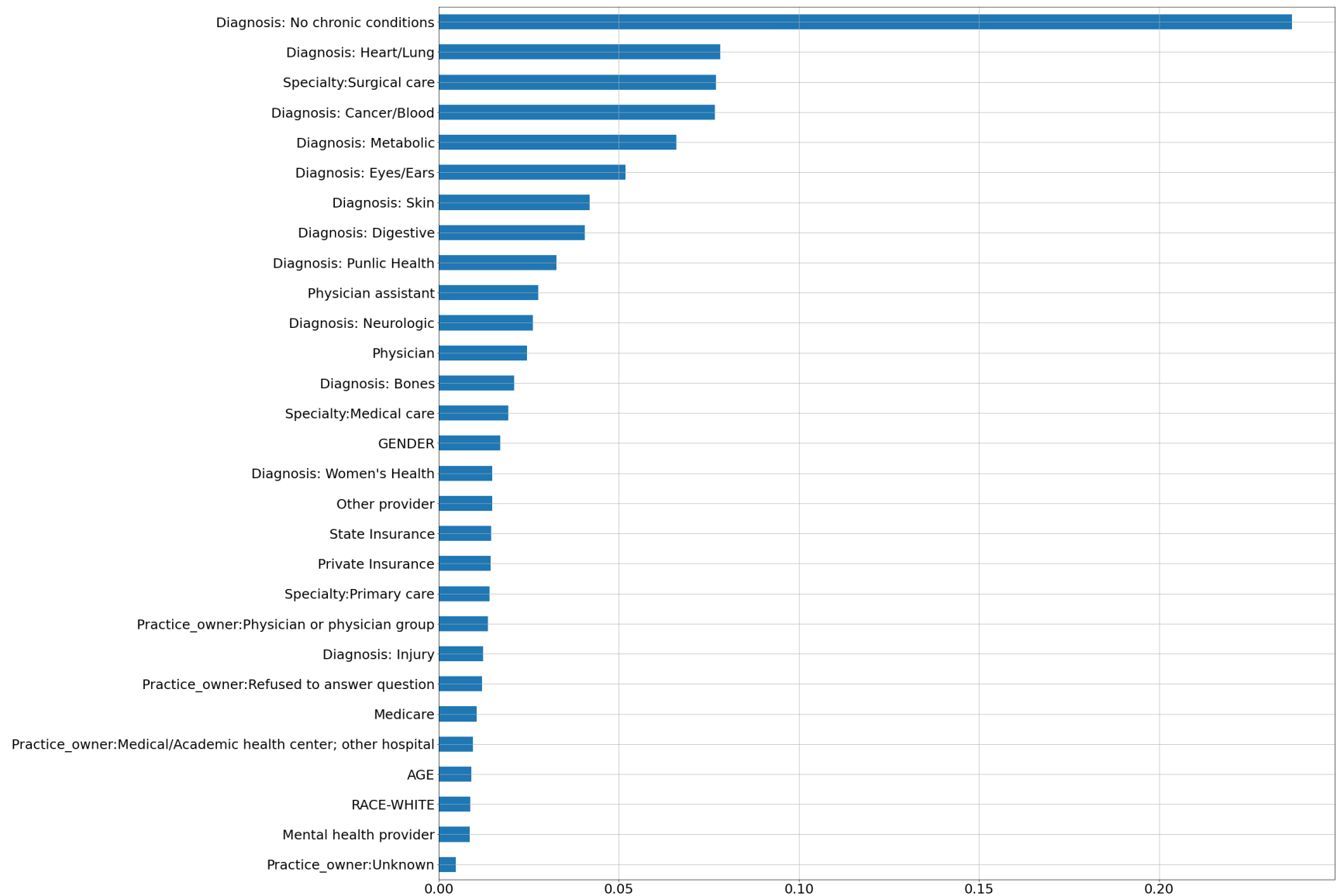
Model scores for namcs after smote

```
In [29]: skmets(ynam_test, prednam)
```

```
Out[29]: {'Accuracy': 0.8363009764503159,  
          'Precision': 0.44947735191637633,  
          'Sensitivity_recall': 0.50390625,  
          'Specificity': 0.8936026936026936,  
          'F1_score': 0.47513812154696133}
```

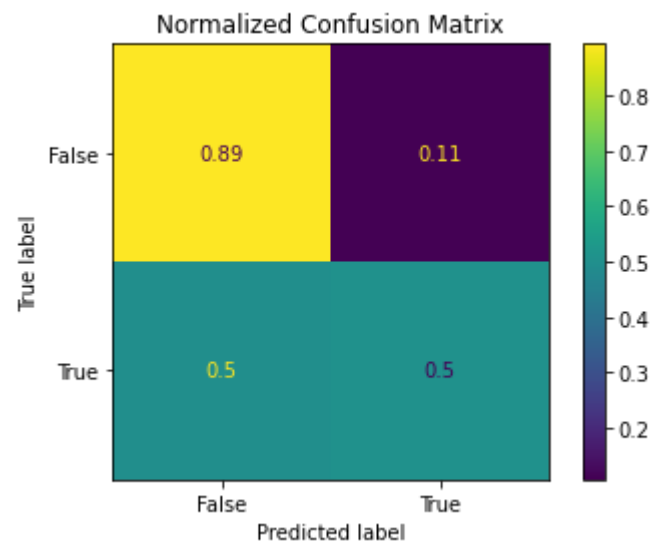
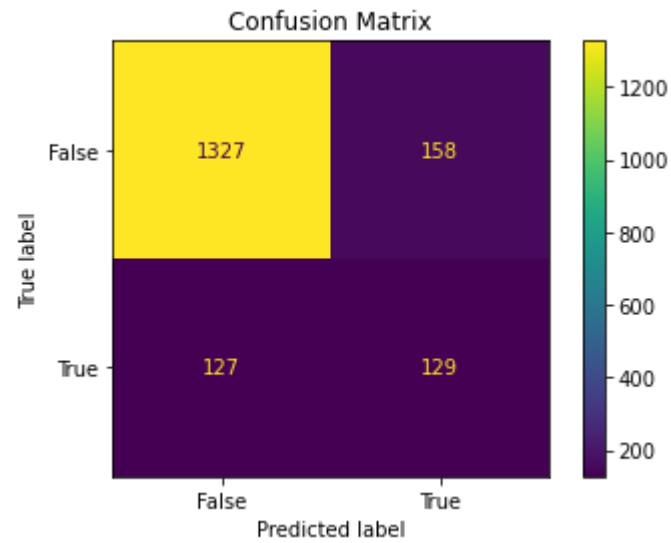
Feature Importance for predicting Depression for namcs after smote

```
In [30]: feat_imp(xgb_cl, namcs)
```



Confusion matrix for namcs after smote

```
In [31]: conf_mat(ynam_test, prednam)
```



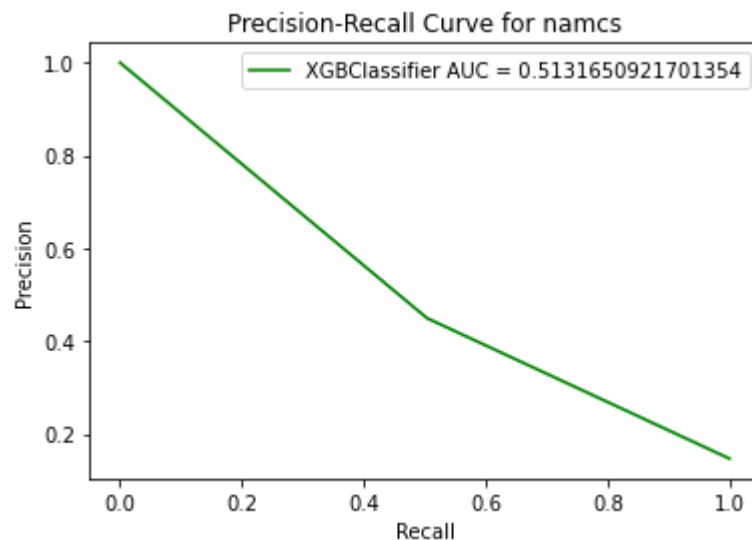
XG BOOST namcs dataset Report after smote

```
In [32]: print(classification_report(ynam_test,prednam))
```

	precision	recall	f1-score	support
0	0.91	0.89	0.90	1485
1	0.45	0.50	0.48	256
accuracy			0.84	1741
macro avg	0.68	0.70	0.69	1741
weighted avg	0.84	0.84	0.84	1741

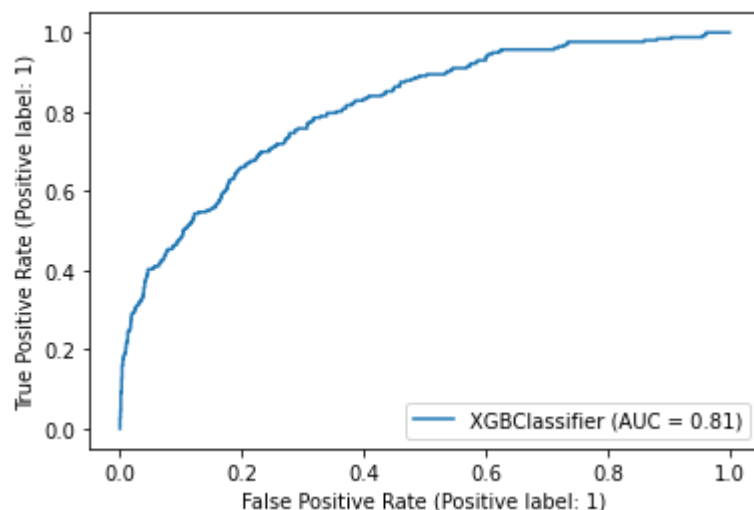
PR Curve for namcs after smote

```
In [33]: #calculate precision and recall
pr(yname_test, predname, 'Precision-Recall Curve for namcs', 'XGBClassifier AUC = ')
```



ROC Curve for namcs after smote

```
In [34]: roc(xgb_cl, Xname_test, yname_test)
```



nhanes dataset

```
In [35]: Xnh_train, Xnh_test, ynh_train, ynh_test = train_test_split(X_nh, y_nh, train_size=0.8, random_state=1000000000)
```

```
In [36]: # nhanes Fit
xgb_cl.fit(Xnh_train, ynh_train)
modelnh_fit = xgb_cl.fit(Xnh_train, ynh_train)
# Predict
prednh = xgb_cl.predict(Xnh_test)
```

Cross Validation Score for nhanes before smote

```
In [37]: cross(xgb_cl, Xnh_train, ynh_train, 'f1_macro')
```

Mean f1_macro of 0.59 with a standard deviation of 0.02

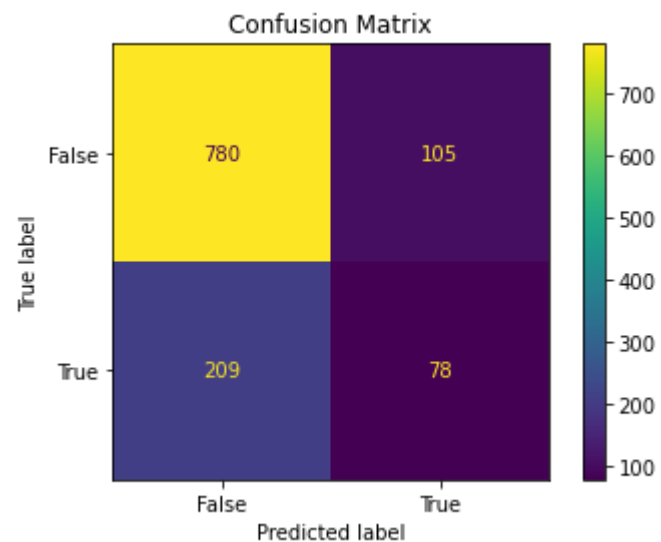
Model scores for nhanes before smote

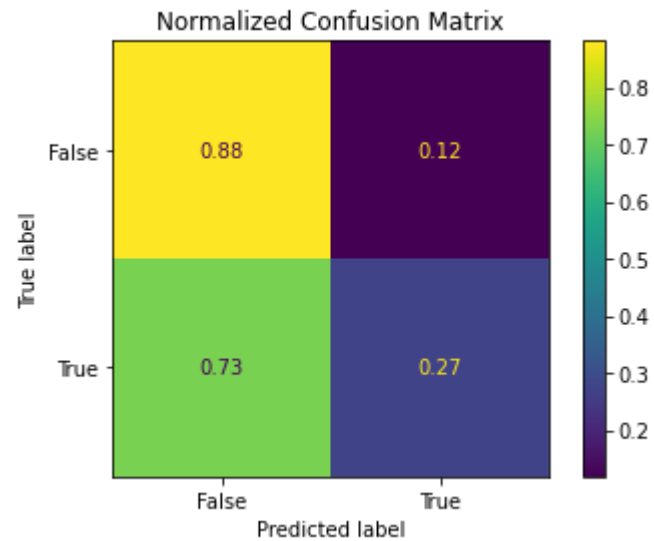
```
In [38]: skmets(ynh_test, prednh)
```

```
Out[38]: {'Accuracy': 0.7320819112627986,  
          'Precision': 0.4262295081967213,  
          'Sensitivity_recall': 0.27177700348432055,  
          'Specificity': 0.8813559322033898,  
          'F1_score': 0.33191489361702126}
```

Confusion matrix for nhanes before smote

```
In [39]: conf_mat(ynh_test, prednh)
```





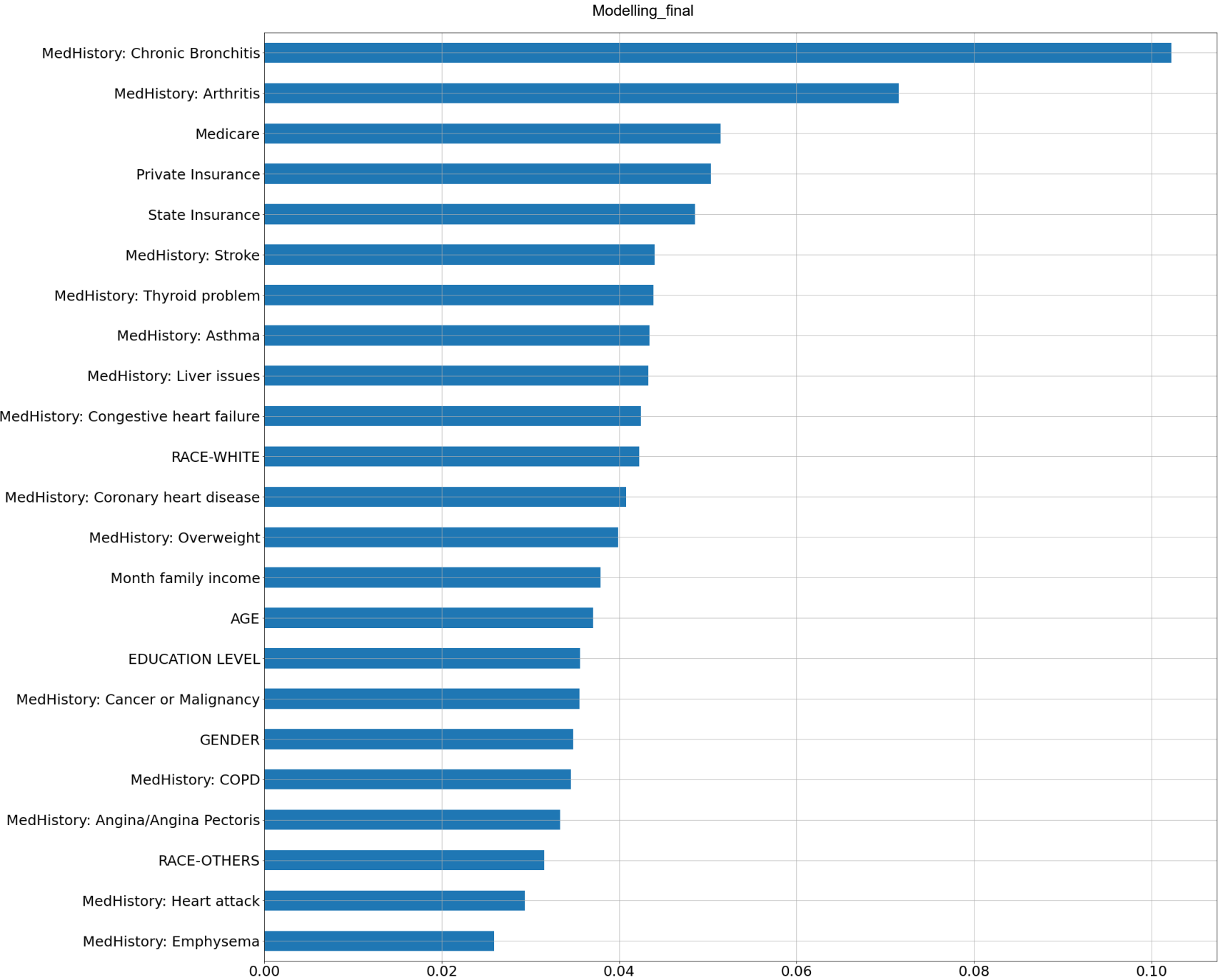
XG BOOST nhanes dataset Report before smote

```
In [40]: print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	0.79	0.88	0.83	885
1	0.43	0.27	0.33	287
accuracy			0.73	1172
macro avg	0.61	0.58	0.58	1172
weighted avg	0.70	0.73	0.71	1172

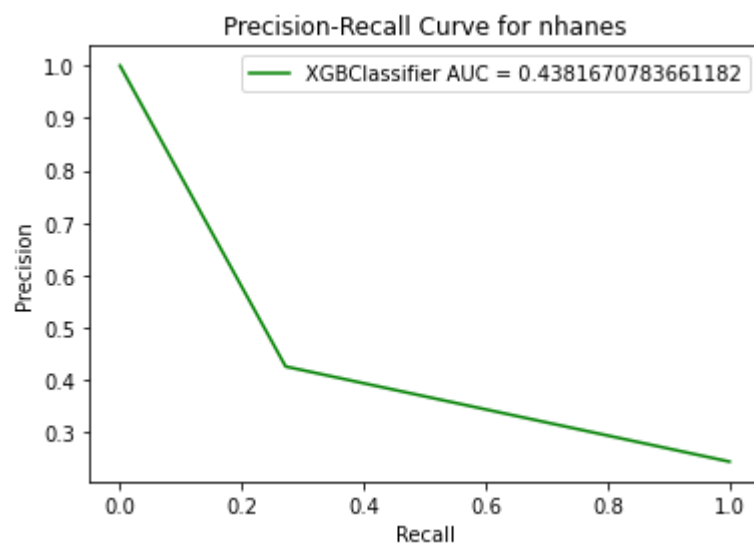
Feature Importance for predicting Depression for nhanes before smote

```
In [41]: feat_imp(xgb_cl, nhanes)
```



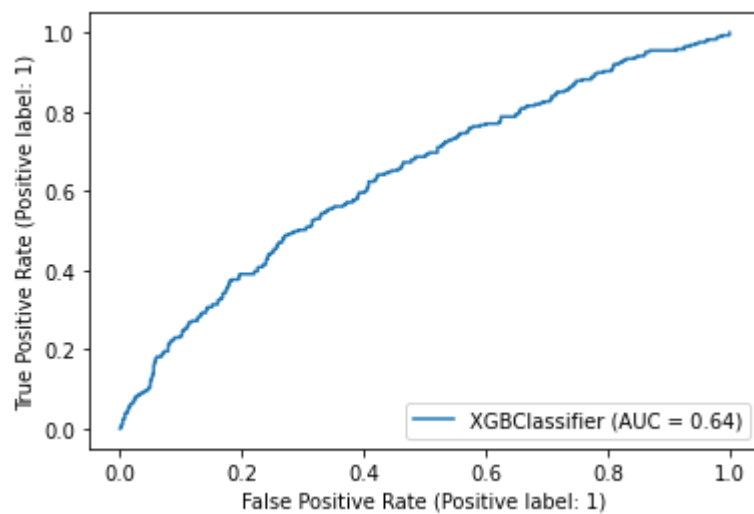
PR Curve for nhanes before smote

```
In [42]: #calculate precision and recall  
pr(ynh_test, prednh, 'Precision-Recall Curve for nhanes', 'XGBClassifier AUC = ')
```



ROC Curve for nhanes before smote

```
In [43]: roc(xgb_cl, Xnh_test, ynh_test)
```



Applying SMOTE for nhanes dataset

```
In [44]: sm = SMOTE()  
Xnh_train, ynh_train = sm.fit_resample(Xnh_train, ynh_train)  
# nhanes Fit  
xgb_cl.fit(Xnh_train, ynh_train)  
modelnh_fit = xgb_cl.fit(Xnh_train, ynh_train)  
# Predict  
prednh = xgb_cl.predict(Xnh_test)
```

Cross Validation Score for nhanes after smote

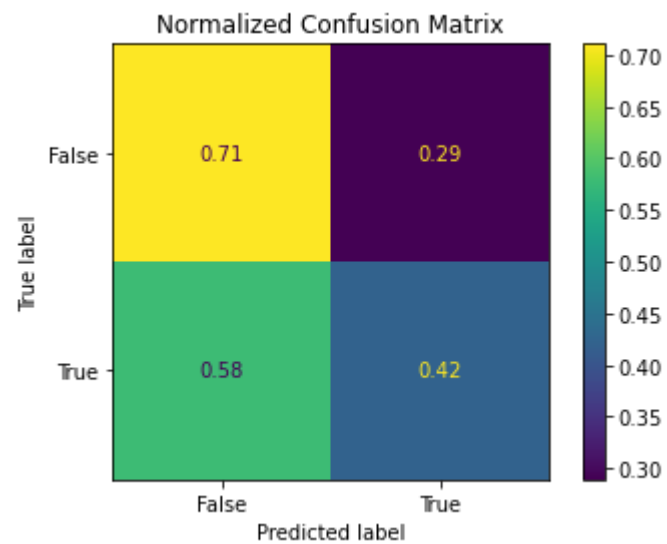
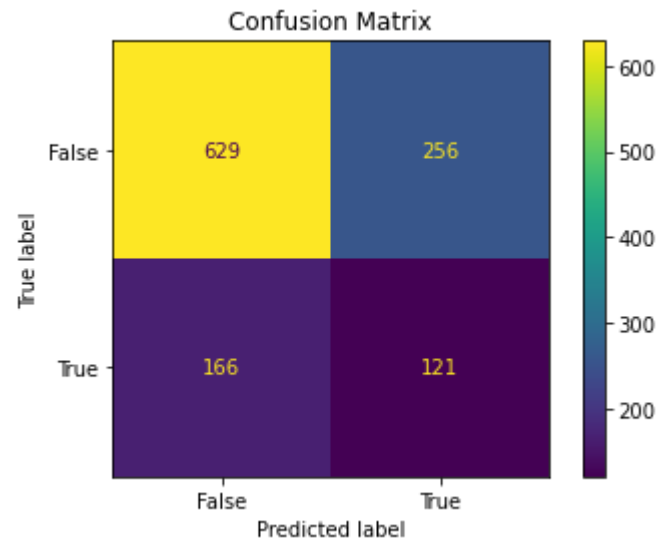
```
In [45]: cross(xgb_cl, Xnh_train, ynh_train, 'f1_macro')  
Mean f1_macro of 0.76 with a standard deviation of 0.07
```

Model scores for nhanes after smote

```
In [46]: skmets(ynh_test, prednh)  
Out[46]: {'Accuracy': 0.6399317406143344,  
          'Precision': 0.3209549071618037,  
          'Sensitivity_recall': 0.42160278745644597,  
          'Specificity': 0.7107344632768362,  
          'F1_score': 0.36445783132530113}
```

Confusion matrix for nhanes after smote

```
In [47]: conf_mat(ynh_test, prednh)
```



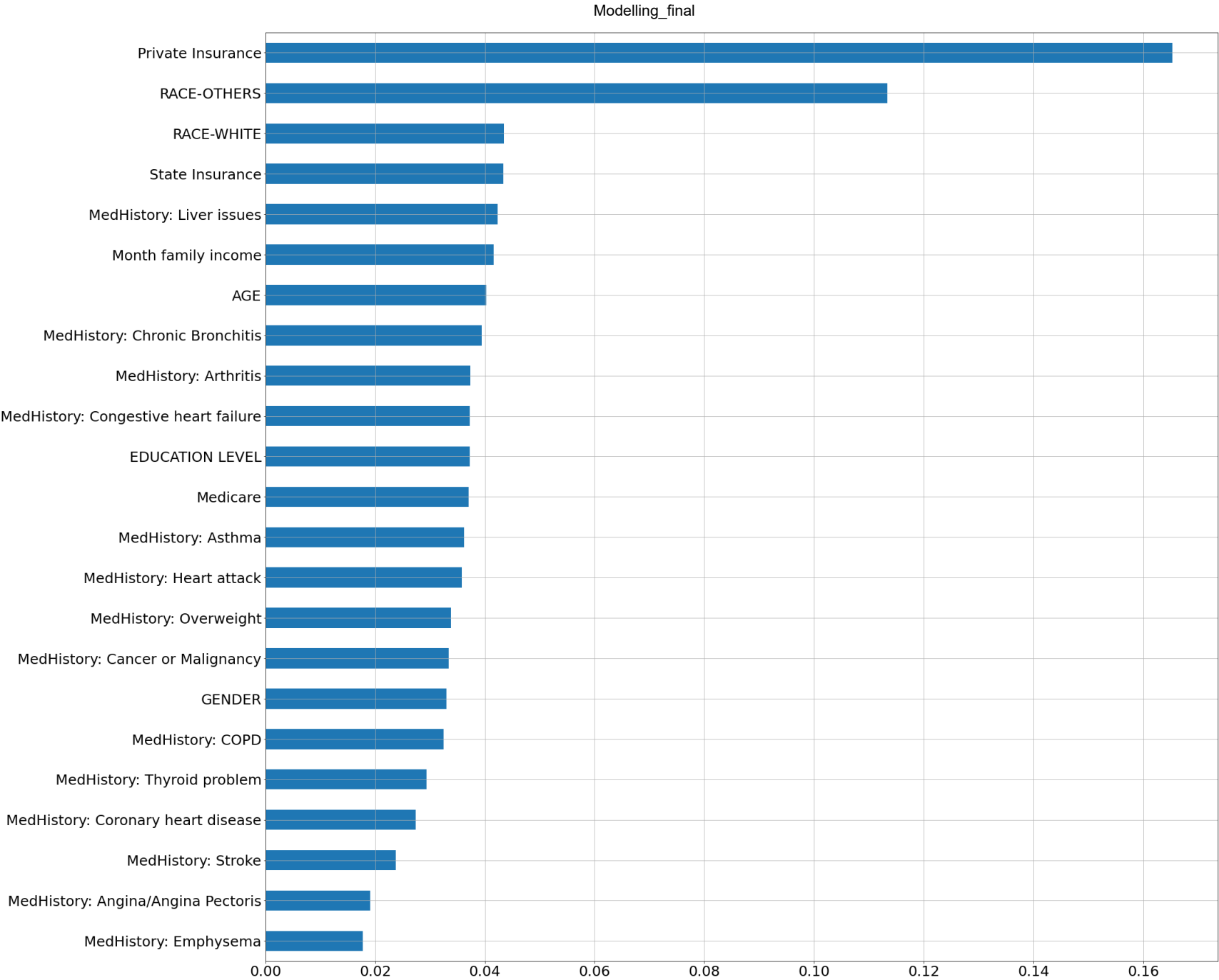
XG BOOST nhanes dataset Report after smote

```
In [48]: print(classification_report(y nh_test, pred nh))
```

	precision	recall	f1-score	support
0	0.79	0.71	0.75	885
1	0.32	0.42	0.36	287
accuracy			0.64	1172
macro avg	0.56	0.57	0.56	1172
weighted avg	0.68	0.64	0.65	1172

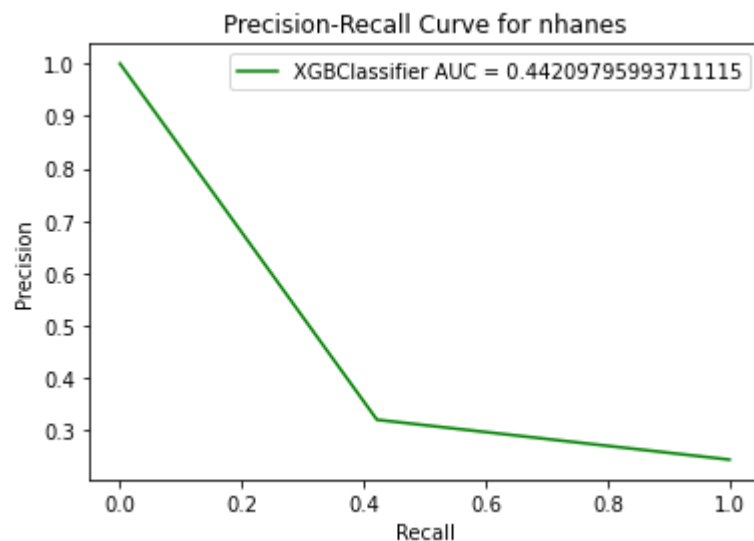
Feature Importance for predicting Depression for nhanes after smote

```
In [49]: feat_imp(xgb_cl, nhanes)
```



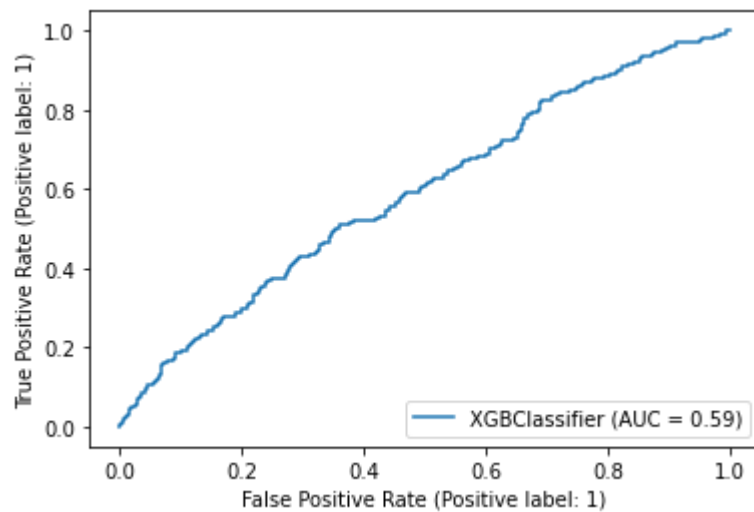
PR Curve for nhanes after smote

```
In [50]: #calculate precision and recall  
pr(ynh_test, prednh, 'Precision-Recall Curve for nhanes', 'XGBClassifier AUC = ')
```



ROC Curve for nhanes after smote

```
In [51]: roc(xgb_cl, Xnh_test, ynh_test)
```



RANDOM FOREST CLASSIFIER

```
In [52]: clf = RandomForestClassifier(criterion = "gini", random_state=1)
```

namcs dataset

```
In [53]: Xnam_train, Xnam_test, ynam_train, ynam_test = train_test_split(X_nam, y_nam, train_size=0.8, random_state=1000000000)
```

```
In [54]: # namcs
clf.fit(Xnam_train, ynam_train)
prednam = clf.predict(Xnam_test)
```

Cross Validation for namcs before smote

```
In [55]: cross(clf, Xnam_train, ynam_train, 'f1_macro')
```

Mean f1_macro of 0.67 with a standard deviation of 0.02

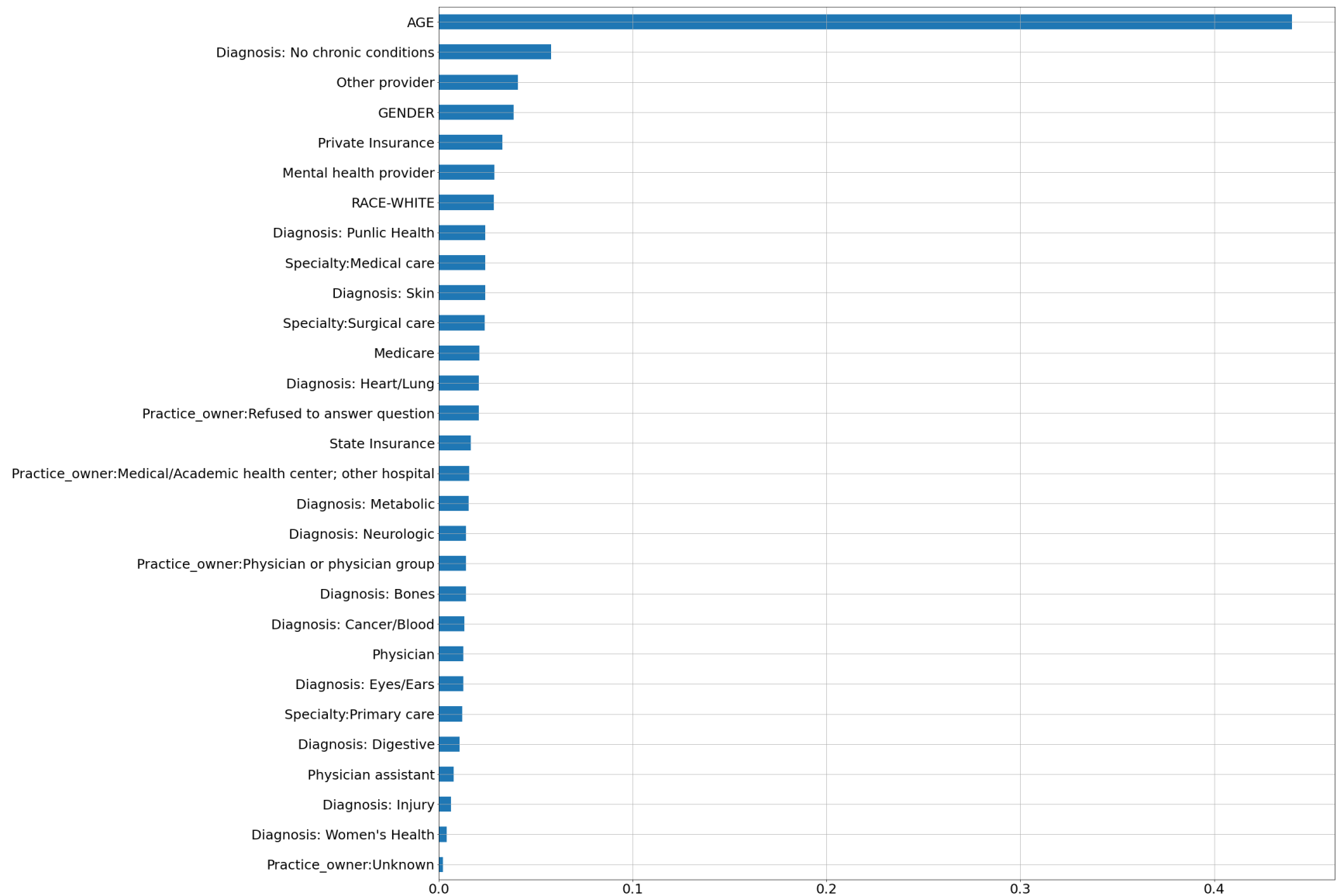
Model scores for namcs before smote

```
In [56]: skmets(ynam_test, prednam)
```

```
Out[56]: {'Accuracy': 0.8575531303848363,
          'Precision': 0.5204081632653061,
          'Sensitivity_recall': 0.3984375,
          'Specificity': 0.9367003367003367,
          'F1_score': 0.4513274336283186}
```

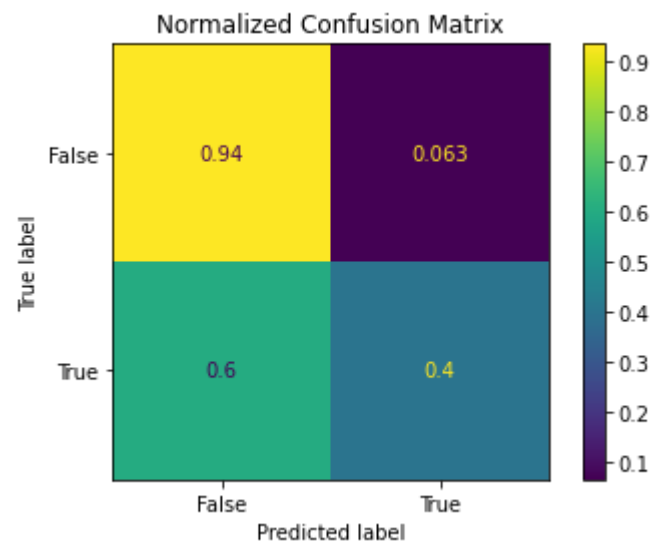
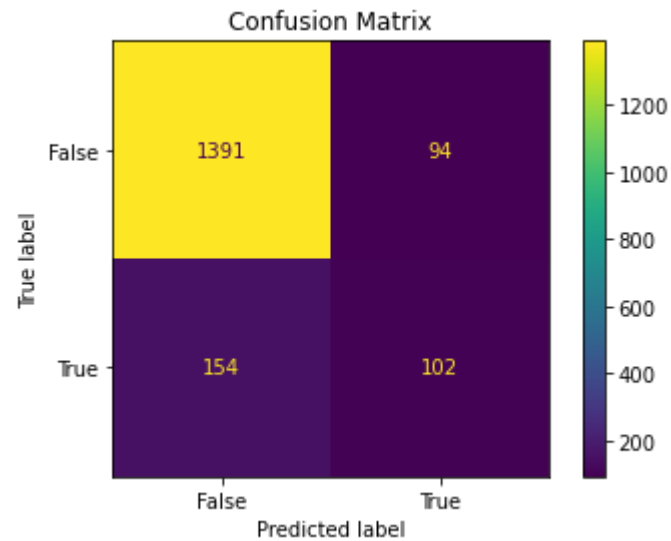
Feature Importance for predicting Depression for namcs before smote

```
In [57]: feat_imp(clf, namcs)
```



Confusion matrix for namcs before smote

```
In [58]: conf_mat(ynam_test, prednam)
```



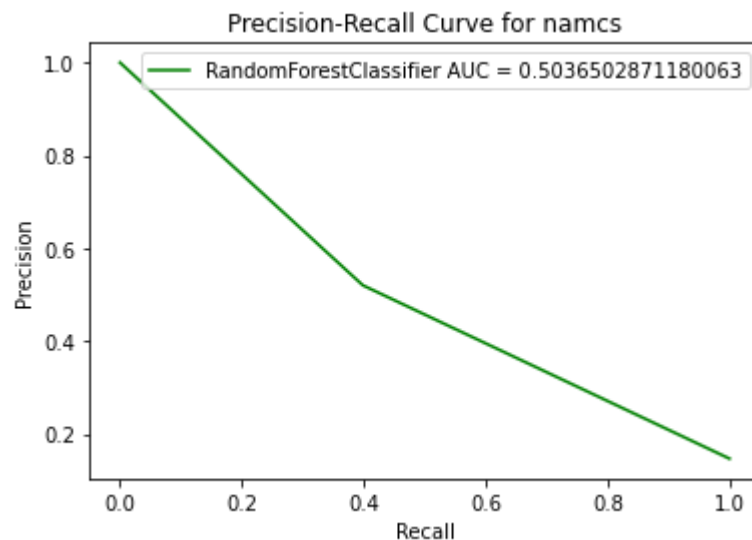
RANDOM FOREST namcs dataset Report smote

```
In [59]: print(classification_report(yname_test, predname))
```

	precision	recall	f1-score	support
0	0.90	0.94	0.92	1485
1	0.52	0.40	0.45	256
accuracy			0.86	1741
macro avg	0.71	0.67	0.68	1741
weighted avg	0.84	0.86	0.85	1741

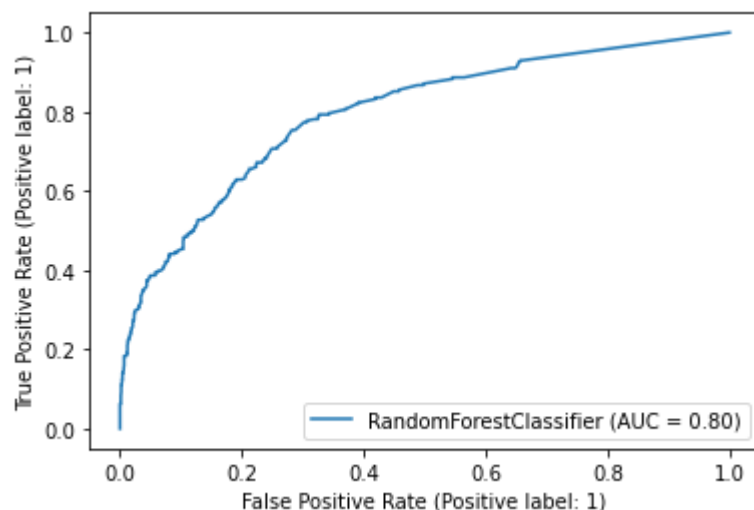
PR Curve for namcs before smote

```
In [60]: #calculate precision and recall
pr(yname_test, predname, 'Precision-Recall Curve for namcs', 'RandomForestClassifier AUC = ')
```



ROC Curve for namcs before smote

```
In [61]: roc(clf, Xname_test, yname_test)
```



Applying SMOTE for namcs dataset

```
In [62]: sm = SMOTE()
Xnh_train, ynh_train = sm.fit_resample(Xnh_train, ynh_train)
# namcs
clf.fit(Xnam_train, ynam_train)
prednam = clf.predict(Xnam_test)
```

Cross Validation for namcs after smote

```
In [63]: cross(clf, Xnam_train, ynam_train, 'f1_macro')

Mean f1_macro of 0.67 with a standard deviation of 0.02
```

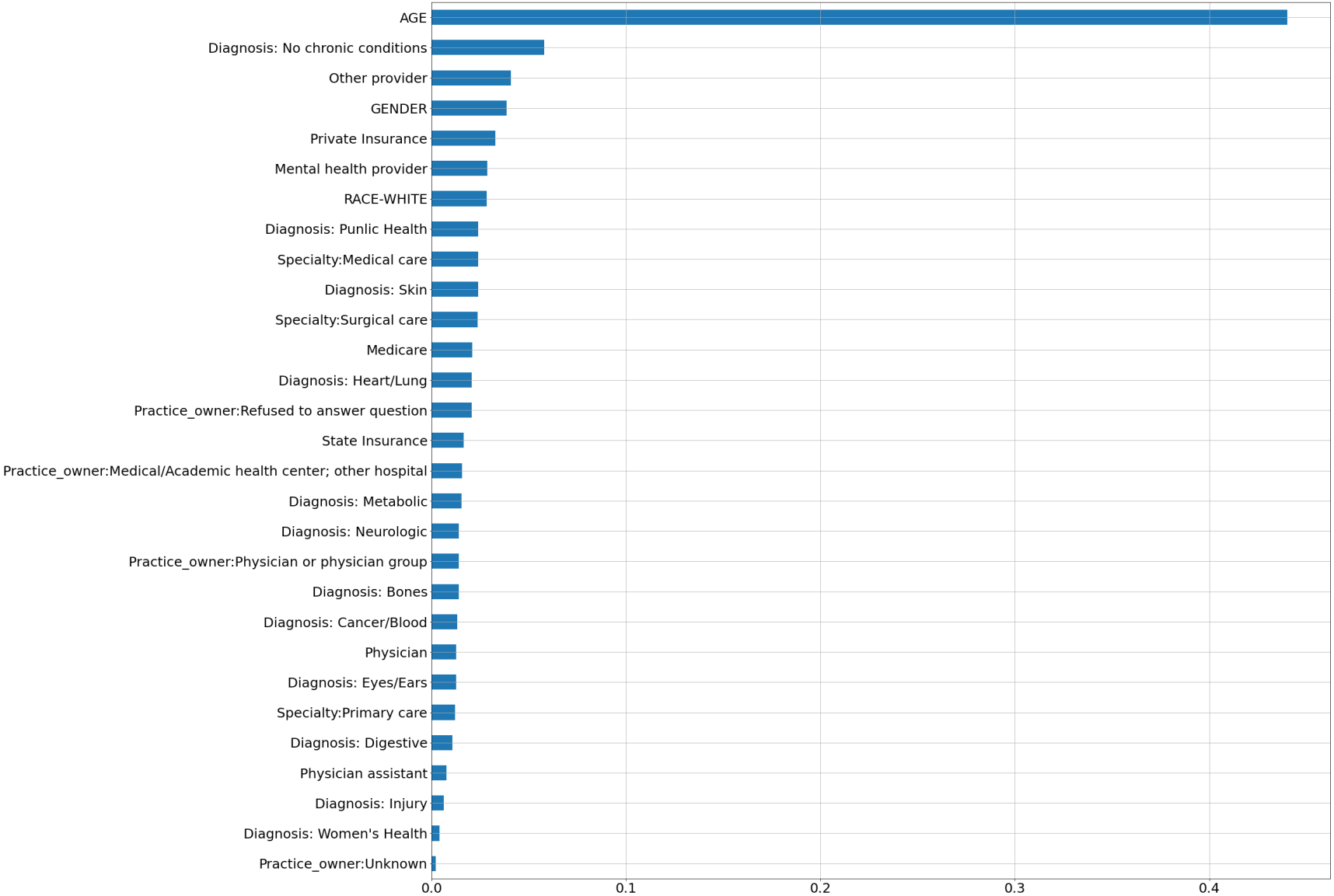
Model scores for namcs after smote

```
In [64]: skmets(ynam_test, prednam)
```

```
Out[64]: {'Accuracy': 0.8575531303848363,
'Precision': 0.5204081632653061,
'Sensitivity_recall': 0.3984375,
'Specificity': 0.9367003367003367,
'F1_score': 0.4513274336283186}
```

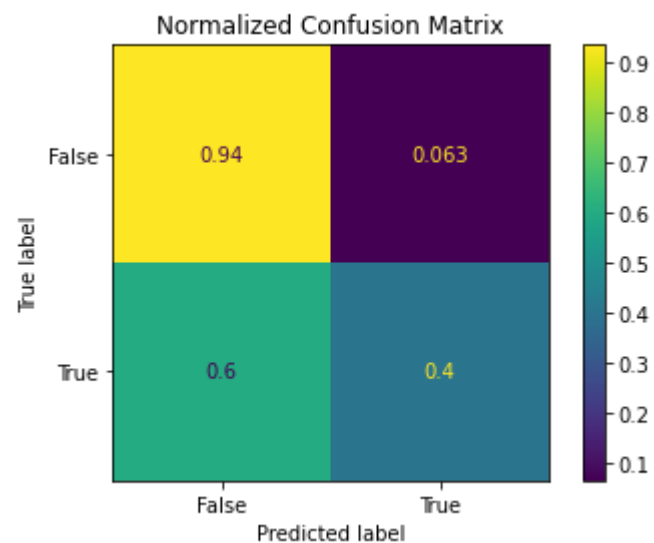
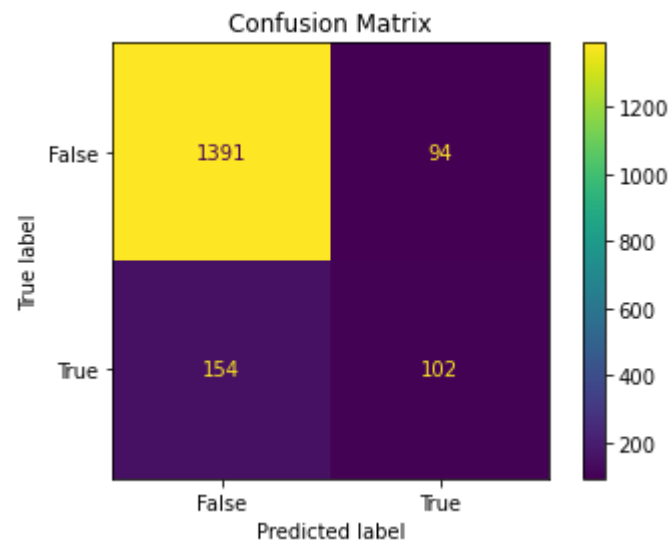
Feature Importance for predicting Depression for namcs after smote

```
In [65]: feat_imp(clf, namcs)
```



Confusion matrix for namcs after smote

```
In [66]: conf_mat(yname_test, predname)
```



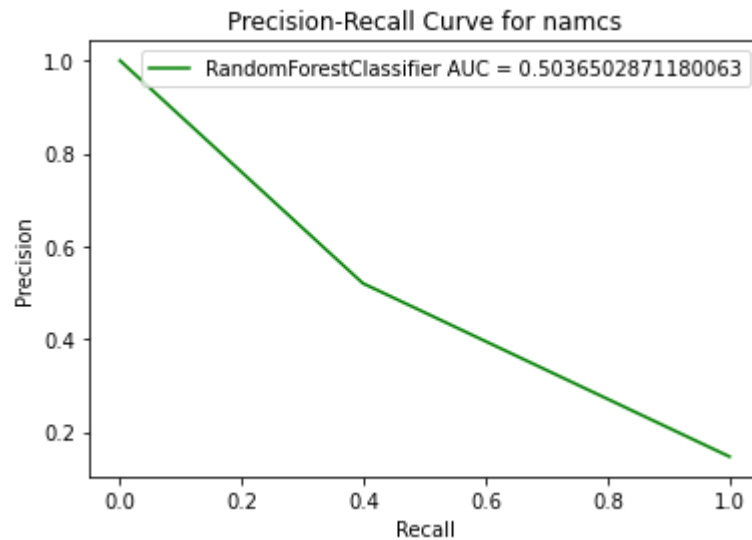
RANDOM FOREST namcs dataset Report after smote

```
In [67]: print(classification_report(yname_test, predname))
```

	precision	recall	f1-score	support
0	0.90	0.94	0.92	1485
1	0.52	0.40	0.45	256
accuracy			0.86	1741
macro avg	0.71	0.67	0.68	1741
weighted avg	0.84	0.86	0.85	1741

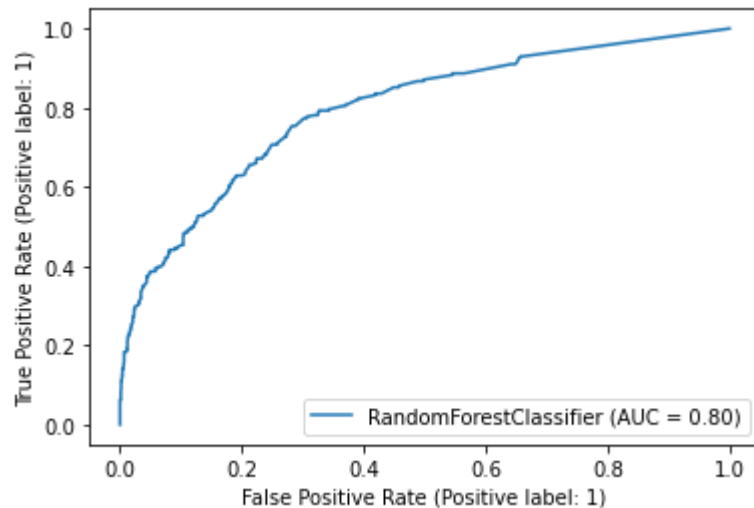
PR Curve for namcs after smote

```
In [68]: #calculate precision and recall
pr(yname_test, predname, 'Precision-Recall Curve for namcs', 'RandomForestClassifier AUC = ')
```



ROC Curve for namcs after smote

```
In [69]: roc(clf, Xname_test, yname_test)
```



nhanes dataset

```
In [70]: Xnh_train, Xnh_test, ynh_train, ynh_test = train_test_split(X_nh, y_nh, train_size=0.8, random_state=1000000000)
```

```
In [71]: # nhanes
clf.fit(Xnh_train, ynh_train)
prednh = clf.predict(Xnh_test)
```

Cross Validation for nhanes before smote

```
In [72]: cross(clf, Xnh_train, ynh_train, 'f1_macro')
```

Mean f1_macro of 0.57 with a standard deviation of 0.01

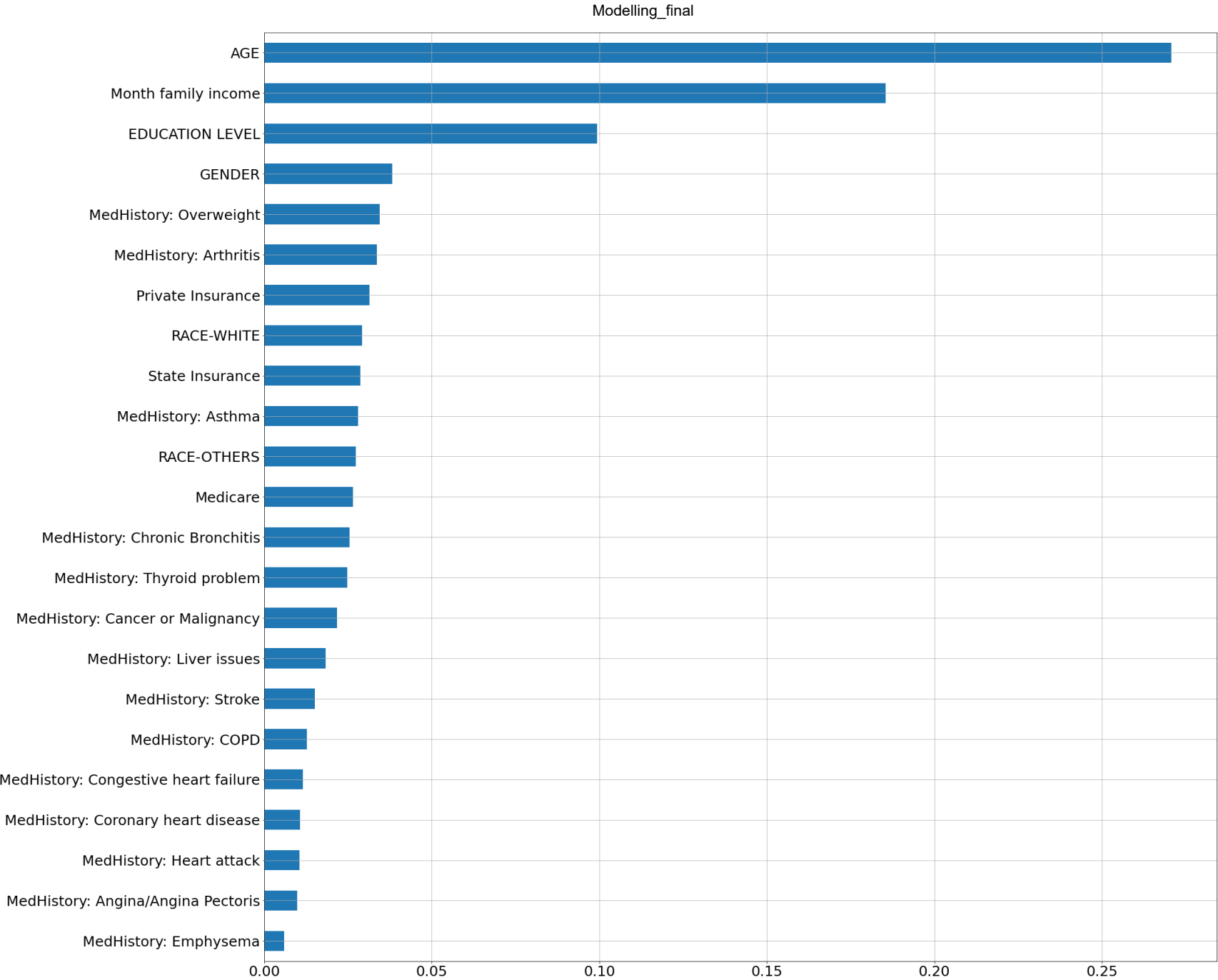
Model scores for nhanes before smote

```
In [73]: skmets(ynh_test, prednh)
```

```
Out[73]: {'Accuracy': 0.7244027303754266,
          'Precision': 0.3783783783783784,
          'Sensitivity_recall': 0.1951219512195122,
          'Specificity': 0.896045197740113,
          'F1_score': 0.2574712643678161}
```

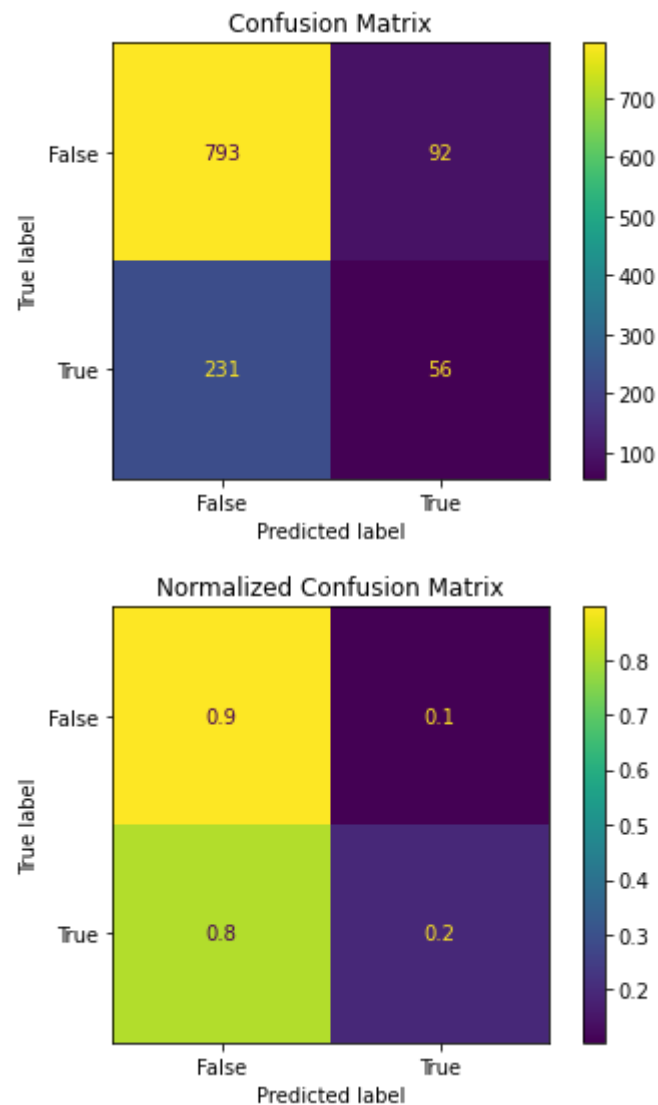
Feature Importance for predicting Depression for nhanes before smote

```
In [74]: feat_imp(clf, nhanes)
```



Confusion matrix for nhanes before smote

```
In [75]: conf_mat(ynh_test, prednh)
```



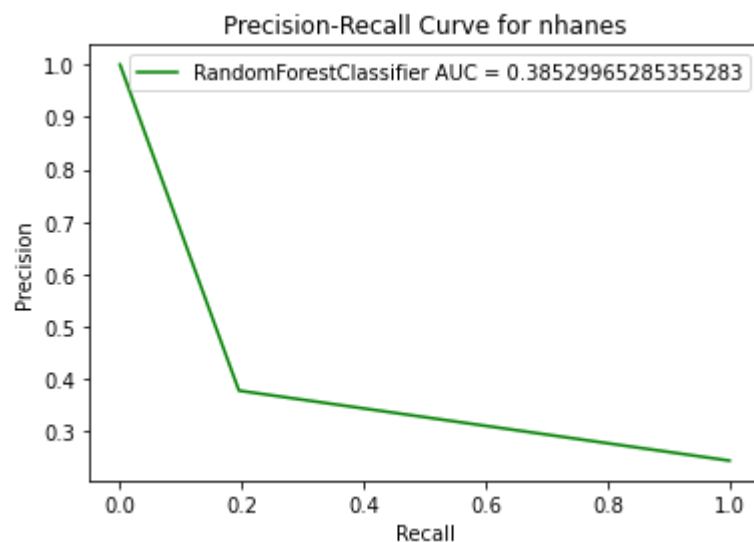
RANDOM FOREST nhanes dataset Report before smote

```
In [76]: print(classification_report(ynh_test, prednh))
```

	precision	recall	f1-score	support
0	0.77	0.90	0.83	885
1	0.38	0.20	0.26	287
accuracy			0.72	1172
macro avg	0.58	0.55	0.54	1172
weighted avg	0.68	0.72	0.69	1172

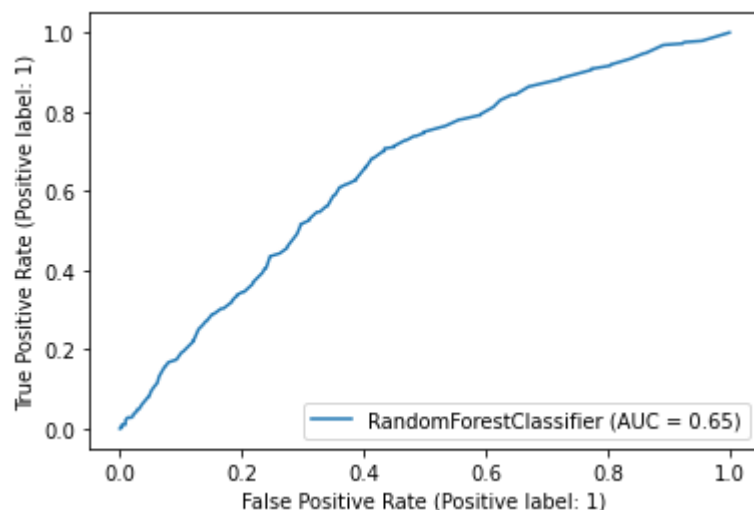
PR Curve for nhanes before smote

```
In [77]: #calculate precision and recall
pr(y nh_test, prednh, 'Precision-Recall Curve for nhanes', 'RandomForestClassifier AUC = ')
```



ROC Curve for nhanes before smote

```
In [78]: roc(clf, Xnh_test, ynh_test)
```



Applying SMOTE for nhanes dataset

```
In [79]: sm = SMOTE()
Xnh_train, ynh_train = sm.fit_resample(Xnh_train, ynh_train)
# nhanes
clf.fit(Xnh_train, ynh_train)
prednh = clf.predict(Xnh_test)
```

Cross Validation for nhanes after smote

```
In [80]: cross(clf, Xnh_train, ynh_train, 'f1_macro')

Mean f1_macro of 0.75 with a standard deviation of 0.07
```

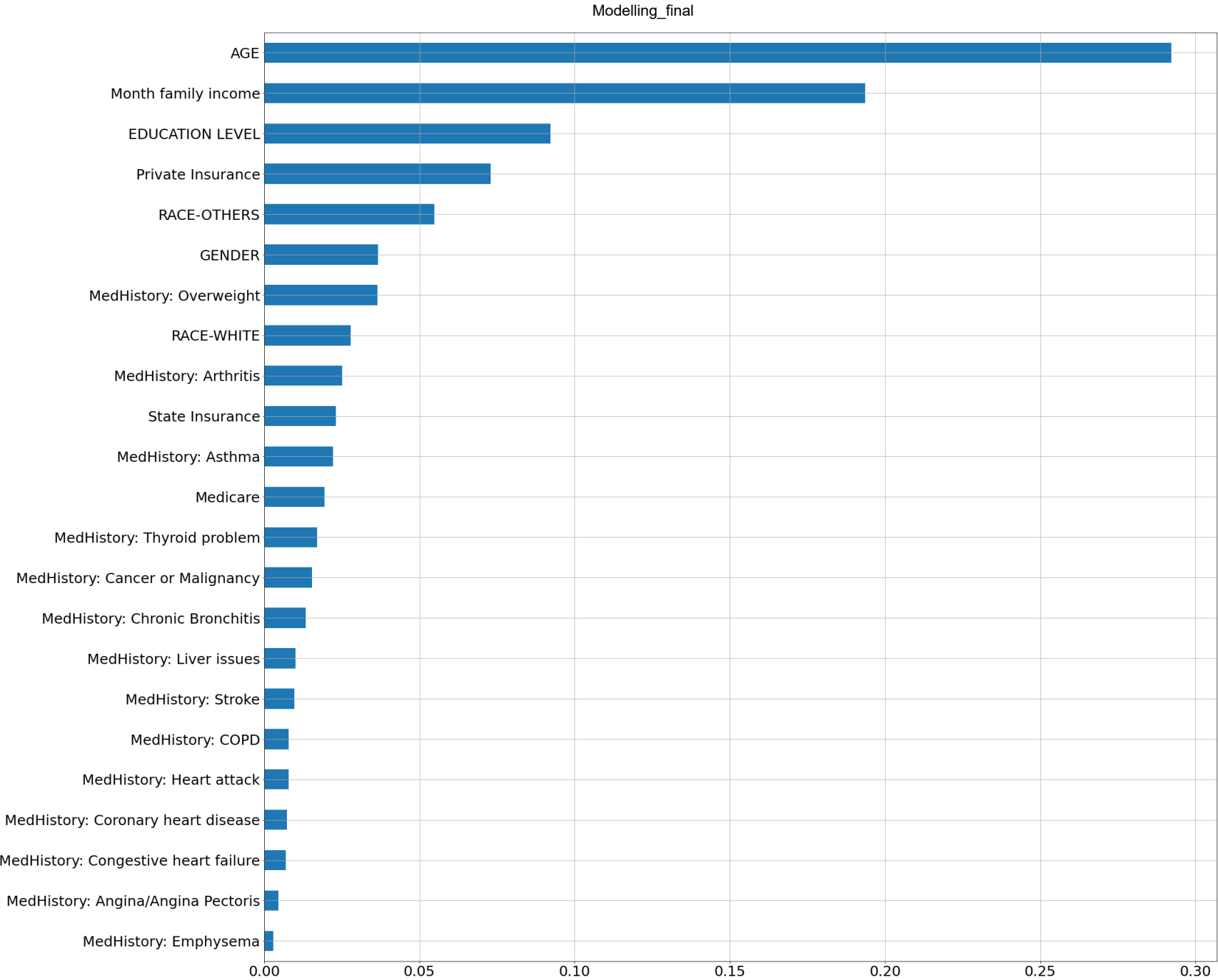
Model scores for nhanes after smote

```
In [81]: skmets(ynh_test, prednh)

Out[81]: {'Accuracy': 0.6331058020477816,
          'Precision': 0.3062330623306233,
          'Sensitivity_recall': 0.39372822299651566,
          'Specificity': 0.7107344632768362,
          'F1_score': 0.34451219512195125}
```

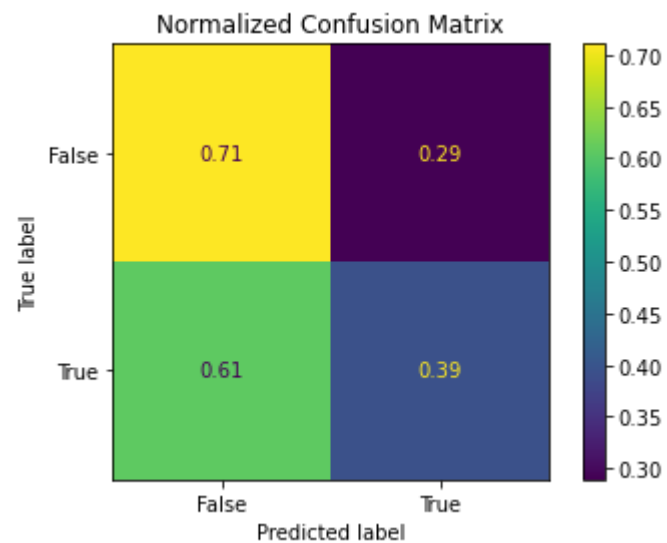
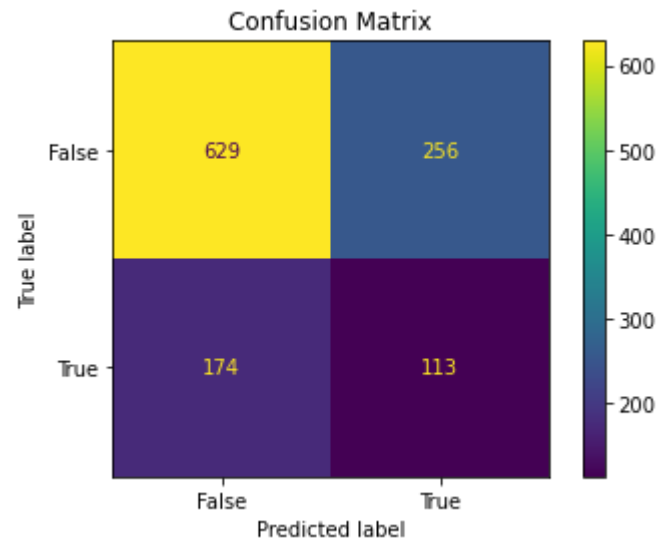
Feature Importance for predicting Depression for nhanes after smote

```
In [82]: feat_imp(clf, nhanes)
```



Confusion matrix for nhanes after smote

```
In [83]: conf_mat(ynh_test, prednh)
```



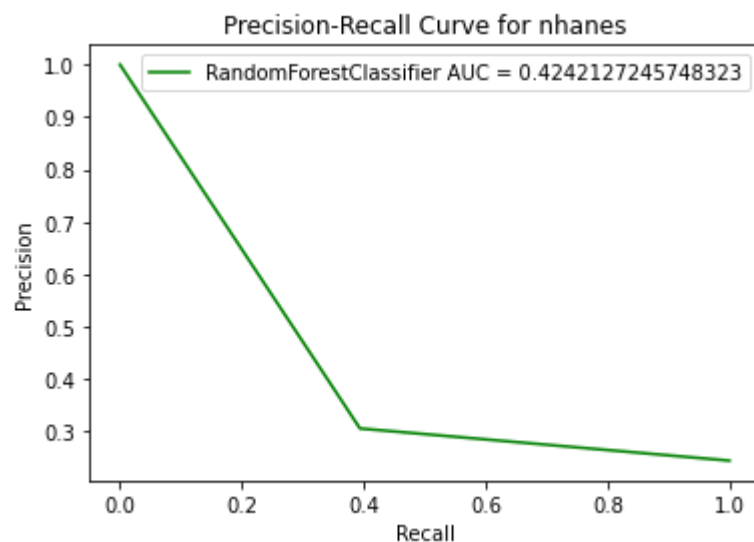
RANDOM FOREST nhanes dataset Report after smote

```
In [84]: print(classification_report(ynh_test, prednh))
```

	precision	recall	f1-score	support
0	0.78	0.71	0.75	885
1	0.31	0.39	0.34	287
accuracy			0.63	1172
macro avg	0.54	0.55	0.54	1172
weighted avg	0.67	0.63	0.65	1172

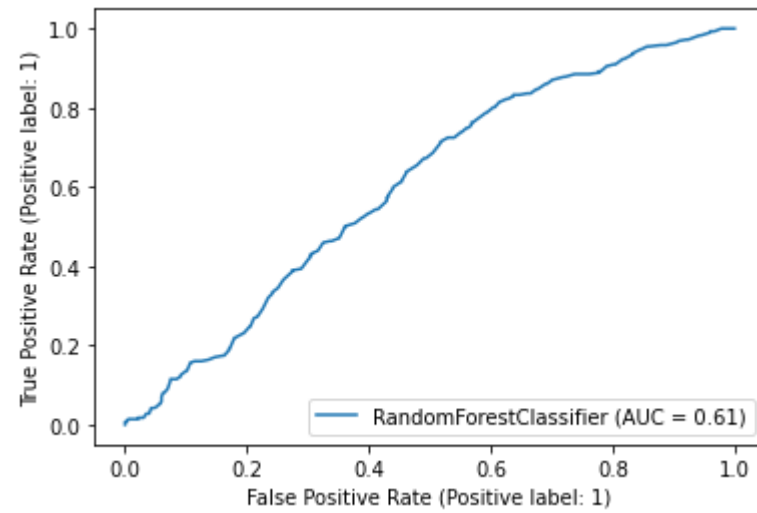
PR Curve for nhanes after smote

```
In [85]: #calculate precision and recall
pr(y nh_test, prednh, 'Precision-Recall Curve for nhanes', 'RandomForestClassifier AUC = ')
```



ROC Curve for nhanes after smote

```
In [86]: roc(clf, Xnh_test, ynh_test)
```



```
In [1]: import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import numpy as np
import statistics as st
from scipy.stats import iqr
```

DESCRIPTIVE STATISTICS

NHANES DATASET

```
In [2]: nhanesdf = pd.read_csv("nhanes_features_FINAL.csv")
print(nhanesdf.shape)
print(nhanesdf.info())
```

```

(5856, 40)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5856 entries, 0 to 5855
Data columns (total 40 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   SEQN        5856 non-null  float64
 1   DPQ010      5856 non-null  float64
 2   DPQ020      5856 non-null  float64
 3   DPQ030      5856 non-null  float64
 4   DPQ040      5856 non-null  float64
 5   DPQ050      5856 non-null  float64
 6   DPQ060      5856 non-null  float64
 7   DPQ070      5856 non-null  float64
 8   DPQ080      5856 non-null  float64
 9   DPQ090      5856 non-null  float64
10  AGE         5856 non-null  float64
11  INDHHIN2    5856 non-null  float64
12  MCQ010      5856 non-null  float64
13  MCQ080      5856 non-null  float64
14  MCQ160A     5856 non-null  float64
15  MCQ160B     5856 non-null  float64
16  MCQ160C     5856 non-null  float64
17  MCQ160D     5856 non-null  float64
18  MCQ160E     5856 non-null  float64
19  MCQ160F     5856 non-null  float64
20  MCQ160M     5856 non-null  float64
21  MCQ160G     5856 non-null  float64
22  MCQ160K     5856 non-null  float64
23  MCQ160O     5856 non-null  float64
24  MCQ220      5856 non-null  float64
25  DLQ140      5856 non-null  float64
26  DLQ150      5856 non-null  float64
27  DLQ170      5856 non-null  float64
28  INSPRVT     5856 non-null  float64
29  INSMCARE    5856 non-null  float64
30  INSNOCH     5856 non-null  float64
31  SEXML       5856 non-null  int64
32  RACEWH      5856 non-null  int64
33  RACEBL      5856 non-null  int64
34  RACEOT      5856 non-null  int64
35  INSSTATE    5856 non-null  int64
36  EDUC        5856 non-null  int64
37  MCQ700      5856 non-null  int64
38  PHQSCR      5856 non-null  float64

```

```

39 DI          5856 non-null    int64
dtypes: float64(32), int64(8)
memory usage: 1.8 MB
None

```

In [3]: `nhanesdf.head()`

Out[3]:

	SEQN	DPQ010	DPQ020	DPQ030	DPQ040	DPQ050	DPQ060	DPQ070	DPQ080	DPQ090	...	INSNOCH	SEXML	RACEWH	RACEBL
0	93705.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	1	0	1
1	93706.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0	0	0
2	93708.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	1	0	0
3	93709.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	1	0	1
4	93711.0	1.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0	0	0

5 rows × 40 columns

In [4]: `nhanesdf.rename(columns={'AGE': 'Age', 'SEXML': 'Gender', 'RACEBL': 'Black', 'RACEOT': 'Other Race', 'RACEWH': 'White', 'EDUC': 'Education', 'INDHHIN2': 'Income', 'INSMCARE': 'Medicare', 'INSNOCH': 'Uninsured', 'INSPRVT': 'Commercial', 'INSSTATE': 'State-funded', 'MCQ1600': 'COPD', 'MCQ160A': 'Arthritis', 'MCQ010': 'Asthma', 'MCQ160K': 'Bronchitis', 'MCQ160B': 'CHF', 'MCQ160C': 'Vascular', 'MCQ160D': 'Chest Pain', 'MCQ080': 'Overweight', 'MCQ160E': 'Heart attack', 'MCQ160F': 'Stroke', 'MCQ160G': 'Emphysema', 'MCQ160M': 'Thyroid', 'MCQ700': 'Liver', 'MCQ220': 'Cancer', 'DPQ010': 'DPQ1', 'DPQ020': 'DPQ2', 'DPQ030': 'DPQ3', 'DPQ040': 'DPQ4', 'DPQ050': 'DPQ5', 'DPQ060': 'DPQ6', 'DPQ070': 'DPQ7', 'DPQ080': 'DPQ8', 'DPQ090': 'DPQ9', 'PHQSCR': 'PHQ Score', 'DLQ140': 'Feels DPRN', 'DLQ150': 'DPRN Meds', 'DLQ170': 'DPRN Level', 'DI': 'DI'}, inplace=True)`

In [5]: `#Descriptive statistics for continuous data
nhanesdf1= nhanesdf[['Age', 'PHQ Score']]
nhanesdf1.describe()`

Out[5]:

	Age	PHQ Score
count	5856.000000	5856.000000
mean	49.886100	2.819672
std	18.775991	4.109342
min	18.000000	0.000000
25%	33.000000	0.000000
50%	51.000000	1.000000
75%	65.000000	4.000000
max	80.000000	25.000000

```
In [6]: print('median of Age:', nhanesdf.loc[:, 'Age'].median())
        print('median of PHQ Score:', nhanesdf.loc[:, 'PHQ Score'].median())
```

```
median of Age: 51.0
median of PHQ Score: 1.0
```

```
In [7]: # Calculating descriptive statistics for int datatypes
        cols=set(nhanesdf.columns)- {'SEQN'}
        nhanesdf2 = nhanesdf[list(cols)]
```

```
In [8]: nhanesdf2.mode().T
```

Out[8]:

	0
Cancer	0.0
DPQ1	0.0
Uninsured	0.0
DPQ4	0.0
Education	4.0
DPQ9	0.0
DPRN Meds	2.0
Overweight	0.0
COPD	0.0
Medicare	0.0
Gender	1.0
Bronchitis	0.0
Other Race	0.0
DPRN Level	0.0
Heart attack	0.0
Feels DPRN	5.0
Age	80.0
Arthritis	0.0
DPQ8	0.0
DPQ6	0.0
DPQ3	0.0
State-funded	0.0
White	0.0
Liver	0.0
Asthma	0.0

	0
DPQ7	0.0
Chest Pain	0.0
Thyroid	0.0
DI	0.0
Vascular	0.0
Income	15.0
Emphysema	0.0
Commercial	0.0
CHF	0.0
Black	0.0
DPQ5	0.0
Stroke	0.0
DPQ2	0.0
PHQ Score	0.0

In [9]:

nhanesdf2.std()

```
Out[9]: Cancer          0.300571
DPQ1          0.722374
Uninsured     0.013068
DPQ4          0.908591
Education     1.209707
DPQ9          0.282530
DPRN Meds     0.386546
Overweight    0.485585
COPD          0.218034
Medicare      0.443276
Gender        0.499817
Bronchitis    0.250825
Other Race    0.494181
DPRN Level    1.145233
Heart attack  0.209734
Feels DPRN    1.270755
Age           18.775991
Arthritis     0.453544
DPQ8          0.512650
DPQ6          0.586790
DPQ3          0.912182
State-funded  0.380029
White         0.476055
Liver         0.218386
Asthma        0.358370
DPQ7          0.637577
Chest Pain    0.163529
Thyroid       0.315210
DI            0.430253
Vascular      0.207876
Income        4.980565
Emphysema     0.133328
Commercial    0.499856
CHF           0.182075
Black         0.420443
DPQ5          0.738442
Stroke        0.210839
DPQ2          0.677685
PHQ Score     4.109342
dtype: float64
```

```
In [10]: nhanesdf2.var()
```

```
Out[10]:
```

Cancer	0.090343
DPQ1	0.521824
Uninsured	0.000171
DPQ4	0.825537
Education	1.463391
DPQ9	0.079823
DPRN Meds	0.149418
Overweight	0.235792
COPD	0.047539
Medicare	0.196494
Gender	0.249817
Bronchitis	0.062913
Other Race	0.244215
DPRN Level	1.311559
Heart attack	0.043988
Feels DPRN	1.614819
Age	352.537836
Arthritis	0.205702
DPQ8	0.262810
DPQ6	0.344322
DPQ3	0.832075
State-funded	0.144422
White	0.226628
Liver	0.047693
Asthma	0.128429
DPQ7	0.406504
Chest Pain	0.026742
Thyroid	0.099357
DI	0.185118
Vascular	0.043212
Income	24.806023
Emphysema	0.017776
Commercial	0.249856
CHF	0.033151
Black	0.176772
DPQ5	0.545297
Stroke	0.044453
DPQ2	0.459257
PHQ Score	16.886691

dtype: float64

```
In [11]: #Calculating IQR
def find_iqr(x):
    return np.subtract(*np.percentile(x, [75, 25]))
nhanesdf.loc[:, ~nhanesdf.columns.isin(['SEQN'])].apply(find_iqr)
```

```
Out[11]: DPQ1          0.0
          DPQ2          0.0
          DPQ3          1.0
          DPQ4          1.0
          DPQ5          0.0
          DPQ6          0.0
          DPQ7          0.0
          DPQ8          0.0
          DPQ9          0.0
          Age          32.0
          Income       10.0
          Asthma        0.0
          Overweight    1.0
          Arthritis     1.0
          CHF           0.0
          Vascular      0.0
          Chest Pain    0.0
          Heart attack  0.0
          Stroke        0.0
          Thyroid       0.0
          Emphysema     0.0
          Bronchitis    0.0
          COPD          0.0
          Cancer        0.0
          Feels DPRN    1.0
          DPRN Meds     0.0
          DPRN Level    2.0
          Commercial    1.0
          Medicare      1.0
          Uninsured     0.0
          Gender        1.0
          White         1.0
          Black         0.0
          Other Race    1.0
          State-funded  0.0
          Education     1.0
          Liver         0.0
          PHQ Score     4.0
          DI            0.0
          dtype: float64
```

```
In [12]: nhanesdf2.skew()
```

```
Out[12]:
```

Cancer	2.659779
DPQ1	2.309571
Uninsured	76.524506
DPQ4	1.330228
Education	-0.522754
DPQ9	7.380971
DPRN Meds	-2.990925
Overweight	0.491801
COPD	4.128892
Medicare	1.044341
Gender	-0.060152
Bronchitis	3.450180
Other Race	0.309028
DPRN Level	0.803400
Heart attack	4.329761
Feels DPRN	-1.398617
Age	-0.055676
Arthritis	0.928797
DPQ8	4.010407
DPQ6	3.180848
DPQ3	1.598542
State-funded	1.710801
White	0.643026
Liver	4.120670
Asthma	1.946713
DPQ7	3.097025
Chest Pain	5.780836
Thyroid	2.463636
DI	1.184736
Vascular	4.376679
Income	0.039484
Emphysema	7.231217
Commercial	0.054679
CHF	5.116961
Black	1.287953
DPQ5	2.317683
Stroke	4.302201
DPQ2	2.478830
PHQ Score	2.076570

dtype: float64

```
In [13]: nhanesdf2.kurt()
```

```

Out[13]:
Cancer          5.076159
DPQ1            4.738619
Uninsured      5856.000000
DPQ4            0.825129
Education       -0.537965
DPQ9            61.527603
DPRN Meds       8.733571
Overweight     -1.758732
COPD            15.052891
Medicare       -0.909664
Gender         -1.997064
Bronchitis      9.907129
Other Race     -1.905152
DPRN Level     -0.855055
Heart attack    16.752549
Feels DPRN      1.189707
Age            -1.167783
Arthritis      -1.137726
DPQ8            16.558272
DPQ6            10.304989
DPQ3            1.439218
State-funded    0.927158
White          -1.587060
Liver           14.985042
Asthma          1.790303
DPQ7            9.268403
Chest Pain     31.428794
Thyroid         4.070895
DI             -0.596604
Vascular        17.161184
Income         -1.174515
Emphysema      50.307686
Commercial     -1.997693
CHF            24.191547
Black          -0.341295
DPQ5            4.681022
Stroke         16.514569
DPQ2            5.843077
PHQ Score      4.672541
dtype: float64

```

NAMCS DATASET

```
In [14]: namcsdf = pd.read_csv("namcs_features_FINAL.csv")  
         print(namcsdf.shape)  
         print(namcsdf.info())
```

```

(8701, 48)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8701 entries, 0 to 8700
Data columns (total 48 columns):
#   Column      Non-Null Count  Dtype
---  -
0   SEQN        8701 non-null  int64
1   AGE         8701 non-null  float64
2   SEXML       8701 non-null  int64
3   RACEWH      8701 non-null  int64
4   RACEBL      8701 non-null  int64
5   RACEOT      8701 non-null  int64
6   NOPROVID    8701 non-null  float64
7   PHYS        8701 non-null  float64
8   PHYSASST   8701 non-null  float64
9   NPNMW       8701 non-null  float64
10  RNLPN       8701 non-null  float64
11  MHP         8701 non-null  float64
12  OTHPROV     8701 non-null  float64
13  PROVNONE    8701 non-null  float64
14  INSMCARE    8701 non-null  int64
15  INSSTATE    8701 non-null  int64
16  INSNOCH     8701 non-null  int64
17  INSPRVT     8701 non-null  int64
18  SPPRC       8701 non-null  int64
19  SPSUR       8701 non-null  int64
20  SPMEC       8701 non-null  int64
21  OWNUNKN     8701 non-null  int64
22  OWNPHYS     8701 non-null  int64
23  OWNHOSP     8701 non-null  int64
24  OWNINSR     8701 non-null  int64
25  DIAGALL     8701 non-null  object
26  NOCHRON     8701 non-null  int64
27  DGRP1       8701 non-null  int64
28  DGRP2       8701 non-null  int64
29  DGRP3       8701 non-null  int64
30  DGRP4       8701 non-null  int64
31  DGRP5       8701 non-null  int64
32  DGRP6       8701 non-null  int64
33  DGRP7       8701 non-null  int64
34  DGRP8       8701 non-null  int64
35  DGRP9       8701 non-null  int64
36  DGRP10      8701 non-null  int64
37  DGRP11      8701 non-null  int64
38  DGRP12      8701 non-null  int64

```

```

39 DGRP13      8701 non-null   int64
40 DGRP14      8701 non-null   int64
41 DGRP15      8701 non-null   int64
42 DGRP16      8701 non-null   int64
43 DEPRN       8701 non-null   float64
44 DEPRESS     8701 non-null   float64
45 MENTAL      8701 non-null   float64
46 PSYCHOTH    8701 non-null   float64
47 DI          8701 non-null   int64
dtypes: float64(13), int64(34), object(1)
memory usage: 3.2+ MB
None

```

In [15]: `namcsdf.head()`

Out[15]:

	SEQN	AGE	SEXML	RACEWH	RACEBL	RACEOT	NOPROVID	PHYS	PHYSASST	NPNMW	...	DGRP12	DGRP13	DGRP14	DGRP15	DGI
0	11000	69.0	1	1	0	0	0.0	1.0	0.0	0.0	...	0	1	0	0	
1	11001	64.0	0	1	0	0	0.0	1.0	0.0	0.0	...	0	0	0	0	
2	11002	81.0	0	1	0	0	0.0	1.0	0.0	0.0	...	0	0	0	0	
3	11003	66.0	0	1	0	0	0.0	1.0	0.0	0.0	...	0	0	0	0	
4	11004	44.0	0	1	0	0	0.0	1.0	0.0	0.0	...	0	0	0	0	

5 rows × 48 columns

In [16]: `namcsdf.rename(columns={'AGE': 'Age', 'SEXML': 'Gender', 'RACEWH': 'White', 'RACEBL': 'Black', 'RACEOT': 'Other Race', 'SPMEC': 'Specialty Care', 'SPPRC': 'Primary Care', 'SPSUR': 'Surgery Care', 'RNLPN': 'Nurse', 'PHYS': 'MD/DO', 'PHYSASST': 'PA', 'NPNMW': 'NPNMW', 'MHP': 'MHP', 'PROVNONE': 'No provider', 'OTHPROV': 'Other provider', 'NOPROVID': 'Provider-withheld', 'OWNPHYS': 'Phys. Owner', 'OWNHOSP': 'Hospital Owner', 'OWNUNK': 'Owner-withheld', 'OWNINSR': 'Insurance Owner', 'OWNUNKN': 'Unk Practice', 'INSPRVT': 'Commercial', 'INSMCARE': 'Medicare', 'INSSTATE': 'State-fun', 'INSNOCH': 'Uninsured', 'DGRP1': 'Infection', 'DGRP10': 'Urinary', 'DGRP11': 'Womens Health', 'DGRP12': 'Genetic', 'DGRP13': 'Other Disorder', 'DGRP14': 'Injury', 'DGRP15': 'Morbidity', 'DGRP16': 'Public health', 'DGRP2': 'Cancer/Blood', 'DGRP3': 'Metabolic', 'DGRP4': 'Neurologic', 'DGRP5': 'Eyes/Ears', 'DGRP6': 'Heart/Lung', 'DGRP7': 'Digestive', 'DGRP8': 'Skin', 'DGRP9': 'Bones', 'NOCHRON': 'No illness', 'DEPRN': 'Chronic DPRN', 'MENTAL': 'Therapy Referral', 'PSYCHOTH': 'MHP Refer', 'DEPRESS': 'DPRN Screen', 'DI': 'DI'}, inplace=True)`

```
In [17]: #Descriptive statistics for continuous data  
namcsdf1= namcsdf['Age']  
namcsdf1.describe()
```

```
Out[17]: count      8701.000000  
mean        58.104586  
std         18.441697  
min         18.000000  
25%         45.000000  
50%         61.000000  
75%         72.000000  
max         93.000000  
Name: Age, dtype: float64
```

```
In [18]: print('median of Age:', namcsdf.loc[:, 'Age'].median())  
  
median of Age: 61.0
```

```
In [19]: # Calculating descriptive statistics for int datatypes  
cols=set(namcsdf.columns)- {'SEQN', 'DIAGALL'}  
namcsdf2 = namcsdf[list(cols)]
```

```
In [20]: namcsdf2.mode().T
```

Out[20]:

	0
Metabolic	0.0
Skin	0.0
Phys. Owner	1.0
Uninsured	0.0
Chronic DPRN	0.0
Insurance Owner	0.0
Digestive	0.0
Provider-withheld	0.0
Other provider	0.0
Medicare	0.0
Gender	0.0
Urinary	0.0
Eyes/Ears	0.0
Other Race	0.0
NPNMW	0.0
Age	71.0
Specialty Care	0.0
Genetic	0.0
PA	0.0
Other Disorder	0.0
State-funded	0.0
Neurologic	0.0
White	1.0
No provider	0.0
MHP Referral	0.0

	0
Womens Health	0.0
Morbidity	0.0
DPRN Screen	0.0
Public health	0.0
No illness	0.0
Nurse	0.0
Surgery Care	0.0
Hospital Owner	0.0
Infection	0.0
Heart/Lung	0.0
DI	0.0
MD/DO	1.0
Cancer/Blood	0.0
Injury	0.0
Commercial	0.0
Unk Practice	0.0
Bones	0.0
MHP	0.0
Therapy Referral	0.0
Black	0.0
Primary Care	0.0

```
In [21]: namcsdf2.std()
```

```

Out[21]:
Metabolic      0.212155
Skin           0.284363
Phys. Owner    0.408861
Uninsured      0.154189
Chronic DPRN   0.304246
Insurance Owner 0.323344
Digestive      0.174650
Provider-withheld 0.032147
Other provider  0.461888
Medicare       0.479065
Gender         0.493946
Urinary        0.228988
Eyes/Ears      0.347109
Other Race     0.234482
NPNMW          0.108679
Age            18.441697
Specialty Care 0.460783
Genetic        0.050223
PA             0.181878
Other Disorder 0.277079
State-funded   0.271472
Neurologic     0.171220
White          0.338943
No provider    0.021437
MHP Referral   0.141956
Womens Health  0.119936
Morbidity      0.000000
DPRN Screen    0.199959
Public health  0.365111
No illness     0.471661
Nurse          0.300919
Surgery Care   0.494494
Hospital Owner 0.214591
Infection      0.042845
Heart/Lung     0.264779
DI             0.361415
MD/DO          0.114698
Cancer/Blood   0.180697
Injury         0.149501
Commercial     0.495847
Unk Practice   0.055622
Bones          0.249644
MHP            0.106065
Therapy Referral 0.138014
Black          0.261810

```

Primary Care
dtype: float64

0.443131

In [22]: `namcsdf2.var()`

```

Out[22]:
Metabolic      0.045010
Skin           0.080863
Phys. Owner    0.167167
Uninsured      0.023774
Chronic DPRN   0.092566
Insurance Owner 0.104551
Digestive      0.030502
Provider-withheld 0.001033
Other provider 0.213341
Medicare       0.229503
Gender         0.243983
Urinary        0.052435
Eyes/Ears      0.120485
Other Race     0.054982
NPNMW          0.011811
Age            340.096187
Specialty Care 0.212321
Genetic        0.002522
PA             0.033080
Other Disorder 0.076773
State-funded   0.073697
Neurologic     0.029316
White          0.114882
No provider    0.000460
MHP Referral   0.020151
Womens Health  0.014385
Morbidity      0.000000
DPRN Screen    0.039983
Public health  0.133306
No illness     0.222464
Nurse          0.090552
Surgery Care   0.244524
Hospital Owner 0.046049
Infection      0.001836
Heart/Lung     0.070108
DI             0.130621
MD/DO         0.013156
Cancer/Blood   0.032651
Injury         0.022350
Commercial     0.245864
Unk Practice   0.003094
Bones          0.062322
MHP            0.011250
Therapy Referral 0.019048
Black          0.068544

```

Primary Care
dtype: float64

0.196365

```
In [23]: #Calculating IQR
def find_iqr(x):
    return np.subtract(*np.percentile(x, [75, 25]))
namcsdf.loc[:, ~namcsdf.columns.isin(['SEQN', 'DIAGALL'])].apply(find_iqr)
```

```
Out[23]:
```

Age	27.0
Gender	1.0
White	0.0
Black	0.0
Other Race	0.0
Provider-withheld	0.0
MD/DO	0.0
PA	0.0
NPNMW	0.0
Nurse	0.0
MHP	0.0
Other provider	1.0
No provider	0.0
Medicare	1.0
State-funded	0.0
Uninsured	0.0
Commercial	1.0
Primary Care	1.0
Surgery Care	1.0
Specialty Care	1.0
Unk Practice	0.0
Phys. Owner	0.0
Hospital Owner	0.0
Insurance Owner	0.0
No illness	1.0
Infection	0.0
Cancer/Blood	0.0
Metabolic	0.0
Neurologic	0.0
Eyes/Ears	0.0
Heart/Lung	0.0
Digestive	0.0
Skin	0.0
Bones	0.0
Urinary	0.0
Womens Health	0.0
Genetic	0.0
Other Disorder	0.0
Injury	0.0
Morbidity	0.0
Public health	0.0
Chronic DPRN	0.0
DPRN Screen	0.0
Therapy Referral	0.0
MHP Referral	0.0

DI
dtype: float64 0.0

In [24]: `namcsdf2.skew()`

```

Out[24]:
Metabolic      4.269213
Skin           2.893264
Phys. Owner    -1.408336
Uninsured      6.170944
Chronic DPRN   2.608973
Insurance Owner 2.359595
Digestive      5.366369
Provider-withheld 31.050140
Other provider  0.829524
Medicare       0.598217
Gender         0.314883
Urinary        3.883100
Eyes/Ears      2.074181
Other Race     3.767605
NPNMW          8.983506
Age            -0.361364
Specialty Care  0.842989
Genetic        19.815093
PA             5.122743
Other Disorder  3.005010
State-funded   3.094169
Neurologic     5.488643
White          -2.169603
No provider    46.615444
MHP Referral   6.756159
Womens Health  8.096247
Morbidity      0.000000
DPRN Screen    4.584810
Public health  1.871792
No illness     0.704122
Nurse          2.654624
Surgery Care   0.300121
Hospital Owner 4.210037
Infection      23.259425
Heart/Lung     3.204521
DI             1.912564
MD/DO         -8.488049
Cancer/Blood   5.161336
Injury         6.384440
Commercial     0.260353
Unk Practice   17.870996
Bones          3.471543
MHP            9.215730
Therapy Referral 6.965755
Black          3.254912

```

Primary Care
dtype: float64

1.045709

In [25]: `namcsdf2.kurt()`

```

Out[25]:
Metabolic      16.229909
Skin           6.372440
Phys. Owner    -0.016593
Uninsured      36.088851
Chronic DPRN   4.807845
Insurance Owner 3.568509
Digestive      26.804079
Provider-withheld 962.332415
Other provider  -1.312192
Medicare       -1.642514
Gender         -1.901286
Urinary        13.081472
Eyes/Ears      2.302755
Other Race     12.197655
NPNMW          78.721478
Age            -0.739763
Specialty Care -1.289665
Genetic        390.727719
PA             24.248071
Other Disorder  7.031703
State-funded   7.575624
Neurologic     28.131671
White          2.707800
No provider    2171.498793
MHP Referral   43.655715
Womens Health  63.563834
Morbidity      0.000000
DPRN Screen    19.024853
Public health  1.503951
No illness     -1.504558
Nurse          5.048187
Surgery Care   -1.910367
Hospital Owner 15.728030
Infection      539.124788
Heart/Lung     8.270854
DI             1.658284
MD/DO          70.063077
Cancer/Blood   24.645058
Injury         38.769989
Commercial     -1.932660
Unk Practice   317.445451
Bones          10.053921
MHP            82.948746
Therapy Referral 46.532440
Black          8.596429

```

Primary Care
dtype: float64

-0.906701

EXPLORATORY DATA ANALYSIS

```
In [26]: namcsdf=pd.read_csv('namcs_features_FINAL.csv')
```

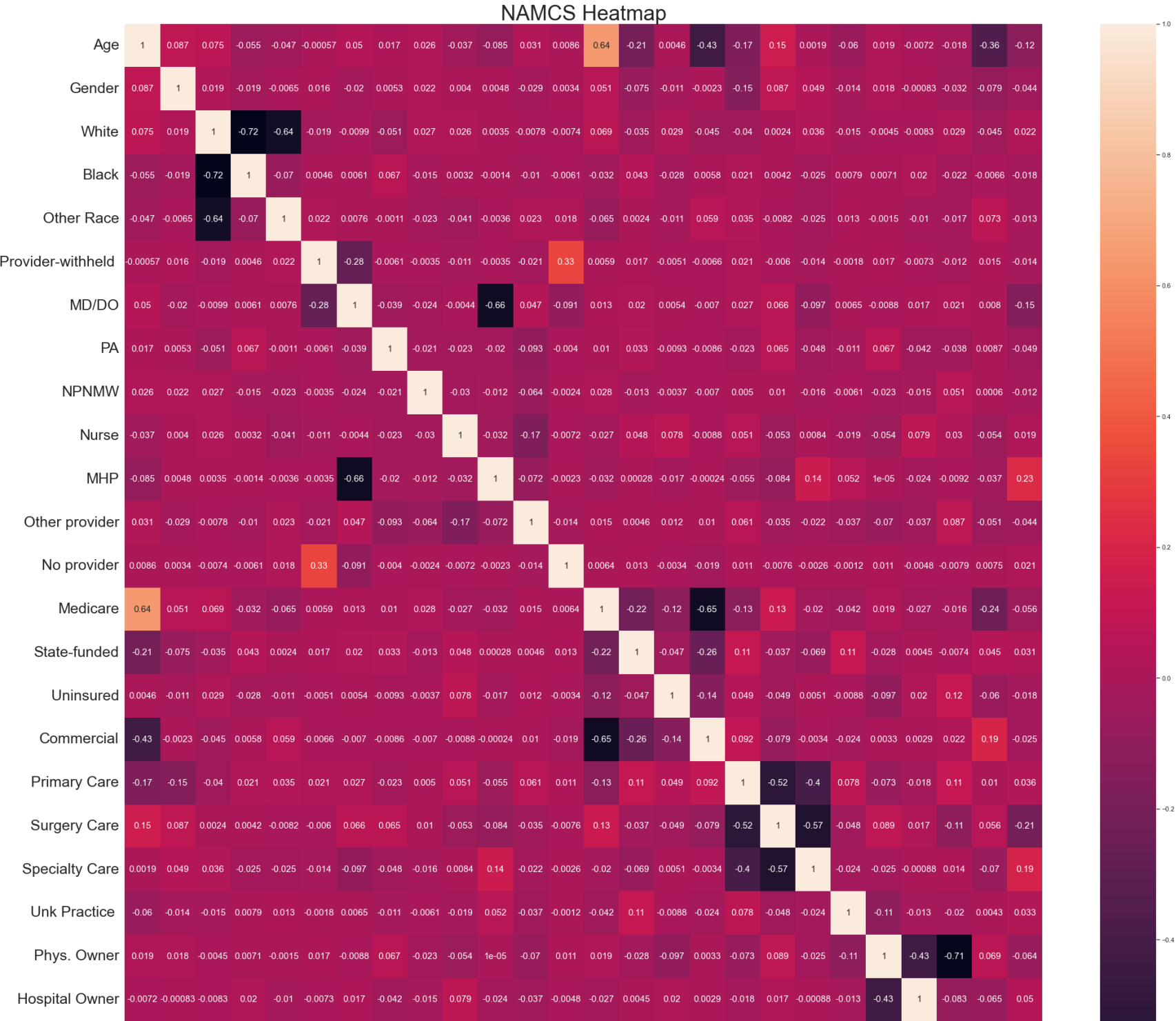
```
In [27]: namcsdf.rename(columns={'AGE': 'Age', 'SEXML': 'Gender', 'RACEWH': 'White', 'RACEBL': 'Black', 'RACEOT': 'Other Race', 'SP
```

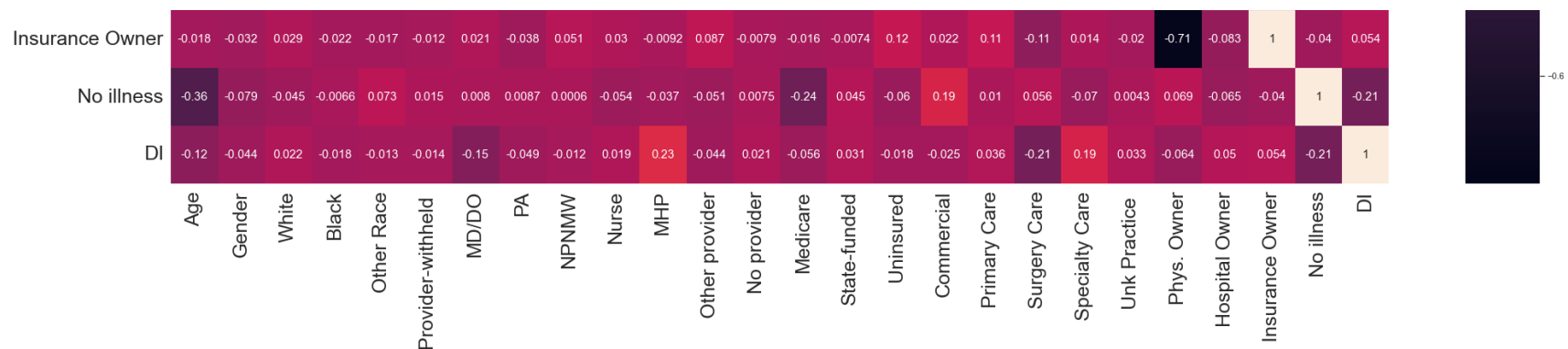
```
In [28]: nhanesdf=pd.read_csv('nhanes_features_FINAL.csv')
```

```
In [29]: nhanesdf.rename(columns={'AGE': 'Age', 'SEXML': 'Gender', 'RACEBL': 'Black', 'RACEOT': 'Other Race', 'RACEWH': 'White', 'EDUC': 'E
```

Heatmap of NAMCS

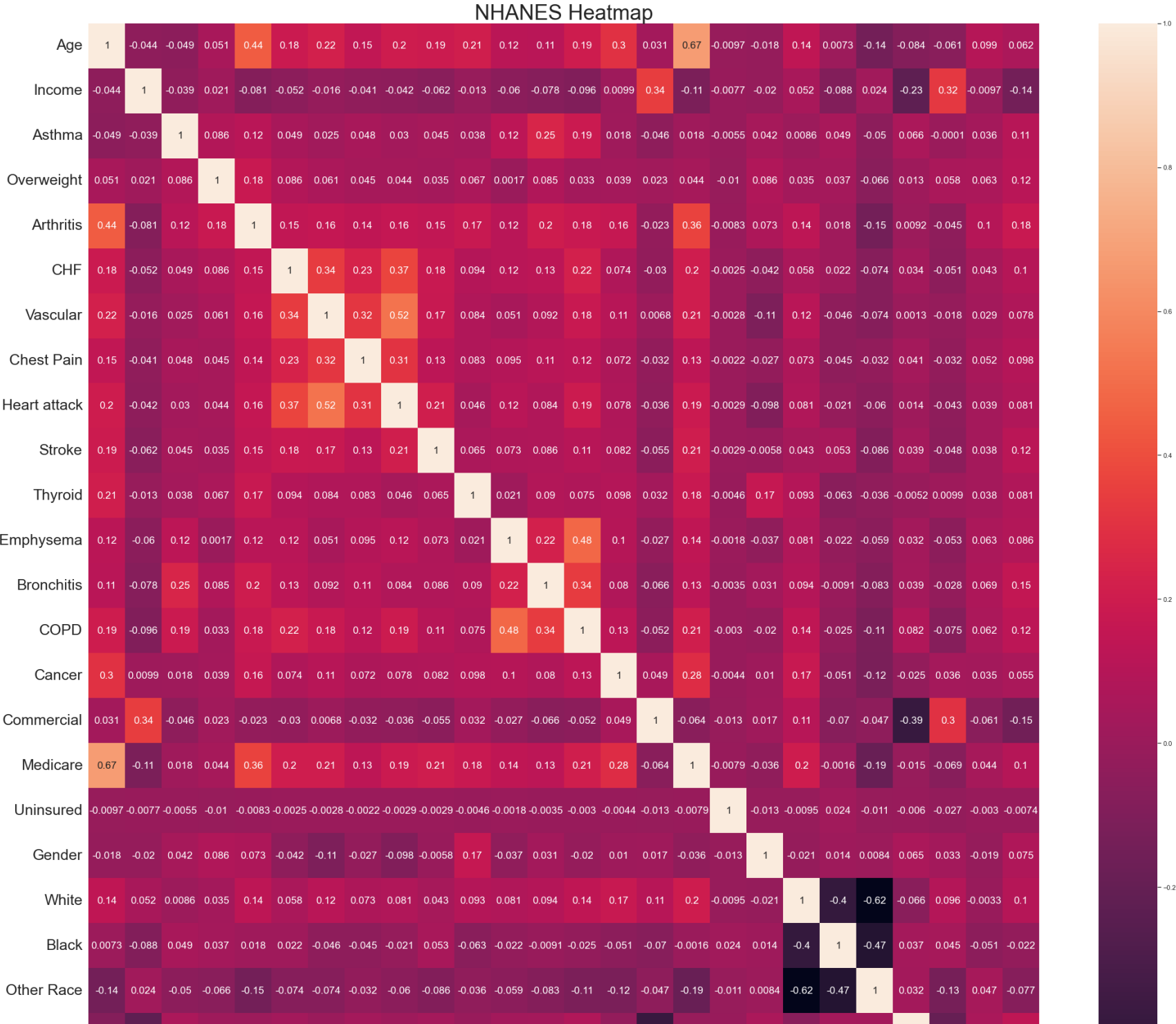
```
In [51]: datanam = namcsdf.drop(['SEQN', 'Chronic DPRN', 'DPRN Screen', 'Therapy Referral', 'MHP Referral', 'Infection', 'Urinary', 'Wo  
g = sns.heatmap(datanam.corr(),annot=True, annot_kws={"fontsize":15})  
g.set_yticklabels(g.get_yticklabels(),size=25, rotation=0)  
g.set_xticklabels(g.get_xticklabels(),size=25, rotation=90)  
g.set_title('NAMCS Heatmap',size=35)  
sns.set(rc = {'figure.figsize':(30,30)})  
plt.tight_layout()  
plt.savefig('fd NAMCS_Heatmap.png',dpi= 1200)  
plt.show()
```

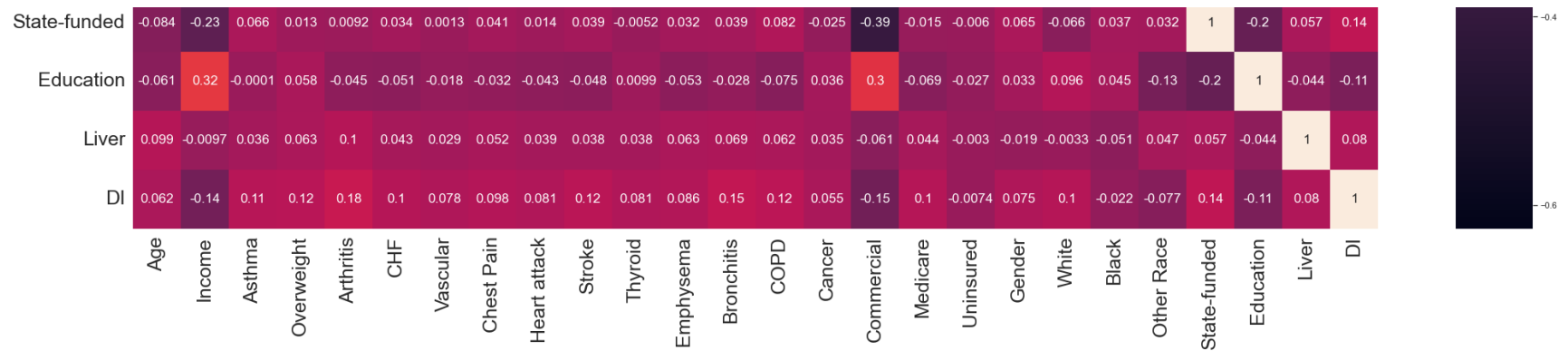




Heatmap of NHANES

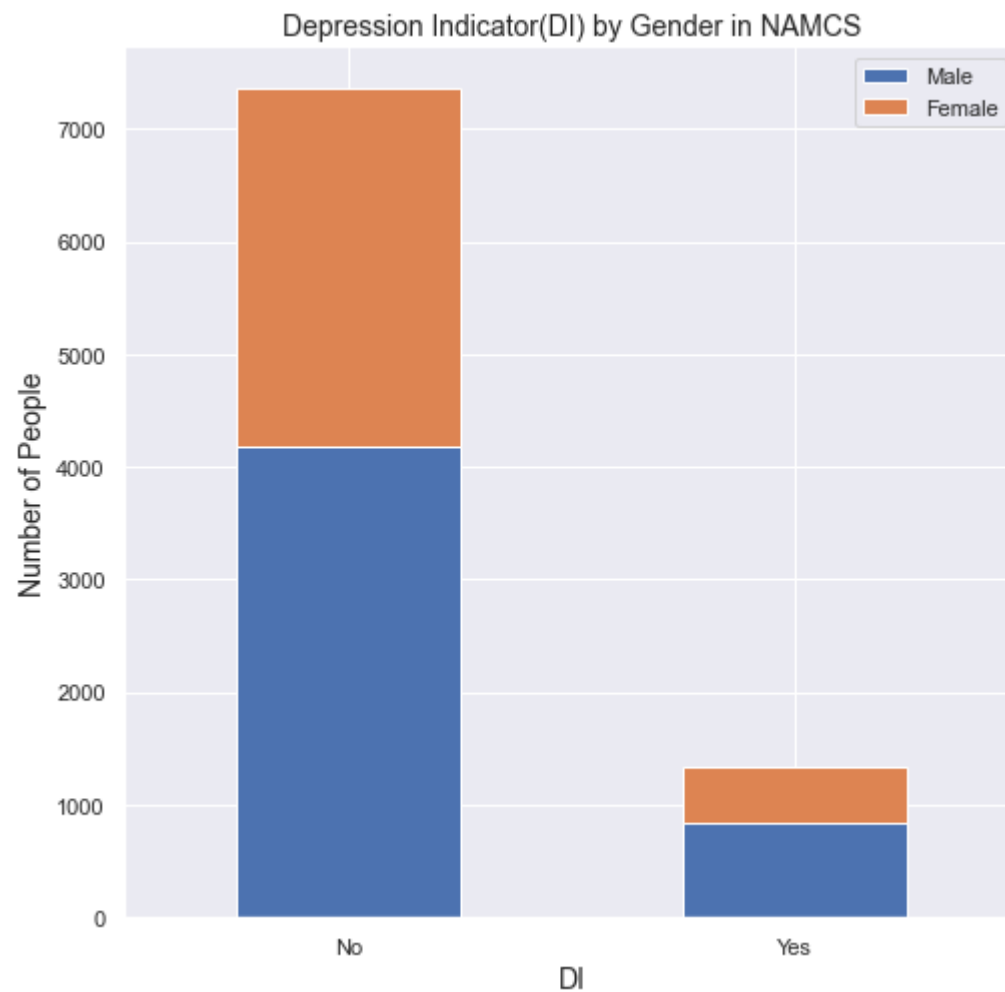
```
In [31]: datanhan = nhanesdf.drop(['SEQN', 'PHQ Score', 'DPRN Level', 'Feels DPRN', 'DPRN Meds', 'DPQ1', 'DPQ2', 'DPQ3', 'DPQ4', 'DPQ5', '
g = sns.heatmap(datanhan.corr(), annot=True, annot_kws={"fontsize":17})
g.set_yticklabels(g.get_yticklabels(), size=25, rotation=0)
g.set_xticklabels(g.get_xticklabels(), size=25, rotation=90)
g.set_title('NHANES Heatmap', size=35)
sns.set(rc = {'figure.figsize':(30,30)})
plt.tight_layout()
plt.savefig('fd NHANES_Heatmap.png', dpi= 1200)
plt.show()
```





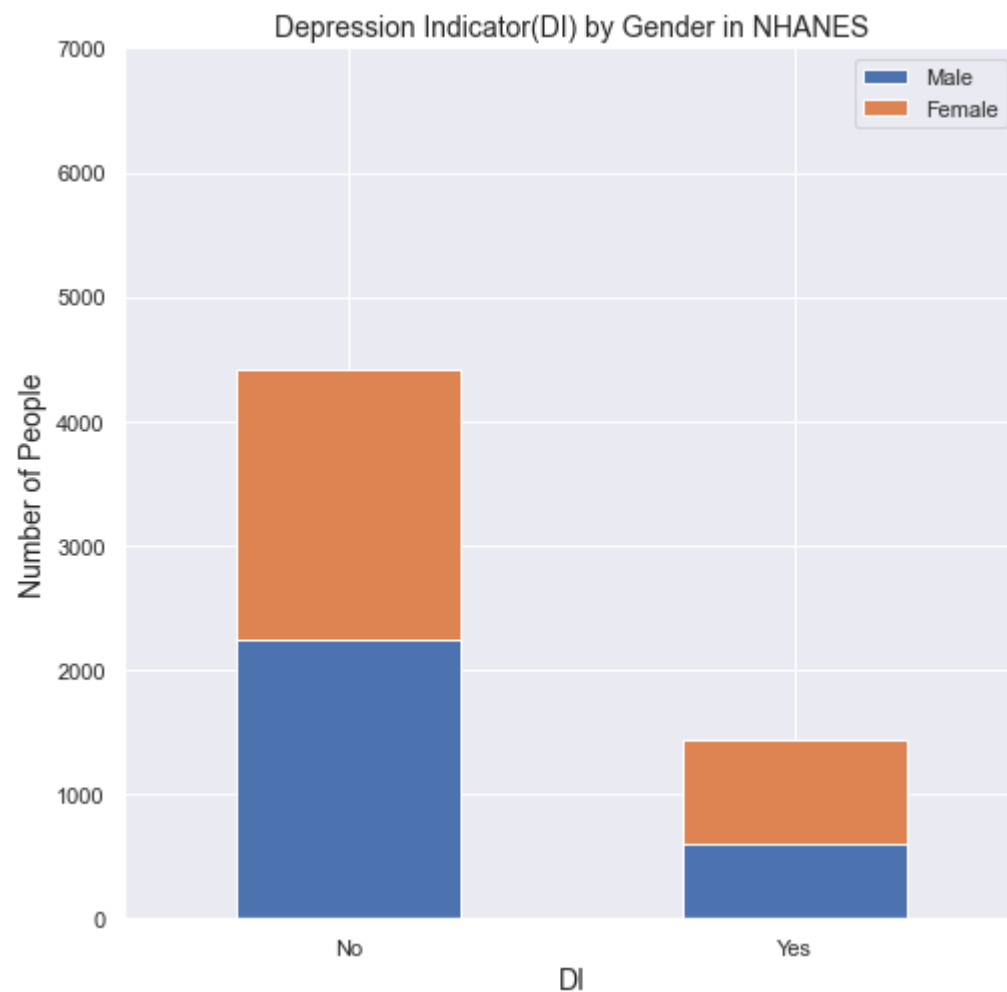
Gender NAMCS

```
In [32]: gendernamcs = pd.crosstab(index=namcsdf["DI"], columns=namcsdf["Gender"])
gendernamcs.plot(kind="bar", figsize=(8,8),stacked=True, legend = True)
plt.xlabel("DI",fontsize=14)
plt.ylabel("Number of People",fontsize=14)
plt.legend(["Male", "Female"])
plt.title("Depression Indicator(DI) by Gender in NAMCS",fontsize=14)
plt.xticks([1, 0], ['Yes', 'No'], rotation=360)
plt.savefig('fd namcs_sex_stack.png', dpi= 1200)
```



Gender NHANES

```
In [33]: gendernhanes = pd.crosstab(index=nhanesdf["DI"], columns=nhanesdf["Gender"])
gendernhanes.plot(kind="bar", figsize=(8,8),stacked=True)
plt.xlabel("DI",fontsize=14)
plt.ylabel("Number of People",fontsize=14)
plt.legend(["Male", "Female"])
plt.title("Depression Indicator(DI) by Gender in NHANES",fontsize=14)
plt.xticks([1, 0], ['Yes', 'No'], rotation=360)
plt.ylim(0,7000)
plt.savefig('fd nhancs_sex_stack.png', dpi= 1200)
```



Co-morbid conditions NAMCS

```
In [34]: cnm1 = pd.crosstab(index=namcsdf["DI"], columns= namcsdf['Infection'])
cnm2 = pd.crosstab(index=namcsdf["DI"], columns= namcsdf['Cancer/Blood'])
cnm3 = pd.crosstab(index=namcsdf["DI"], columns= namcsdf['Metabolic'])
cnm4 = pd.crosstab(index=namcsdf["DI"], columns= namcsdf['Neurologic'])
cnm5 = pd.crosstab(index=namcsdf["DI"], columns= namcsdf['Eyes/Ears'])
cnm6 = pd.crosstab(index=namcsdf["DI"], columns= namcsdf['Heart/Lung'])
cnm7 = pd.crosstab(index=namcsdf["DI"], columns= namcsdf['Digestive'])
cnm8 = pd.crosstab(index=namcsdf["DI"], columns= namcsdf['Skin'])
cnm9 = pd.crosstab(index=namcsdf["DI"], columns= namcsdf['Bones'])
cnm10 = pd.crosstab(index=namcsdf["DI"], columns= namcsdf['Urinary'])
```

```

cnm11 = pd.crosstab(index=namcsdf["DI"], columns= namcsdf['Womens Health'])
cnm12 = pd.crosstab(index=namcsdf["DI"], columns= namcsdf['Genetic'])
cnm13 = pd.crosstab(index=namcsdf["DI"], columns= namcsdf['Other Disorder'])
cnm14 = pd.crosstab(index=namcsdf["DI"], columns= namcsdf['Injury'])
cnm15 = pd.crosstab(index=namcsdf["DI"], columns= namcsdf['Morbidity'])
cnm16 = pd.crosstab(index=namcsdf["DI"], columns= namcsdf['Public health'])

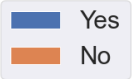
```

```

In [35]: fig, axes = plt.subplots(nrows=4, ncols=4)
cnm1.plot(kind="bar", stacked=True, ax=axes[0,0])
cnm2.plot(kind="bar", stacked=True, ax=axes[0,1])
cnm3.plot(kind="bar", stacked=True, ax=axes[0,2])
cnm4.plot(kind="bar", stacked=True, ax=axes[0,3])
cnm5.plot(kind="bar", stacked=True, ax=axes[1,0])
cnm6.plot(kind="bar", stacked=True, ax=axes[1,1])
cnm7.plot(kind="bar", stacked=True, ax=axes[1,2])
cnm8.plot(kind="bar", stacked=True, ax=axes[1,3])
cnm9.plot(kind="bar", stacked=True, ax=axes[2,0])
cnm10.plot(kind="bar", stacked=True, ax=axes[2,1])
cnm11.plot(kind="bar", stacked=True, ax=axes[2,2])
cnm12.plot(kind="bar", stacked=True, ax=axes[2,3])
cnm13.plot(kind="bar", stacked=True, ax=axes[3,0])
cnm14.plot(kind="bar", stacked=True, ax=axes[3,1])
cnm15.plot(kind="bar", stacked=True, ax=axes[3,2])
cnm16.plot(kind="bar", stacked=True, ax=axes[3,3])
fig.text(0.04, 0.5, 'Number of People', va='center', rotation='vertical', fontsize=30)
fig.text(0.5, 0.04, 'DI', ha='center', fontsize=30)
fig.legend(["Yes", "No"], fontsize= 30)
fig.suptitle("Depression Indicator(DI) vs Co-morbid in NAMCS", fontsize= 50)
plt.setp(axes, xticks=[1, 0], xticklabels= ['Yes', 'No'])
plt.savefig('fd DI vs Comorbid in NAMCS.png', dpi= 1200)

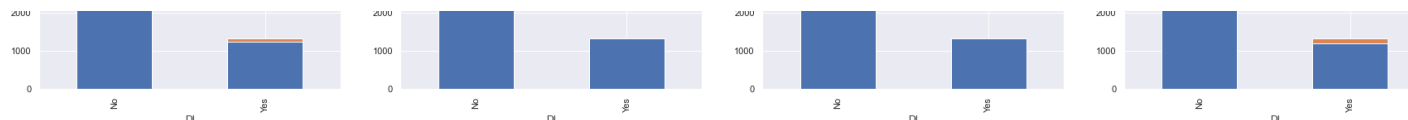
```

Depression Indicator(DI) vs Co-morbid in NAMCS



Number of People





DI

Co-morbid conditions NHANES

```
In [36]: ct1 = pd.crosstab(index=nhanesdf["DI"], columns=nhanesdf["Asthma"])
ct2 = pd.crosstab(index=nhanesdf["DI"], columns=nhanesdf["Overweight"])
ct3 = pd.crosstab(index=nhanesdf["DI"], columns=nhanesdf["Arthritis"])
ct4 = pd.crosstab(index=nhanesdf["DI"], columns=nhanesdf["CHF"])
ct5 = pd.crosstab(index=nhanesdf["DI"], columns=nhanesdf["Vascular"])
ct6 = pd.crosstab(index=nhanesdf["DI"], columns=nhanesdf["Chest Pain"])
ct7 = pd.crosstab(index=nhanesdf["DI"], columns=nhanesdf["Heart attack"])
ct8 = pd.crosstab(index=nhanesdf["DI"], columns=nhanesdf["Stroke"])
ct9 = pd.crosstab(index=nhanesdf["DI"], columns=nhanesdf["Thyroid"])
ct10 = pd.crosstab(index=nhanesdf["DI"], columns=nhanesdf["Emphysema"])
ct11 = pd.crosstab(index=nhanesdf["DI"], columns=nhanesdf["Bronchitis"])
ct12 = pd.crosstab(index=nhanesdf["DI"], columns=nhanesdf["COPD"])
ct13 = pd.crosstab(index=nhanesdf["DI"], columns=nhanesdf["Liver"])
ct14 = pd.crosstab(index=nhanesdf["DI"], columns=nhanesdf["Cancer"])
```

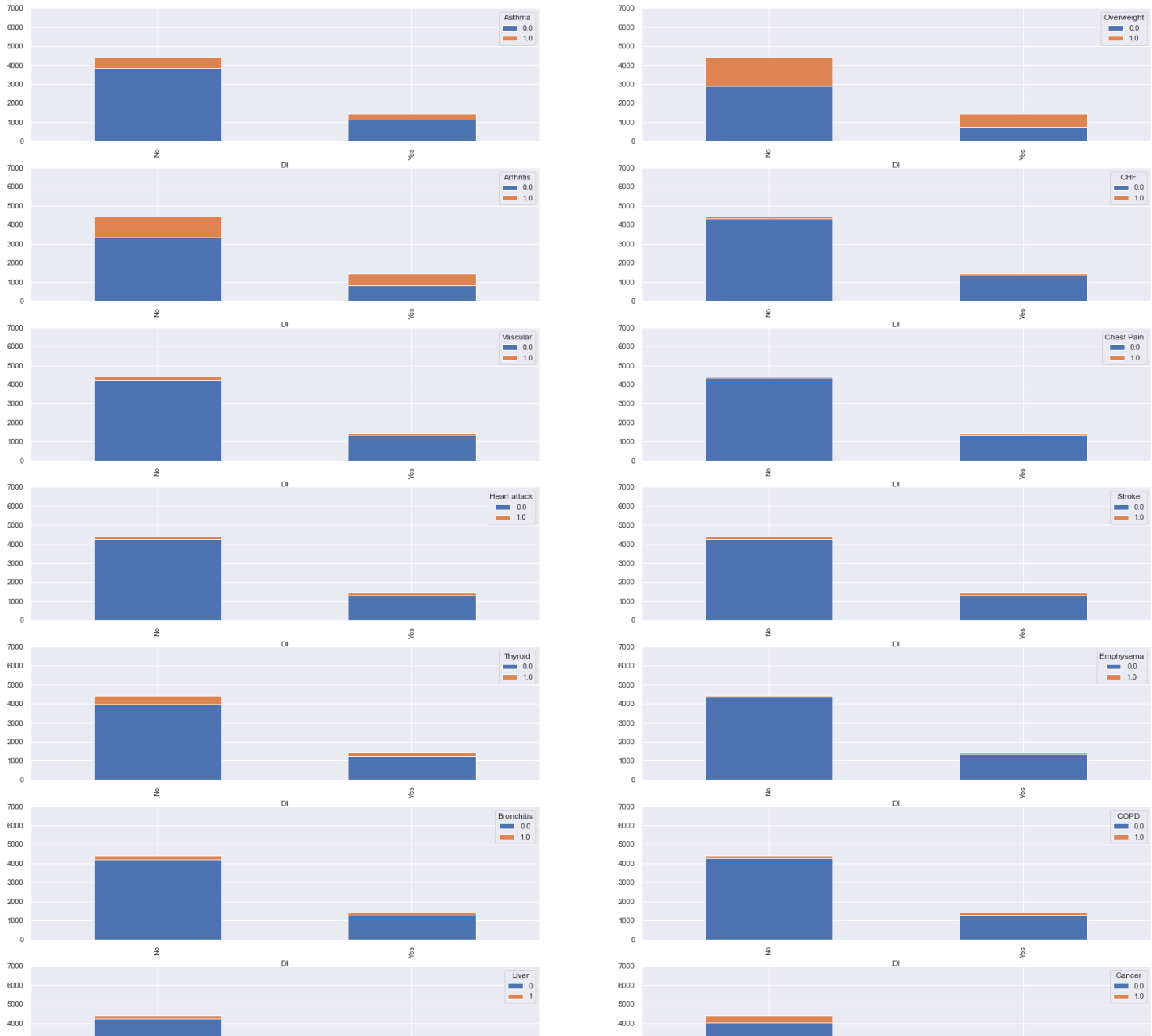
```
In [37]: fig, axes = plt.subplots(nrows=7, ncols=2)
ct1.plot(kind="bar", stacked=True, ax=axes[0,0])
ct2.plot(kind="bar", stacked=True, ax=axes[0,1])
ct3.plot(kind="bar", stacked=True, ax=axes[1,0])
ct4.plot(kind="bar", stacked=True, ax=axes[1,1])
ct5.plot(kind="bar", stacked=True, ax=axes[2,0])
ct6.plot(kind="bar", stacked=True, ax=axes[2,1])
ct7.plot(kind="bar", stacked=True, ax=axes[3,0])
ct8.plot(kind="bar", stacked=True, ax=axes[3,1])
ct9.plot(kind="bar", stacked=True, ax=axes[4,0])
ct10.plot(kind="bar", stacked=True, ax=axes[4,1])
ct11.plot(kind="bar", stacked=True, ax=axes[5,0])
ct12.plot(kind="bar", stacked=True, ax=axes[5,1])
ct13.plot(kind="bar", stacked=True, ax=axes[6,0])
ct14.plot(kind="bar", stacked=True, ax=axes[6,1])
custom_ylim = (0, 7000)
plt.setp(axes, ylim=custom_ylim)
fig.text(0.04, 0.5, 'Number of People', va='center', rotation='vertical', fontsize=30)
fig.text(0.5, 0.04, 'DI', ha='center', fontsize=30)
```

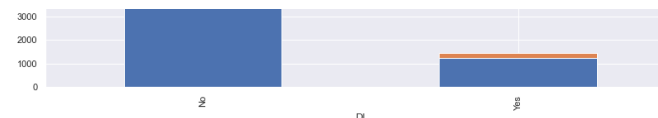
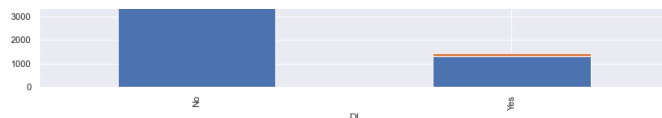
```
fig.legend(["Yes", "No"], fontsize= 30)
fig.suptitle("Depression Indicator(DI) vs Co-morbid in NHANES",fontsize= 50)
plt.setp(axes,xticks=[1, 0], xticklabels= ['Yes', 'No'])
plt.savefig('fd comorbid_stack_nhanes.png', dpi= 1200)
```

Depression Indicator(DI) vs Co-morbid in NHANES



Number of People





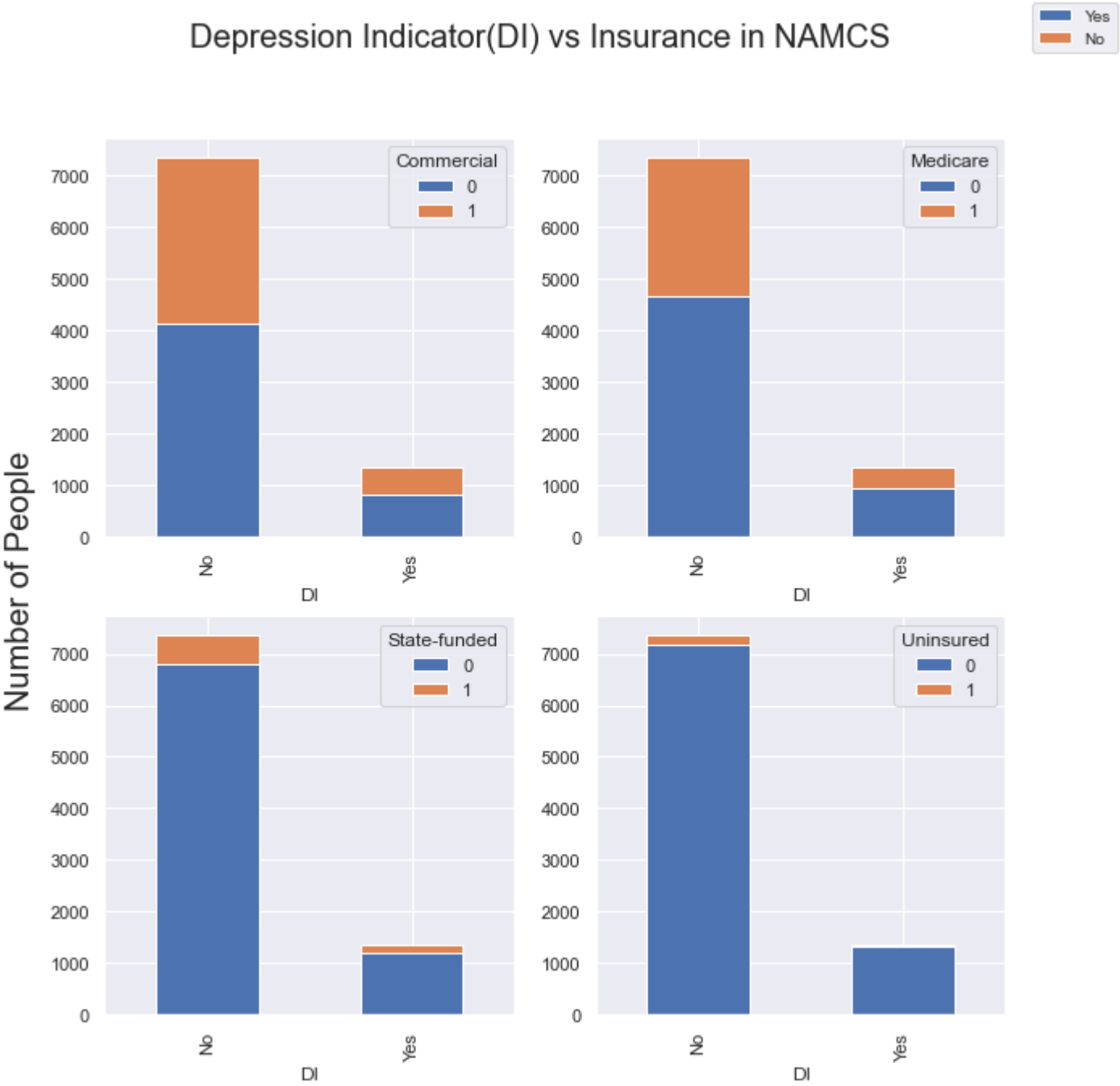
DI

Insurance NAMCS

```
In [38]: insnam1 = pd.crosstab(index=namcsdf["DI"], columns= namcsdf["Commercial"])
insnam2 = pd.crosstab(index=namcsdf["DI"], columns= namcsdf["Medicare"])
insnam3 = pd.crosstab(index=namcsdf["DI"], columns= namcsdf['State-funded'])
insnam4 = pd.crosstab(index=namcsdf["DI"], columns= namcsdf['Uninsured'])

fig, axes = plt.subplots(nrows=2, ncols=2)

insnam1.plot(kind="bar", figsize=(10,10),stacked=True, ax=axes[0,0])
insnam2.plot(kind="bar", figsize=(10,10),stacked=True, ax=axes[0,1])
insnam3.plot(kind="bar", figsize=(10,10),stacked=True, ax=axes[1,0])
insnam4.plot(kind="bar", figsize=(10,10),stacked=True, ax=axes[1,1])
fig.text(0.04, 0.5, 'Number of People', va='center', rotation='vertical', fontsize=20)
fig.legend(["Yes", "No"], fontsize= 10)
fig.suptitle("Depression Indicator(DI) vs Insurance in NAMCS",fontsize= 20)
plt.setp(axes,xticks=[1, 0], xticklabels= ['Yes', 'No'])
plt.savefig('fd DI vs Insurance in NAMCS.png', dpi= 1200)
```

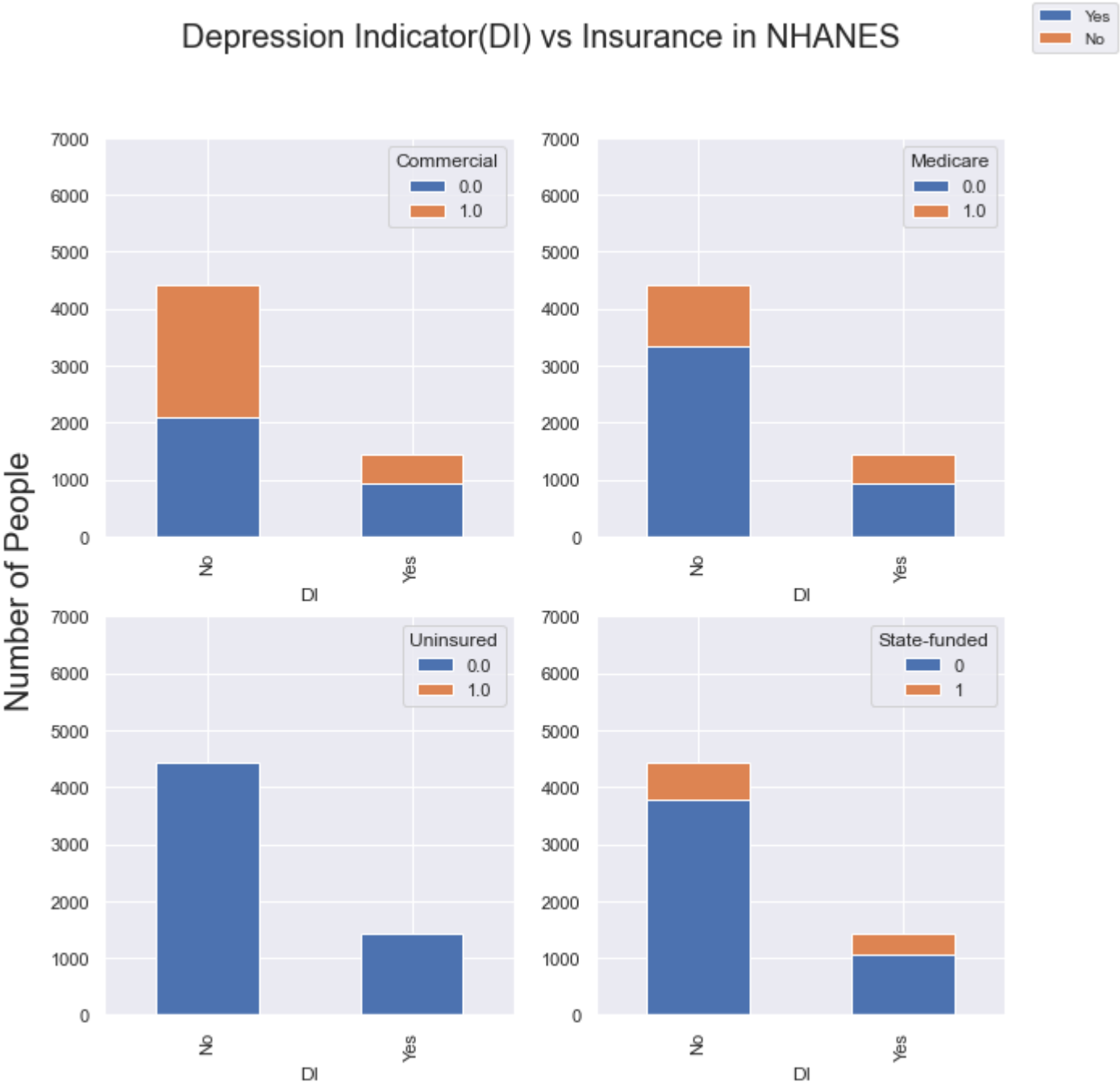


Insurance NHANES

```
In [39]: deprprvt = pd.crosstab(index=nhanesdf["DI"], columns= nhanesdf["Commercial"])
deprcare = pd.crosstab(index=nhanesdf["DI"], columns= nhanesdf["Medicare"])
deprnoch = pd.crosstab(index=nhanesdf["DI"], columns= nhanesdf["Uninsured"])
deprstate = pd.crosstab(index=nhanesdf["DI"], columns= nhanesdf["State-funded"])

fig, axes = plt.subplots(nrows=2, ncols=2)

deprprvt.plot(kind="bar", figsize=(10,10),stacked=True, ax=axes[0,0])
deprcare.plot(kind="bar", figsize=(10,10),stacked=True, ax=axes[0,1])
deprnoch.plot(kind="bar", figsize=(10,10),stacked=True, ax=axes[1,0])
deprstate.plot(kind="bar", figsize=(10,10),stacked=True, ax=axes[1,1])
custom_ylim = (0, 7000)
plt.setp(axes, ylim=custom_ylim)
fig.text(0.04, 0.5, 'Number of People', va='center', rotation='vertical', fontsize=20)
fig.legend(["Yes", "No"], fontsize= 10)
fig.suptitle("Depression Indicator(DI) vs Insurance in NHANES", fontsize= 20)
plt.setp(axes,xticks=[1, 0], xticklabels= ['Yes', 'No'])
plt.savefig('fd Insurance in NHANES.png', dpi= 1200)
```



Provider NAMCS

```

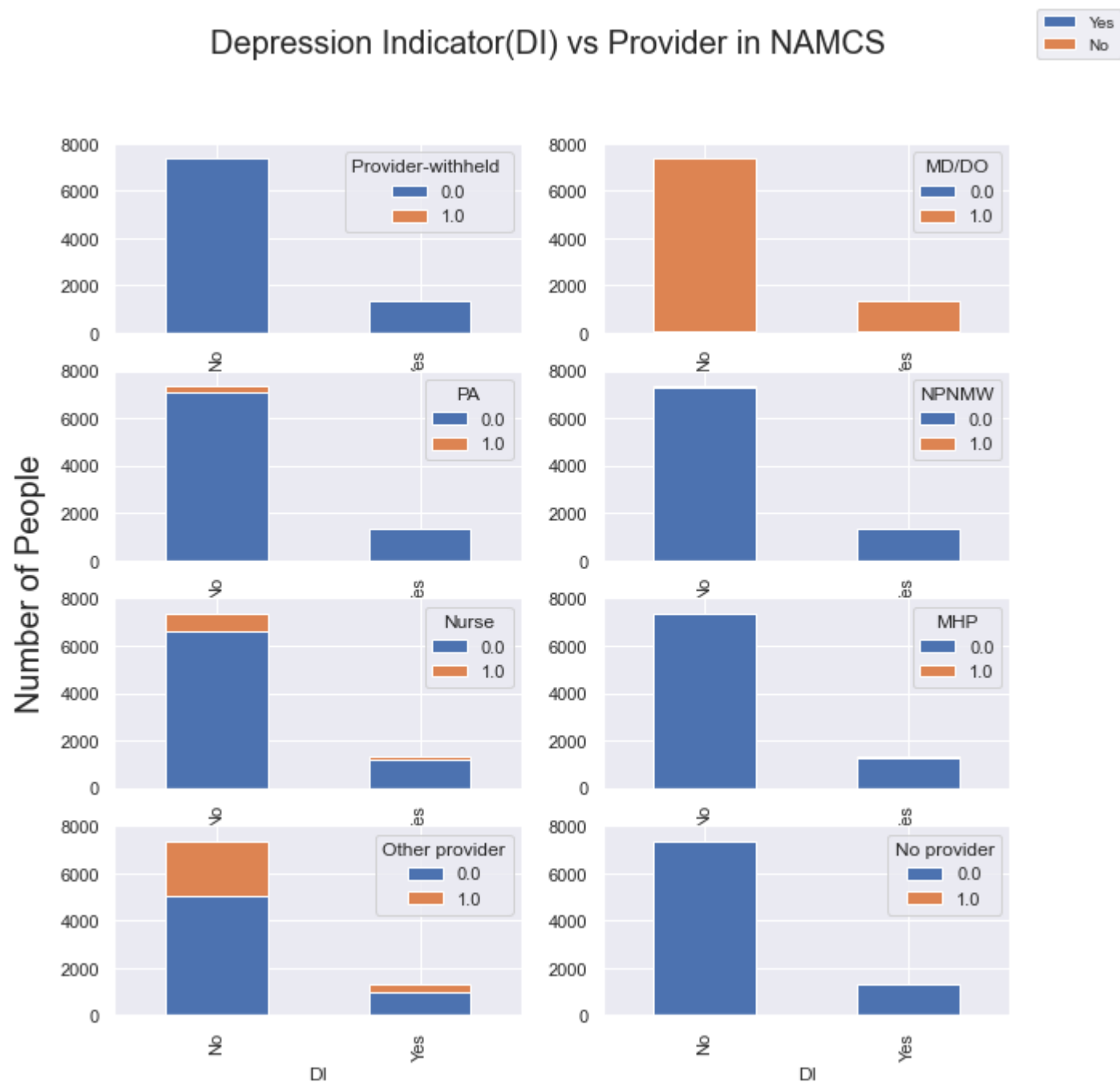
In [40]: pr1 = pd.crosstab(index=namcsdf["DI"], columns= namcsdf["Provider-withheld "])
pr2 = pd.crosstab(index=namcsdf["DI"], columns= namcsdf["MD/DO"])
pr3 = pd.crosstab(index=namcsdf["DI"], columns= namcsdf['PA'])
pr4 = pd.crosstab(index=namcsdf["DI"], columns= namcsdf['NPNMW'])
pr5 = pd.crosstab(index=namcsdf["DI"], columns= namcsdf['Nurse'])
pr6 = pd.crosstab(index=namcsdf["DI"], columns= namcsdf['MHP'])
pr7 = pd.crosstab(index=namcsdf["DI"], columns= namcsdf['Other provider'])
pr8 = pd.crosstab(index=namcsdf["DI"], columns= namcsdf['No provider'])

fig, axes = plt.subplots(nrows=4, ncols=2)

pr1.plot(kind="bar", figsize=(10,10),stacked=True, ax=axes[0,0])
pr2.plot(kind="bar", figsize=(10,10),stacked=True, ax=axes[0,1])
pr3.plot(kind="bar", figsize=(10,10),stacked=True, ax=axes[1,0])
pr4.plot(kind="bar", figsize=(10,10),stacked=True, ax=axes[1,1])
pr5.plot(kind="bar", figsize=(10,10),stacked=True, ax=axes[2,0])
pr6.plot(kind="bar", figsize=(10,10),stacked=True, ax=axes[2,1])
pr7.plot(kind="bar", figsize=(10,10),stacked=True, ax=axes[3,0])
pr8.plot(kind="bar", figsize=(10,10),stacked=True, ax=axes[3,1])

custom_ylim = (0, 8000)
plt.setp(axes, ylim=custom_ylim)
fig.text(0.04, 0.5, 'Number of People', va='center', rotation='vertical', fontsize=20)
fig.legend(["Yes", "No"], fontsize= 10)
fig.suptitle("Depression Indicator(DI) vs Provider in NAMCS", fontsize= 20)
plt.setp(axes,xticks=[1, 0], xticklabels= ['Yes', 'No'])
plt.savefig('fd DI vs PROVIDER in NAMCS.png', dpi= 1200)

```

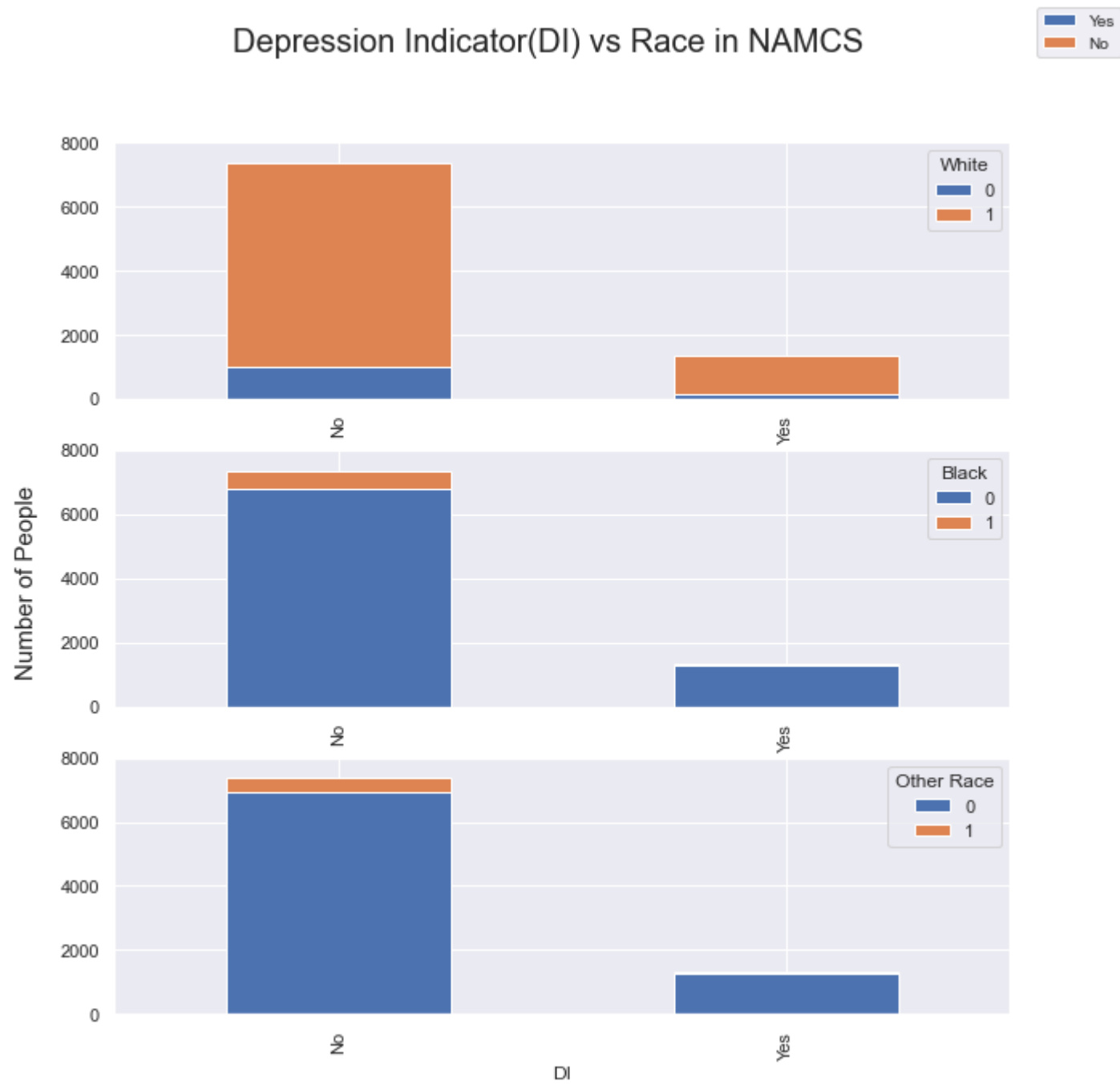


```
In [41]: depracewh = pd.crosstab(index=namcsdf["DI"], columns= namcsdf["White"])
depracebl = pd.crosstab(index=namcsdf["DI"], columns=namcsdf["Black"])
```

```
depraceot = pd.crosstab(index=namcsdf["DI"], columns= namcsdf["Other Race"])
deprage = pd.crosstab(index=namcsdf["DI"], columns= namcsdf["Age"])

fig, axes = plt.subplots(nrows=3, ncols=1)

depracewh.plot(kind="bar", figsize=(10,10), stacked=True, ax=axes[0])
depracebl.plot(kind="bar", figsize=(10,10), stacked=True, ax=axes[1])
depraceot.plot(kind="bar", figsize=(10,10), stacked=True, ax=axes[2])
custom_ylim = (0, 8000)
plt.setp(axes, ylim=custom_ylim)
fig.text(0.04, 0.5, 'Number of People', va='center', rotation='vertical', fontsize=15)
fig.legend(["Yes", "No"], fontsize= 10)
fig.suptitle("Depression Indicator(DI) vs Race in NAMCS", fontsize= 20)
plt.setp(axes, xticks=[1, 0], xticklabels= ['Yes', 'No'])
plt.savefig('fd RaceF in NAMCS.png', dpi= 1200)
```

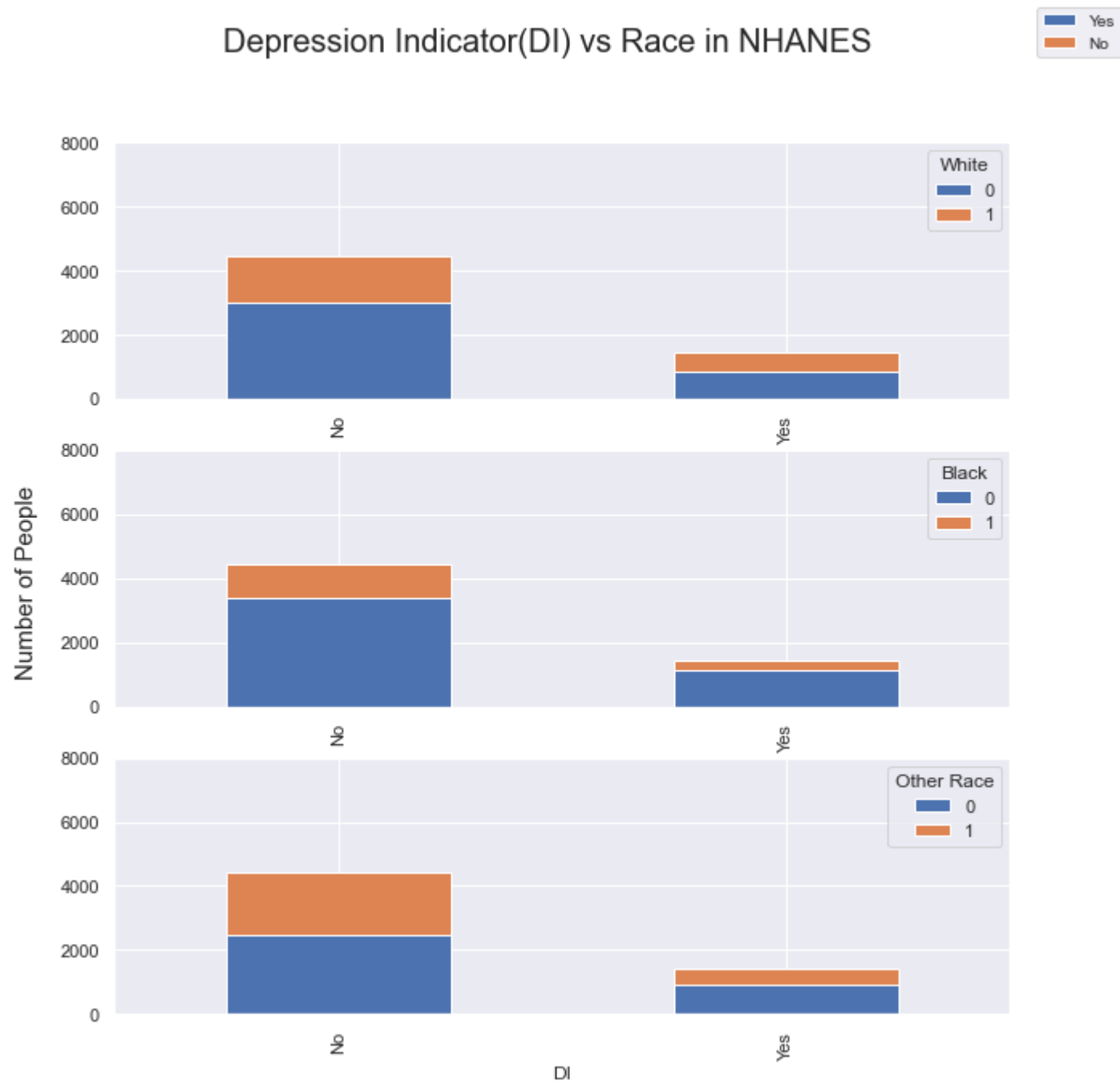


```
In [42]: depracewh = pd.crosstab(index=nhanesdf["DI"], columns= nhanesdf["White"])
depracebl = pd.crosstab(index=nhanesdf["DI"], columns=nhanesdf["Black"])
```

```
depraceot = pd.crosstab(index=nhanesdf["DI"],columns= nhanesdf["Other Race"])

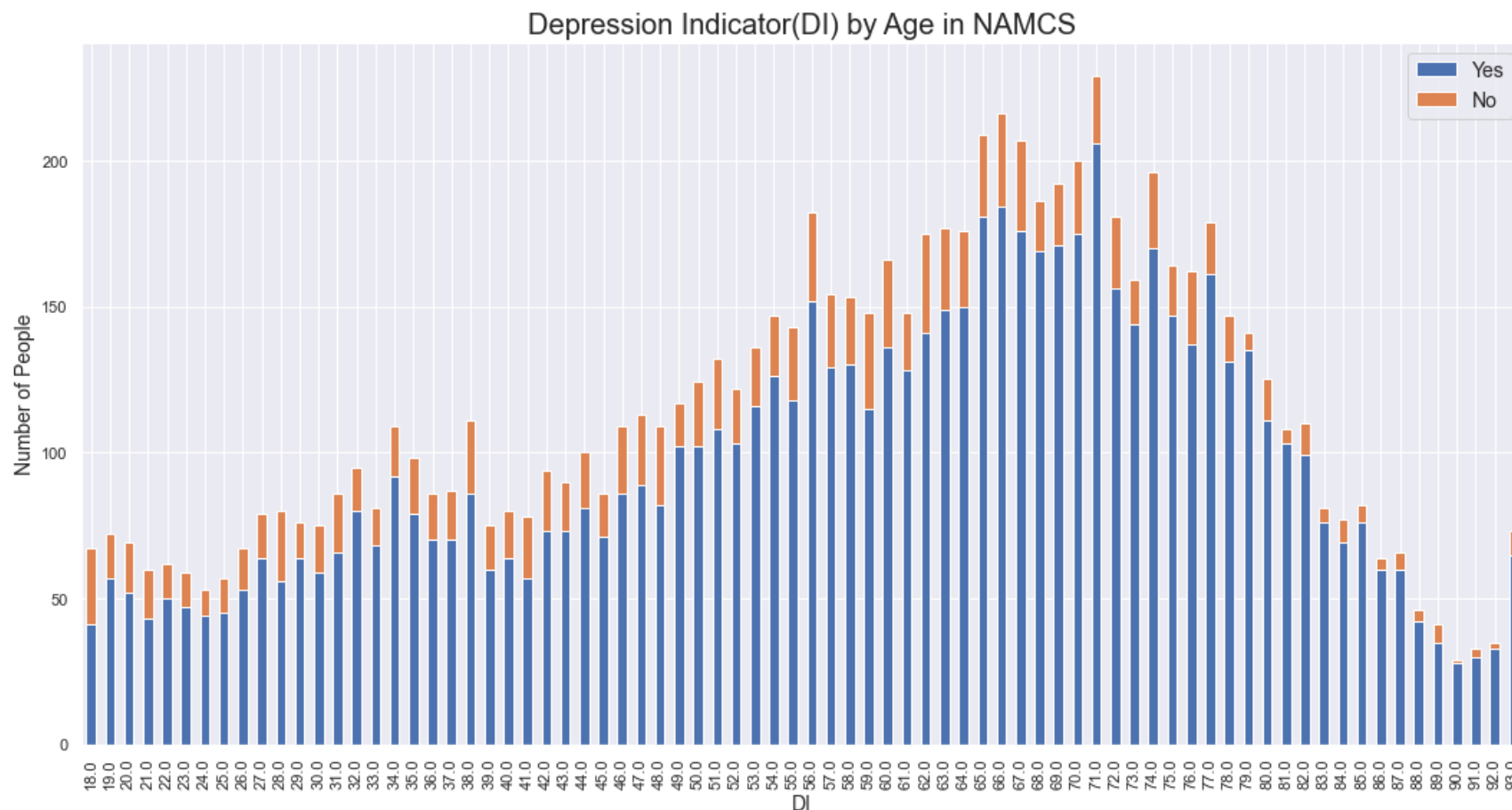
fig, axes = plt.subplots(nrows=3, ncols=1)

depracewh.plot(kind="bar", figsize=(10,10),stacked=True, ax=axes[0])
depracebl.plot(kind="bar", figsize=(10,10),stacked=True, ax=axes[1])
depraceot.plot(kind="bar", figsize=(10,10),stacked=True, ax=axes[2])
fig.text(0.04, 0.5, 'Number of People', va='center', rotation='vertical', fontsize=15)
fig.legend(["Yes", "No"], fontsize= 10)
fig.suptitle("Depression Indicator(DI) vs Race in NHANES",fontsize= 20)
plt.setp(axes,xticks=[1, 0], xticklabels= ['Yes', 'No'])
custom_ylim = (0, 8000)
plt.setp(axes, ylim=custom_ylim)
plt.savefig('fd Race in NHANES.png', dpi= 1000)
```

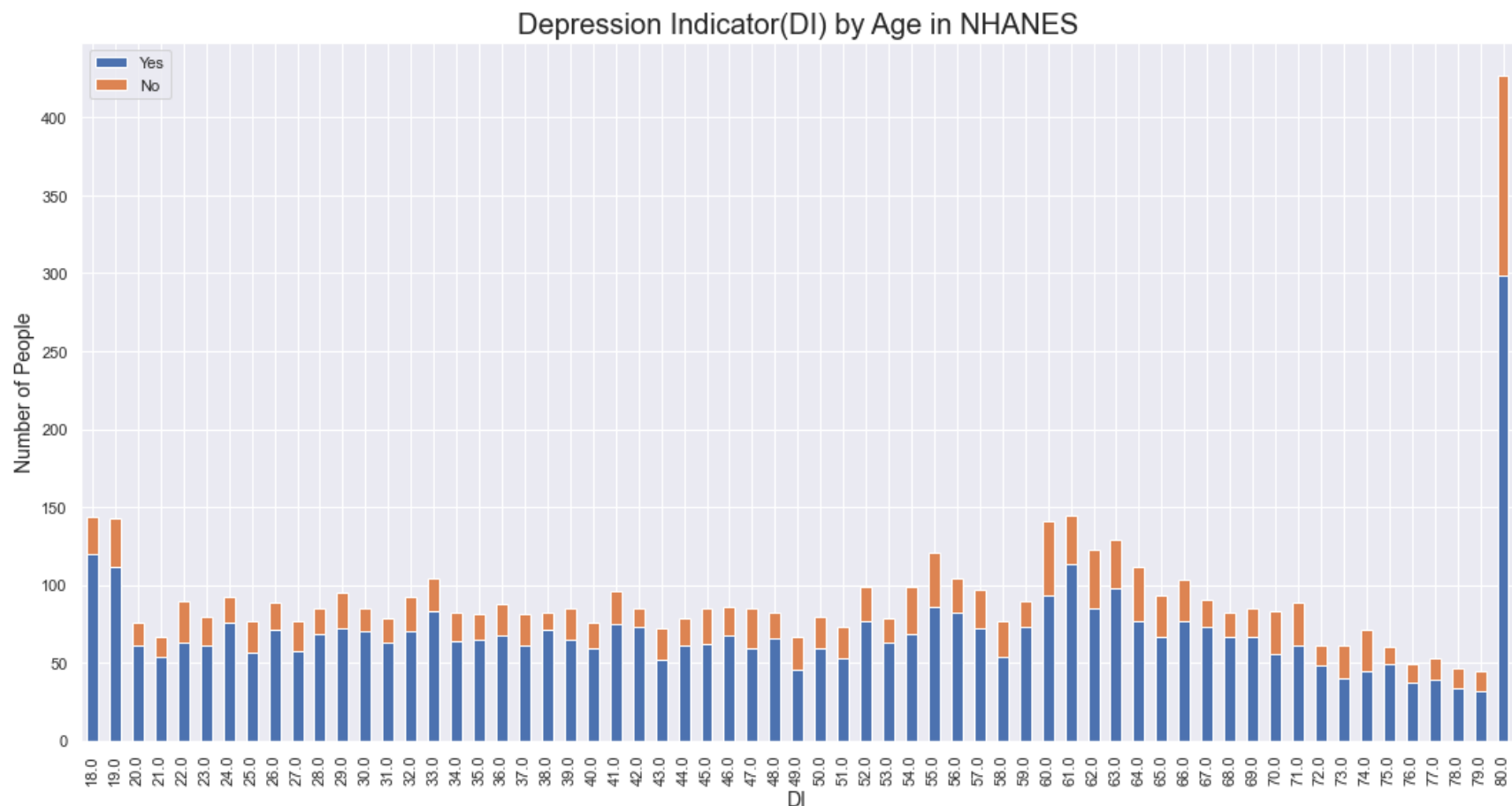


```
In [43]: deprage.T.plot(kind="bar",figsize=(18,9),stacked=True,legend=True)
plt.xlabel("DI",fontsize=14)
```

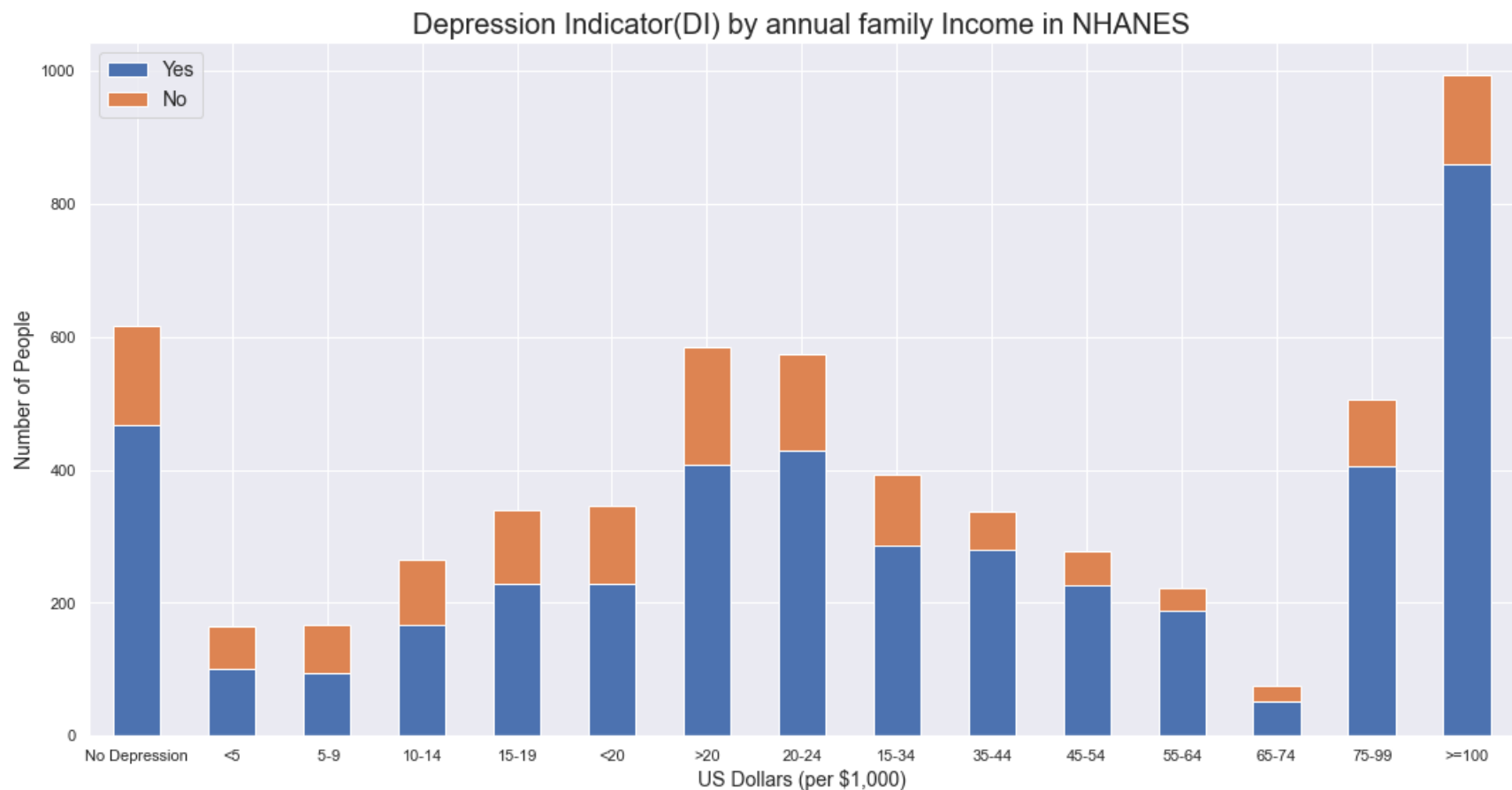
```
plt.ylabel("Number of People",fontsize=14)
plt.legend(["Yes", "No"],fontsize=14)
plt.title("Depression Indicator(DI) by Age in NAMCS", fontsize= 20)
plt.savefig('fd Age in NAMCS.png', dpi= 1200)
```



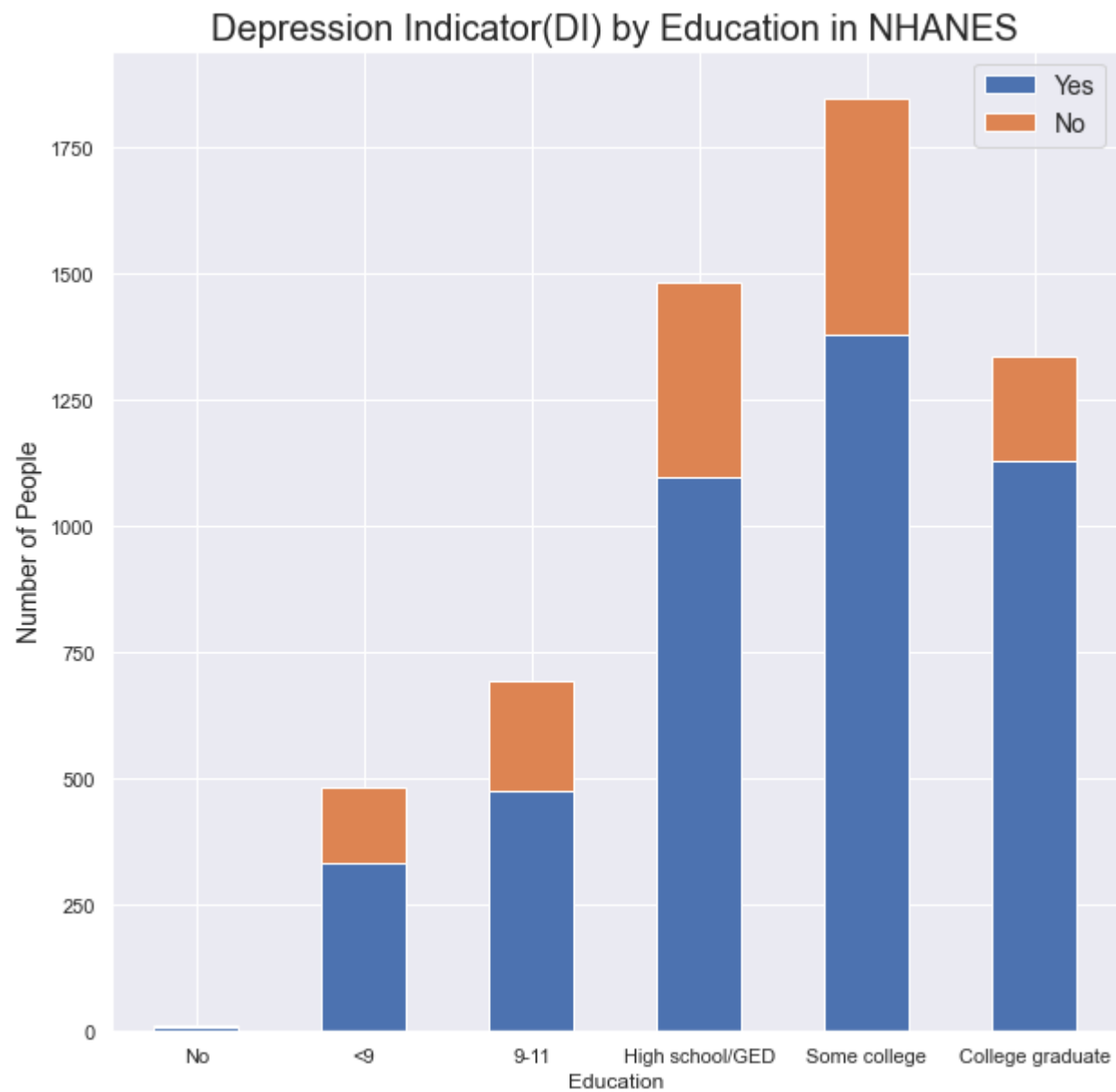
```
In [44]: deprage = pd.crosstab(index=nhanesdf["DI"],columns= nhanesdf["Age"])
deprage.T.plot(kind="bar",figsize=(18,9),stacked=True,legend=True)
plt.xlabel("DI",fontsize=14)
plt.ylabel("Number of People",fontsize=14)
plt.legend(["Yes", "No"])
plt.title("Depression Indicator(DI) by Age in NHANES", fontsize= 20)
plt.savefig('fd Age in NHANES.png', dpi= 1200)
```



```
In [45]: deprincome = pd.crosstab(index=nhanesdf["DI"], columns= nhanesdf["Income"])
deprincome.T.plot(kind="bar", figsize=(18,9), stacked=True, legend=True)
plt.xlabel("US Dollars (per $1,000)", fontsize=14)
plt.ylabel("Number of People", fontsize=14)
plt.legend(["Yes", "No"], fontsize=14)
plt.xticks([0,1,2,3,4,5,6,7,8,9,10,11,12,13,14], ['No Depression', '<5', ' 5-9', '10-14', '15-19', '<20' , '>20', '20-24'])
plt.title("Depression Indicator(DI) by annual family Income in NHANES", fontsize= 20)
plt.savefig('fd Income in NHANES.png', dpi= 1200)
```

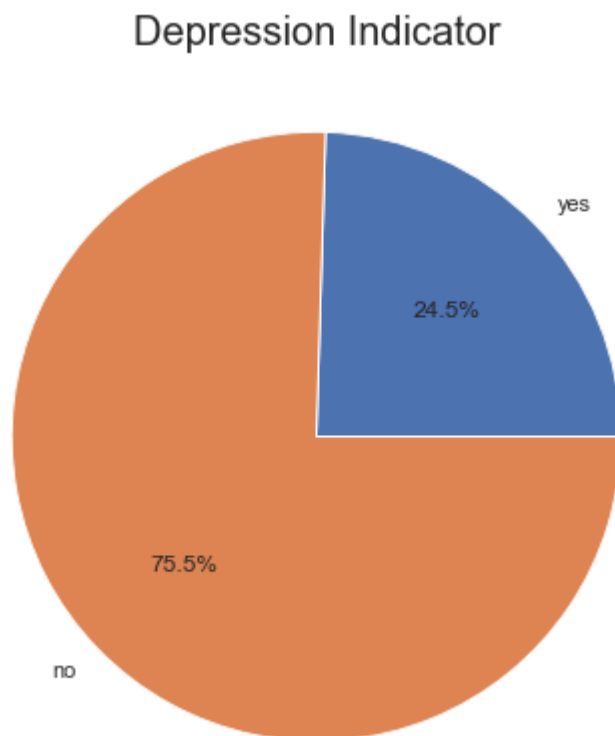


```
In [46]: depredu = pd.crosstab(index=nhanesdf["DI"], columns= nhanesdf["Education"])
fig, axes
depredu.T.plot(kind="bar", figsize=(10,10), stacked=True, legend=True)
plt.ylabel("Number of People", fontsize=14)
plt.legend(["Yes", "No"], fontsize=14)
plt.title("Depression Indicator(DI) by Education in NHANES", fontsize= 20)
plt.xticks([0, 1, 2, 3, 4, 5], ['No', '<9', '9-11', 'High school/GED', 'Some college', 'College graduate'], rotation=360)
plt.savefig(' fd DI vs Educ in NHANES.png', dpi= 1200)
```



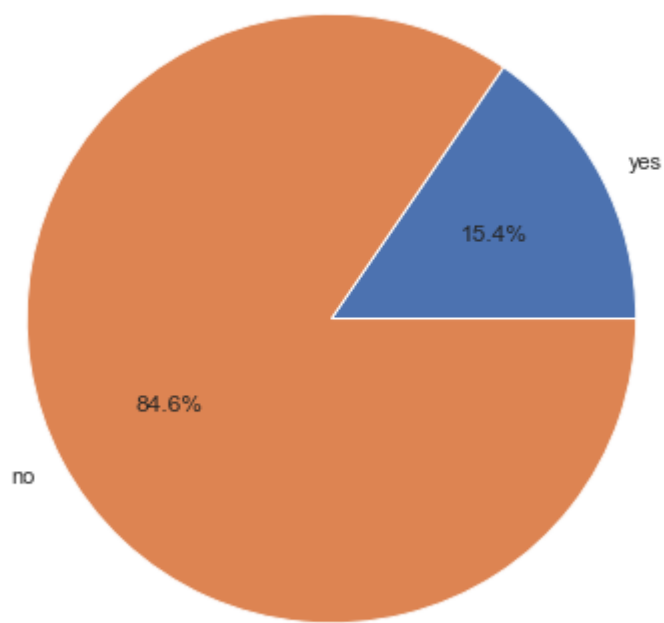
```
In [47]: ones=nhanesdf['DI'].sum()
zeros = nhanesdf.shape[0]-ones
size=[ones,zeros]
lab = ['yes','no']
fig = plt.figure(figsize=(10, 7))
plt.pie(size,labels=lab,autopct='%1.1f%%')
```

```
plt.title('Depression Indicator', fontsize=20)  
plt.savefig('fd DI in NHANES.png', dpi= 1200)  
plt.show()
```

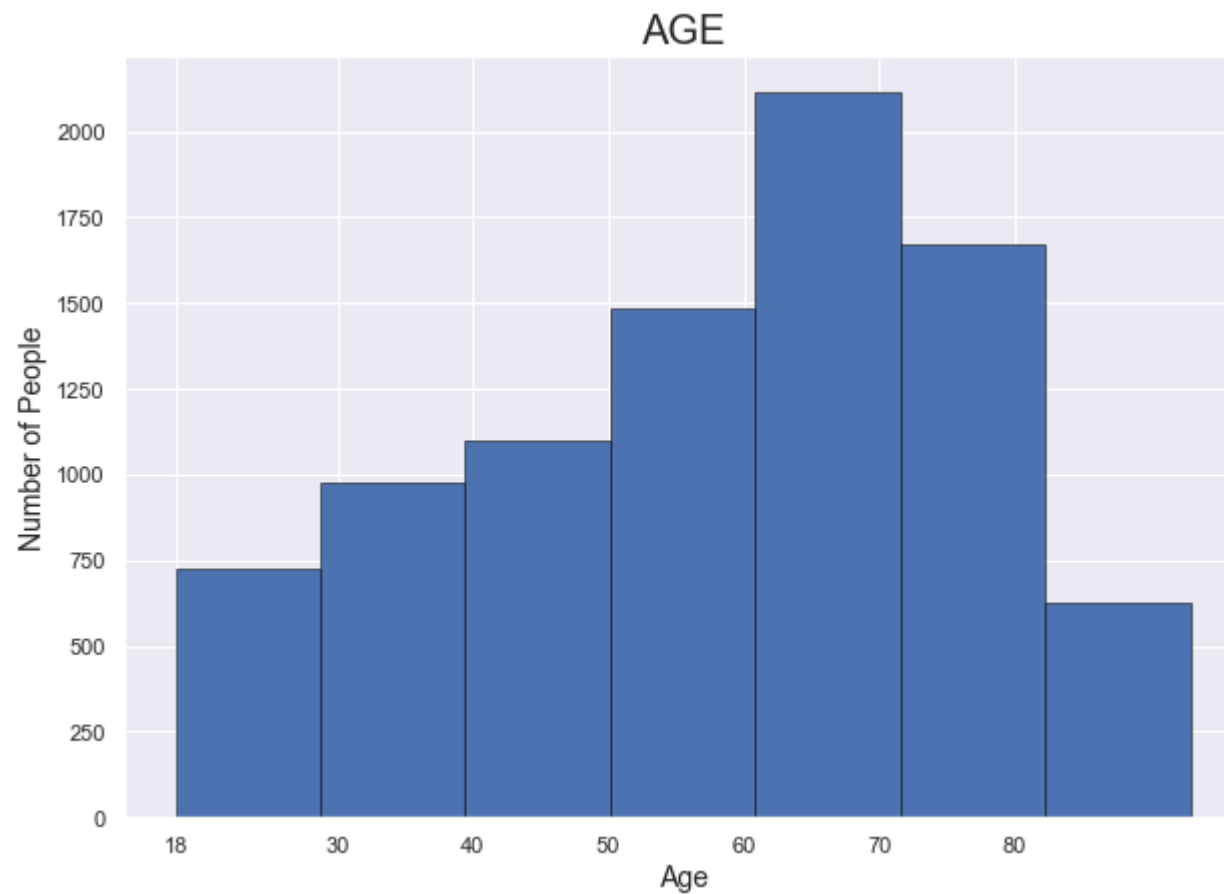


```
In [48]: ones=namcsdf['DI'].sum()  
zeros = namcsdf.shape[0]-ones  
size=[ones,zeros]  
lab = ['yes','no']  
fig = plt.figure(figsize =(10, 7))  
plt.pie(size,labels=lab,autopct='%1.1f%%')  
plt.title('Depression Indicator', fontsize=20)  
plt.savefig('fd DI in NAMCS.png', dpi= 1200)  
plt.show()
```

Depression Indicator



```
In [49]: fig=plt.figure(figsize =(10, 7))
plt.hist(namcsdf['Age'], edgecolor='black', linewidth=0.5, bins=7)
plt.title("AGE",fontsize=20)
plt.xlabel("Age",fontsize=14)
plt.ylabel("Number of People",fontsize=14)
plt.xticks([18, 30, 40, 50, 60, 70, 80])
plt.savefig('fd only Age in NAMCS.png', dpi= 1200)
```



```
In [50]: fig=plt.figure(figsize =(10, 7))
plt.hist(nhanesdf['Age'], edgecolor='black', linewidth=0.5, bins=7)
plt.title("AGE",fontsize=20)
plt.xlabel("Age",fontsize=14)
plt.ylabel("Number of People",fontsize=14)
plt.xticks([18, 30, 40, 50, 60, 70, 80])
plt.savefig('fd only Age in NHANES.png', dpi= 1200)
```

